AWS · CLOUD · DEVOPS PROJECT

Deploy a Three-Tier Web Application on AWS

Highly Available · Highly Secured · Highly Scalable · Fault Tolerant

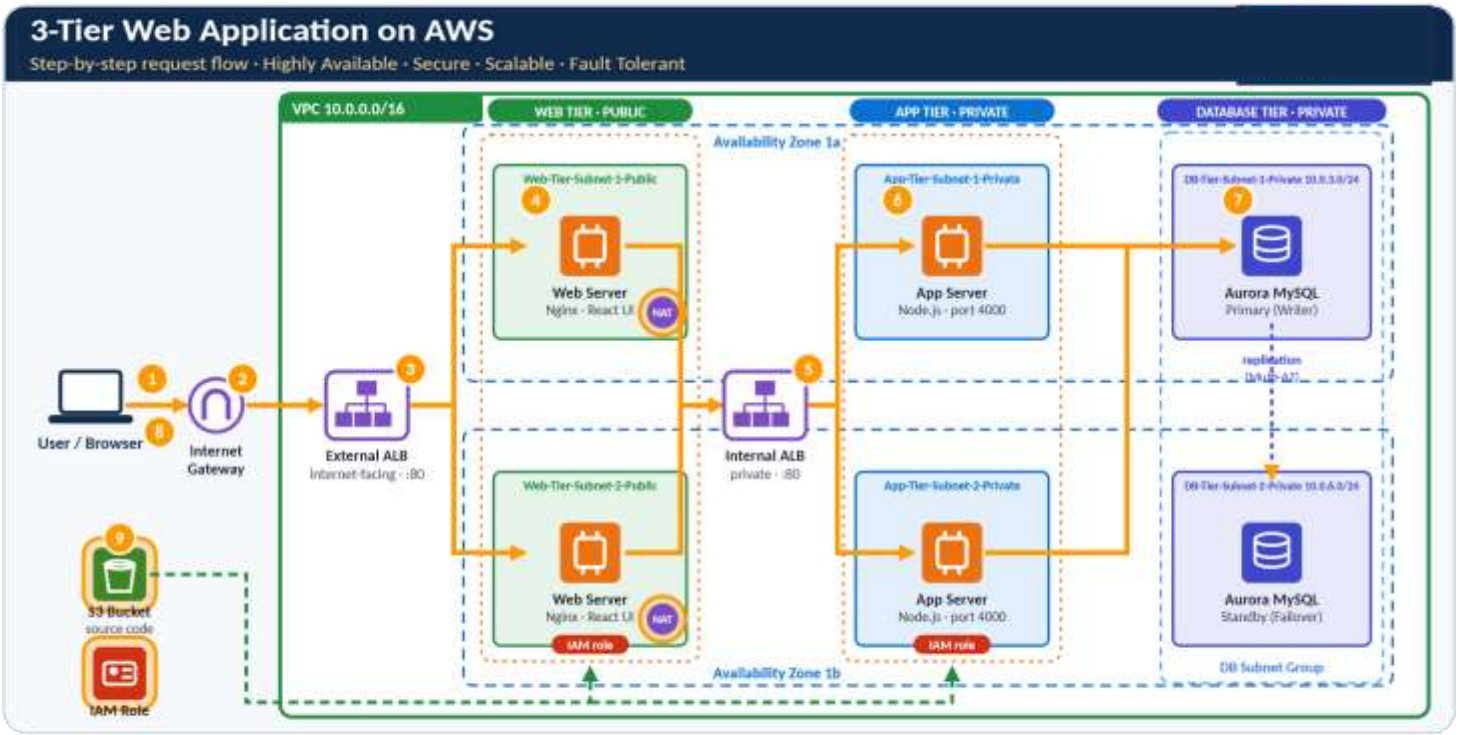
Vinit Shimpi

Associate Cloud Engineer · AWS Certified Cloud Practitioner

GitHub: Vinit Shimpi

Join Me On Linkedin

Architecture at a glance · follow the numbered flow (animated version: 3tier_architecture_flow.gif)



1 User sends a request

A user opens the app URL in a browser (HTTP, port 80).

2 Internet Gateway

The VPC's front door; the Web-Tier route table sends 0.0.0.0/0 to the IGW.

3 External ALB

Internet-facing load balancer spreads requests across both AZs.

4 Web tier (Nginx)

Public-subnet EC2 servers in an Auto Scaling Group serve the UI.

5 **Internal ALBApp tier (Node.js)**
A private load balancer routes API calls to the app

6
tier.Private-subnet EC2 servers run the logic on port 4000.

7 **Database tier (Aurora)Response returns**
Reads/writes on port 3306; Multi-AZ standby for failover.Data

8
flows back DB → App → ALB → Web → ALB → IGW → user.

Project at a Glance

REGION **Mumbai (ap-south-1)**	VPC CIDR **10.0.0.0/16**	AVAILABILITY ZONES **2 (1a and 1b)**
SUBNETS **6 (2 public, 4 private)**	NAT GATEWAYS **2 (one per AZ)**	ROUTE TABLES **3**
SECURITY GROUPS **5 (chained tier by tier)**	DATABASE **Amazon Aurora (MySQL compatible)**	WEB TIER **Nginx on EC2 + Auto Scaling**
APP TIER **Node.js 16 + PM2, port 4000**	LOAD BALANCERS **1 external + 1 internal ALB**	AWS COMPONENTS **14 services**

Table of Contents

Introduction 3

Project Overview 3

Project Implementation 5

1. Create VPC and Subnets 5

2. Create Internet Gateway, NAT Gateways and Route Tables 10

3. Create Security Groups 22

4. Create Database Subnet Group and RDS 26

5. Create IAM Role 32

6. Create an S3 Bucket and Upload the Source Code 34

7. Create Test App Server and Set It Up 39

8. Create AMI of the App Server 48

9. Create Target Group, Launch Template and Internal Load Balancer for App Tier 49

10. Create Test Web Server and Set It Up 57

11. Create AMI of the Web Server 62

12. Create Target Group, Launch Template and External Load Balancer for Web Tier 63

13. Create Auto Scaling Groups 72

14. Access the Application 82

15. Cleaning the Infrastructure 84

Introduction

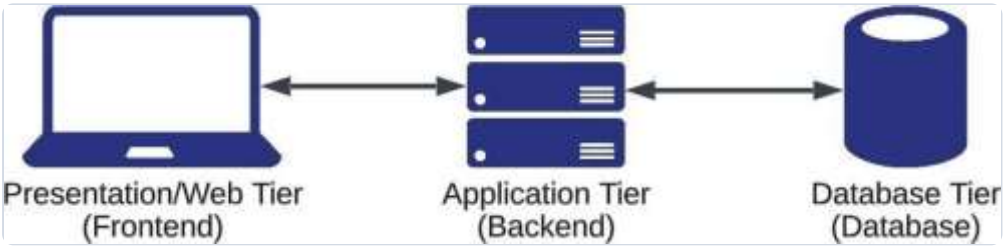
The three-tier web application architecture is a robust and scalable design pattern that separates an application into three distinct layers: the presentation/web tier, the application tier, and the database tier. This architecture enhances security, manageability, and flexibility by isolating each layer's responsibilities.

In this documentation, let’s see how to deploy a three-tier web application on AWS.

A three-tier architecture is a common architectural pattern used in software development and deployment, particularly in cloud environments like AWS. As I mentioned before, it separates an application into three main layers or tiers: the presentation/Web tier, the application tier, and the database tier.

- **Presentation/Web Tier (Frontend)** - This is the user interface layer where users interact with the application. It's responsible for presenting information to the user and collecting user inputs.
- **Application Tier (Backend)** - This is the logic layer where the actual application logic, processing, and control flow take place. It processes user inputs, performs calculations, and makes decisions.
- **Database Tier (Database)** - This layer is responsible for data storage and management. It stores, retrieves, and manages data that the application tier needs.

Users interact with the application via the presentation tier. These requests are sent to the application tier, where the logic processes the request. The application tier interacts with the database tier to retrieve or store data. The processed data or output is sent back to the presentation tier, which displays the results to the user.



Project Overview

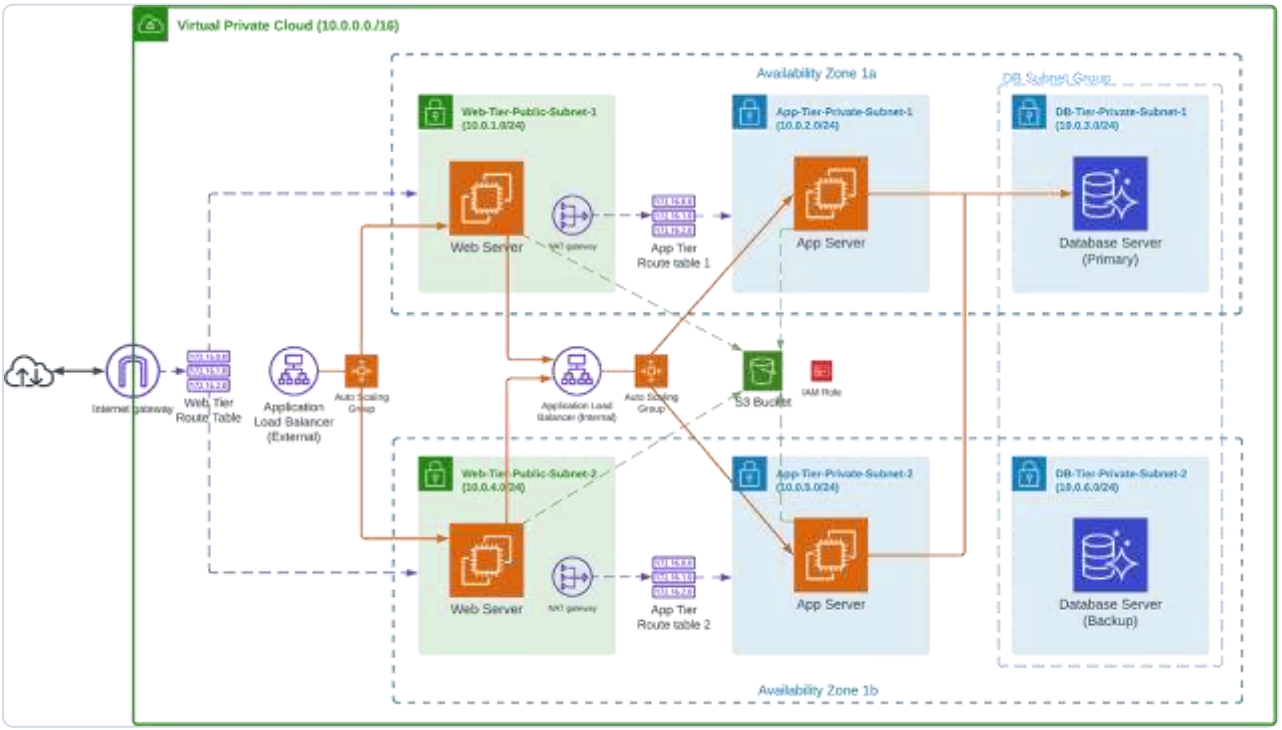
In this project we are implementing a **Highly Available, Highly Secured, Highly Scalable** and **Fault tolerant** three tier application on AWS.

Throughout this project, we have to interact with the following components on AWS.

- Amazon Elastic Compute Cloud (EC2)
- Amazon Simple Storage Service (S3)
- Virtual Private Cloud (VPC)
- Auto Scaling Group (ASG)
- Application Load Balancer (ALB)
- Identity and Access Management (IAM)
- Amazon Relational Database Service (RDS)
  Amazon Machine Image (AMI)
- Launch Templates
- Target Groups
- Internet Gateway (IGW)
- NAT Gateway
- Route Table
- Security Groups



The following diagram shows the overview of the project. It shows connection between each component and how they are distributed in two availability zones Let's have a look at architecture diagram



As you can see, we use public subnets for web tier because it should be open to the internet to let users interact with the application. App tier and Database tier should be secured. Therefore, private subnets are assigned for them. Resources are distributed in two availability zones, and all are included in our custom VPC. Observe how CIDR blocks are assigned for each subnet.

An S3 bucket is used to store the source code of the application. It makes the server configuration process easier.

Source code: https://github.com/sukhada30github/Three-Tier-Project.gihttps://github.com/dhruvsonwane13-ship-it/Three-tier-project.git

Let’s get started with the implementation.

Project Implementation

Go to the **AWS Console** and **choose a region** you like (I chose the Mumbai region) and create the following services as shown in the figures. To reach each service, use the search bar and search for it.

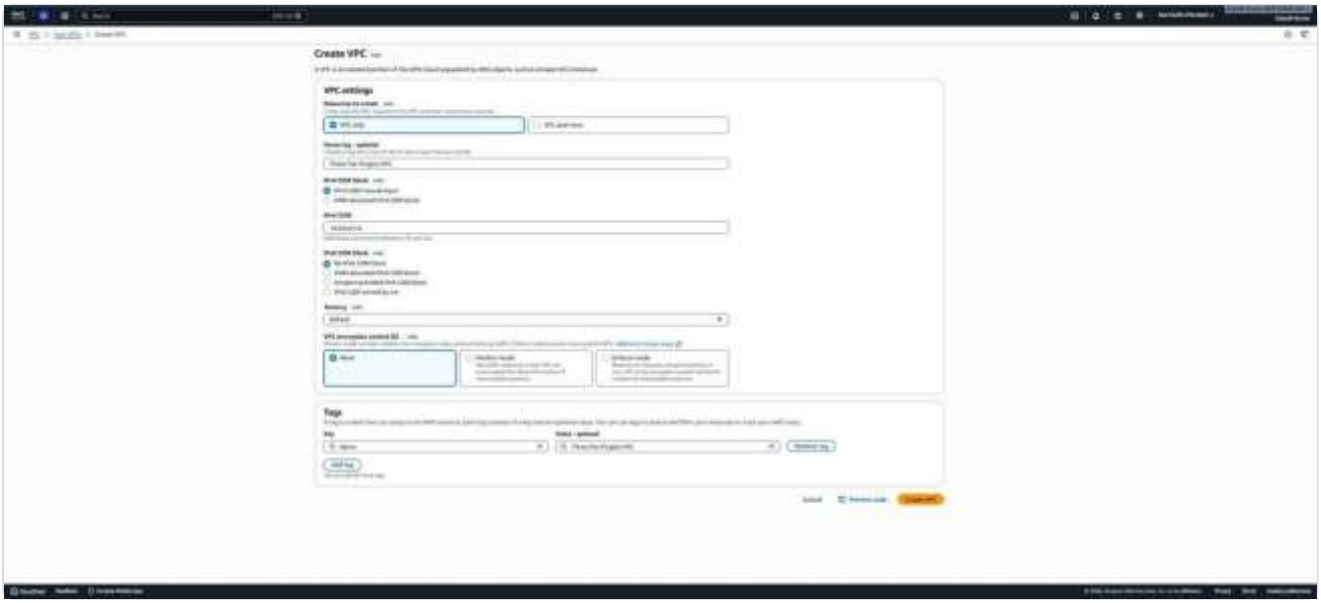


> **Note:** When naming components you create, you can always use names of your choice.

Step 1 Create VPC and Subnets

Create VPC

We have to create a VPC. Search for VPC and go to **Your VPCs > Create VPC**. Setup it as follows. Use **10.0.0.0/16** as CIDR block.



Create Subnets

Next, we Have to create six subnets in this VPC. (Refer the diagram)

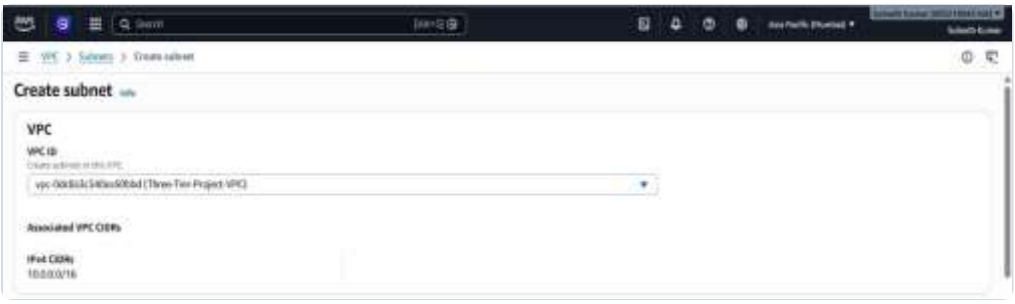
In Availability Zone 1a

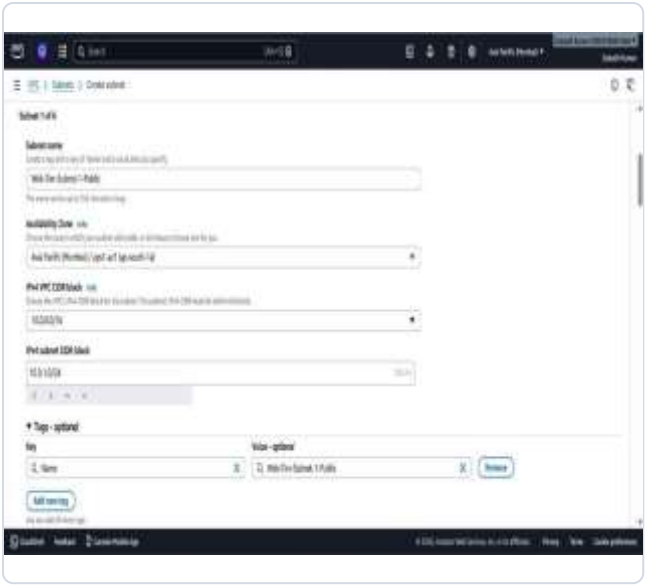
- Web-Tier-Subnet-1-Public
- App-Tier-Subnet-1-Private DB-Tier-Subnet-1-
- Private

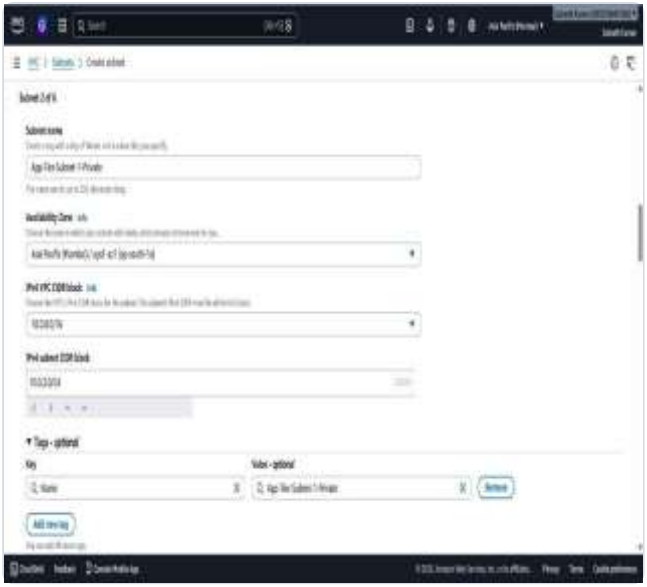
In Availability Zone 1b

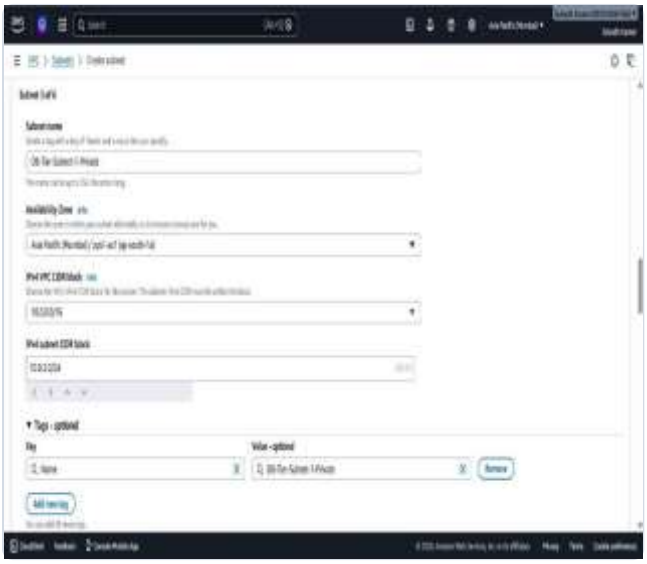
- Web-Tier-Subnet-2-Public
- App-Tier-Subnet-2-Private
- DB-Tier-Subnet-2-Private

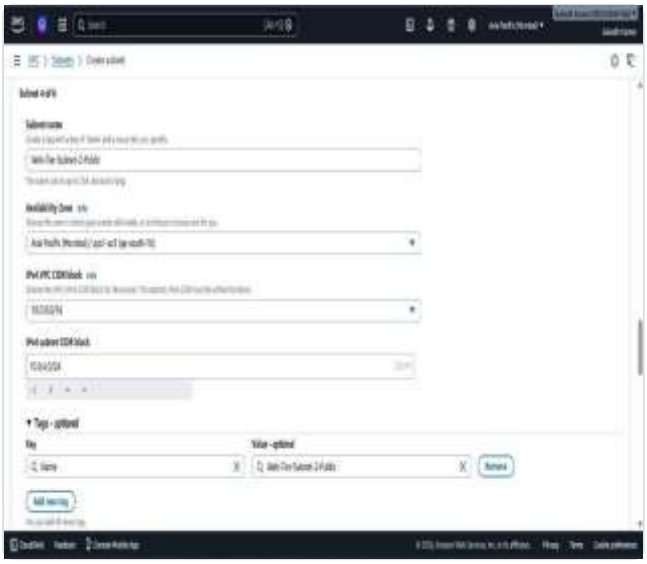
Go to **VPC dashboard > Subnets > Create subnets.** And setup them as follows. And Click create subnets


Create subnet
VPC
VPC ID
Associated VPC CIDRs
IPv4 CIDRs

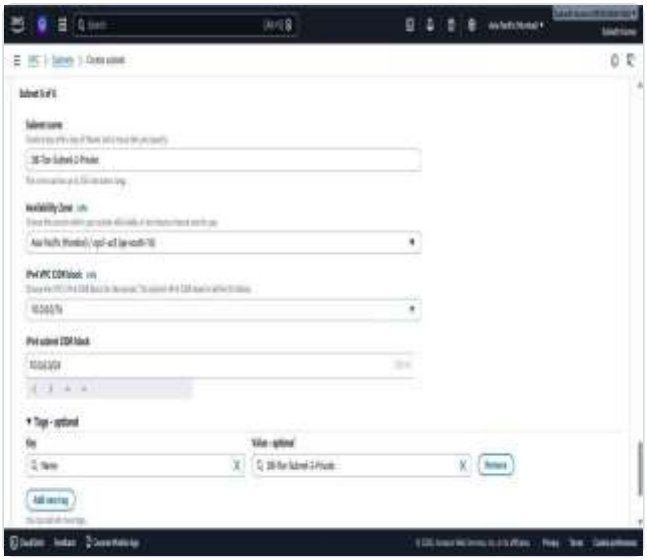




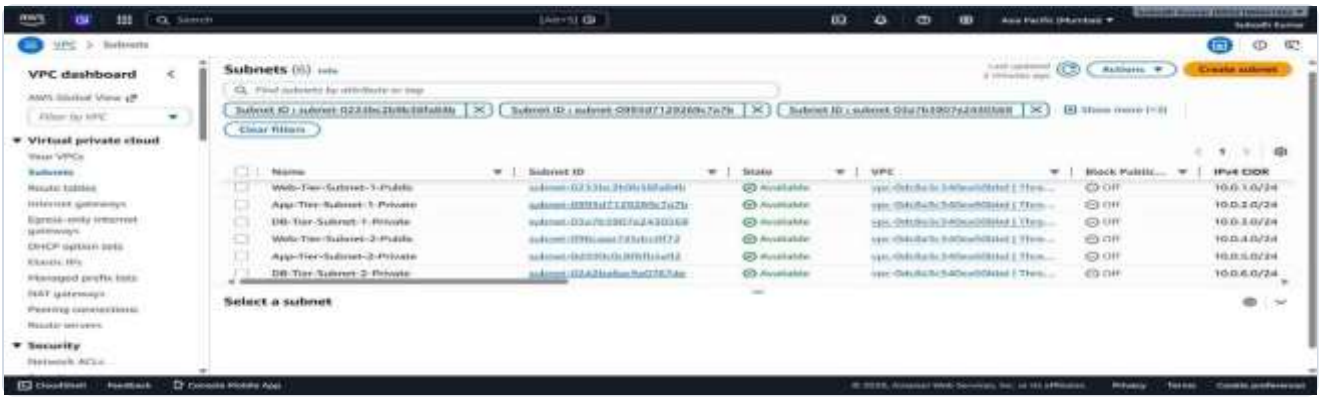




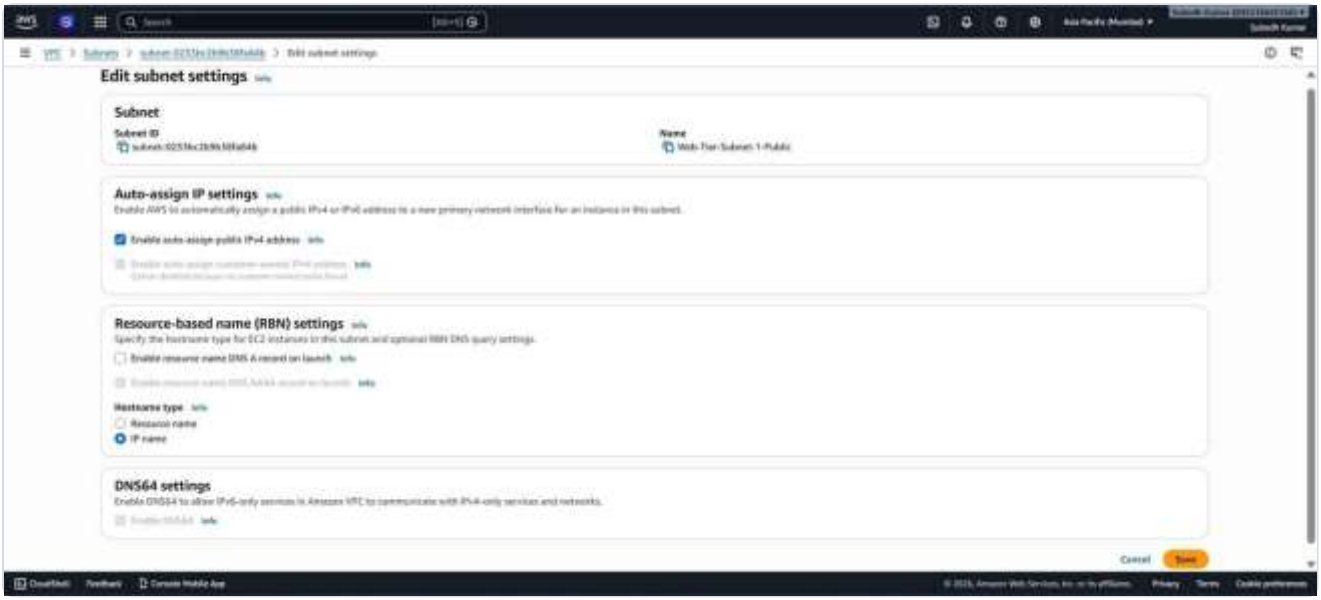


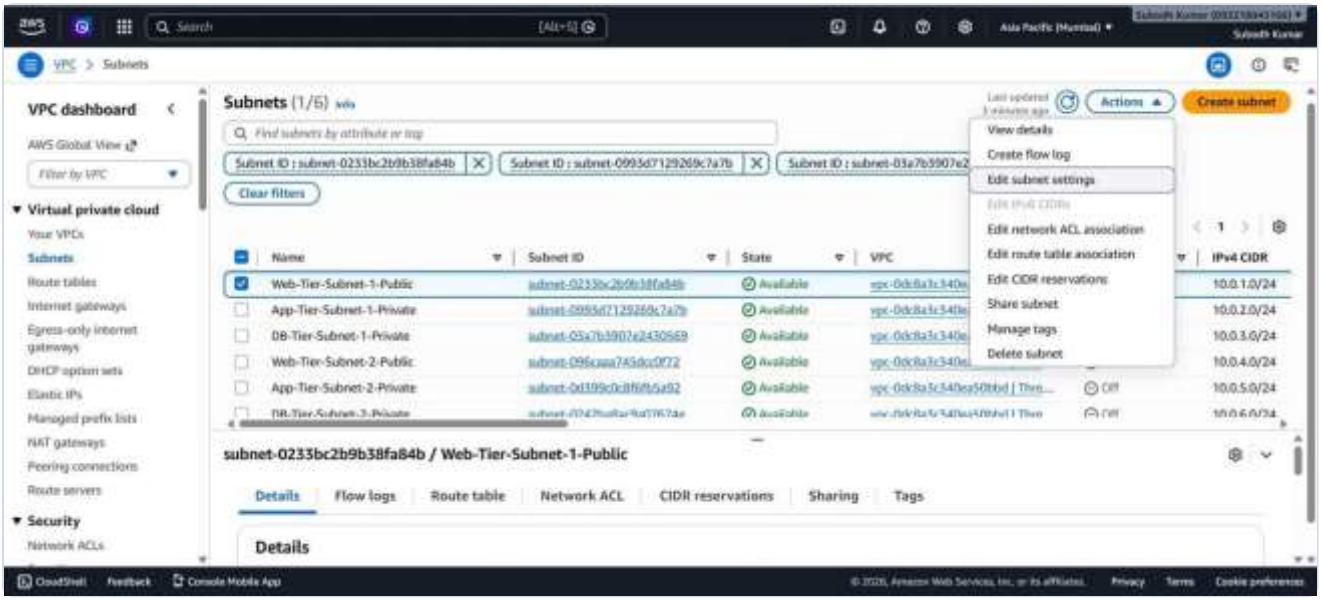


They will create as follows

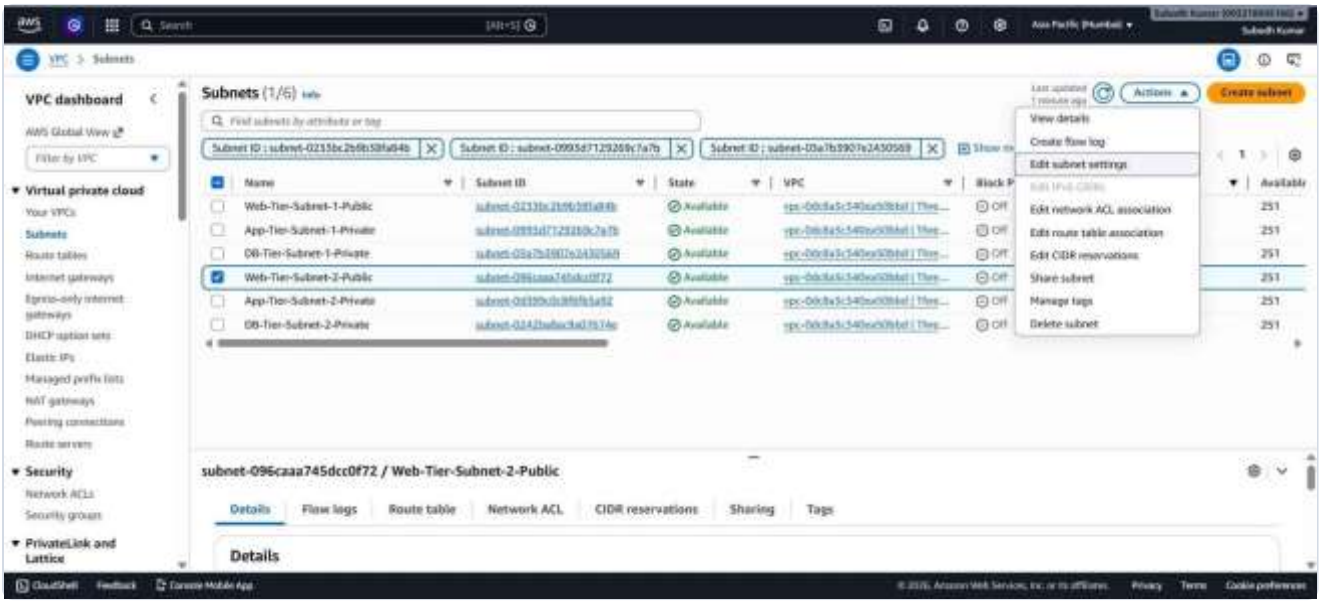


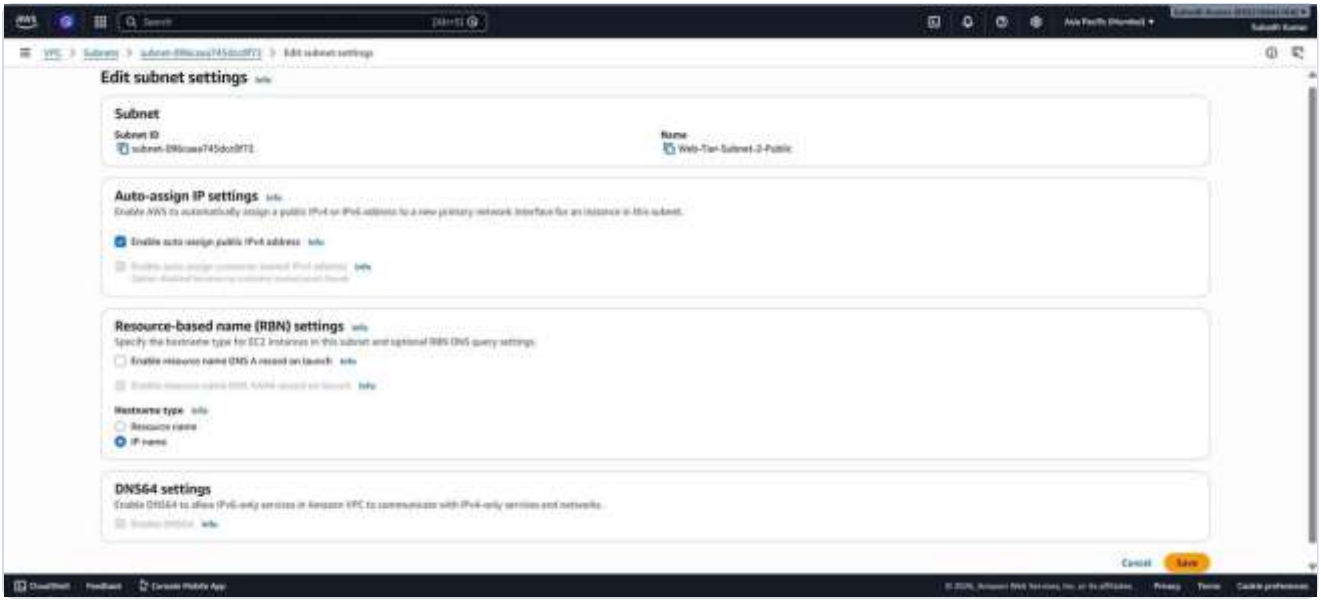
Now select **Web-Tier-Subnet-1-Public.** Go to **Action > Edit subnet settings.** Under Auto-assign IP settings, select **Enable auto-assign public IPv4 address** and save**.**





Do the same for **Web-Tier-Subnet-2-Public** as well.

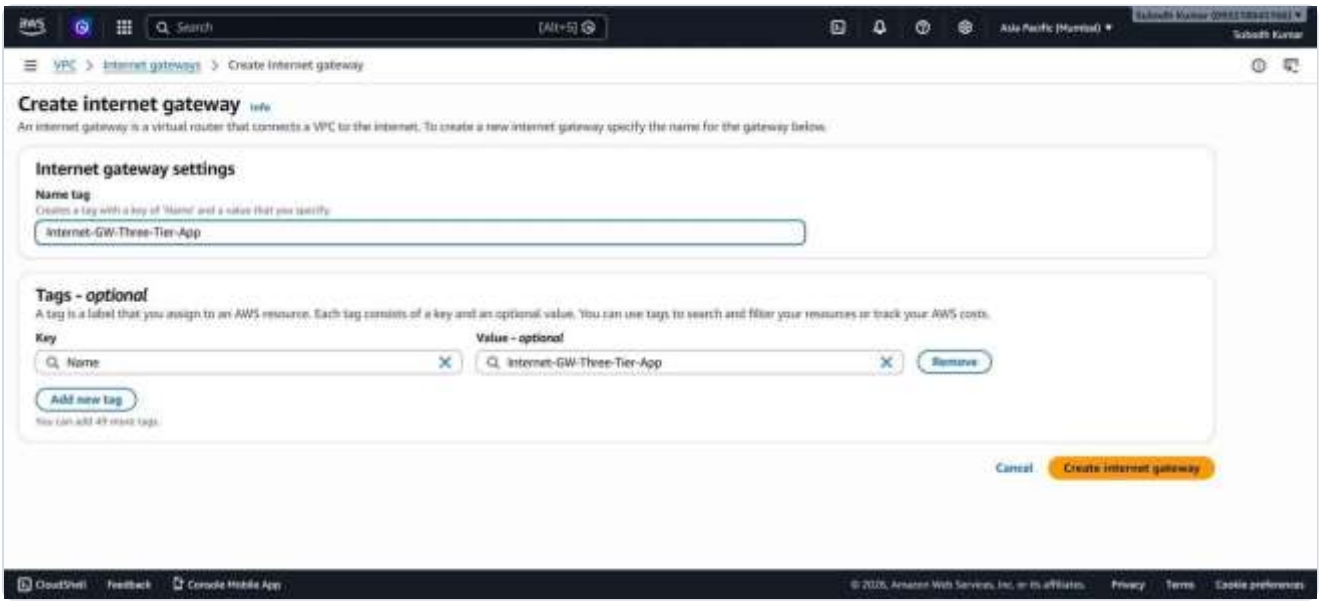

VPC dashboard
Subnets (1/6)
Actions
Create subnet
View details
Create flow log
Edit subnet settings
Edit network ACL association
Edit route table association
Edit CIDR reservations
Share subnet
Manage tags
Delete subnet
Web-Tier-Subnet-1-Public
App-Tier-Subnet-1-Private
DB-Tier-Subnet-1-Private
Web-Tier-Subnet-2-Public
App-Tier-Subnet-2-Private
DB-Tier-Subnet-2-Private
subnet-096caaa745dcc0f72 / Web-Tier-Subnet-2-Public
Details
Flow logs
Route table
Network ACL
CIDR reservations
Sharing
Tags


Edit subnet settings
Subnet
Auto-assign IP settings
Resource-based name (RBN) settings
DNS64 settings
Cancel
Save

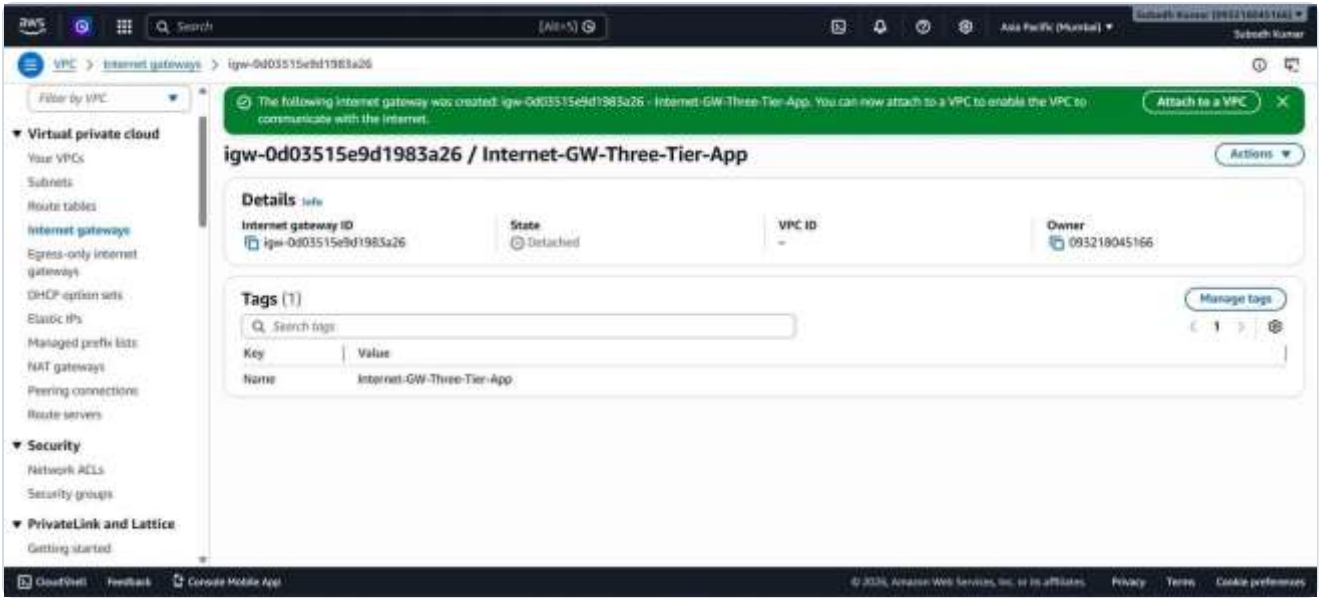
Step 2 Create Internet Gateway, NAT Gateways and Route Tables

Create Internet Gateway

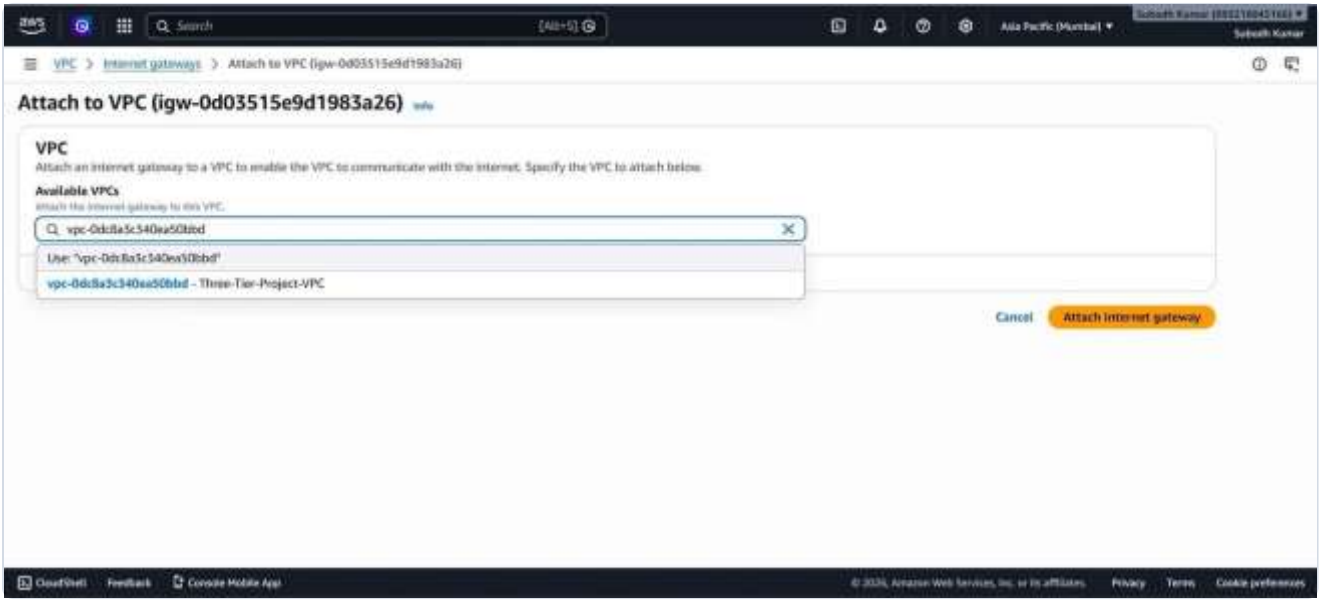
Go to VPC Dashboard **Internet gateways > Create internet Gateway.** Enter a name then click on Create internet gateway icon.



Then Go to internet gateways list and select our internet gateway and go to **Actions > Attach to VPC or Click directly on pop-up > Attach to VPC or Directly click on Pop-up menu.**

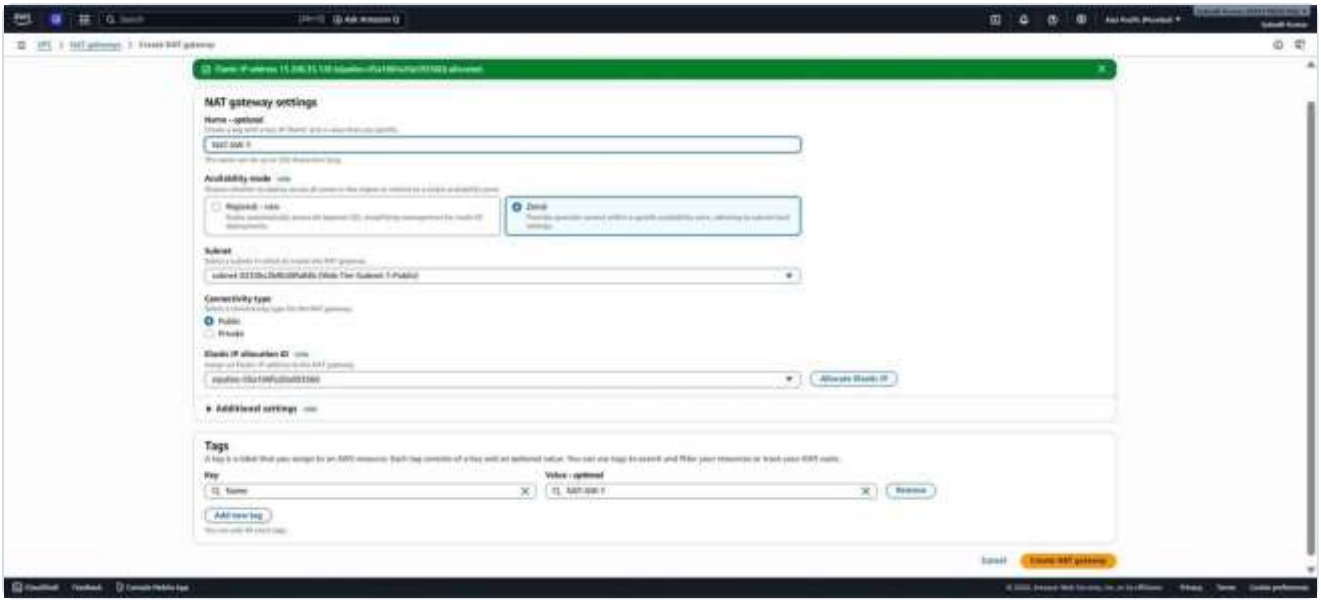


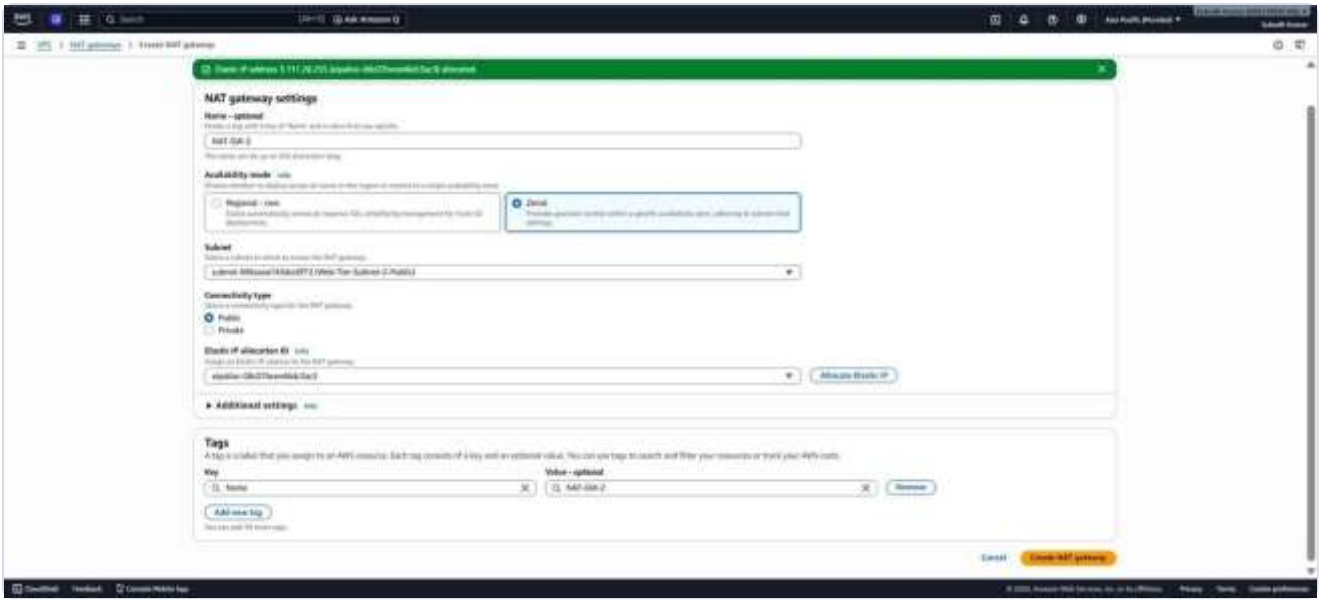
Then select our **VPC**. Finally, click on **Attach internet gateway** icon.



Create NAT Gateways

Go to VPC Dashboard **NAT gateways > Create NAT Gateway.** Create two NAT Gateways. Create it as follows.



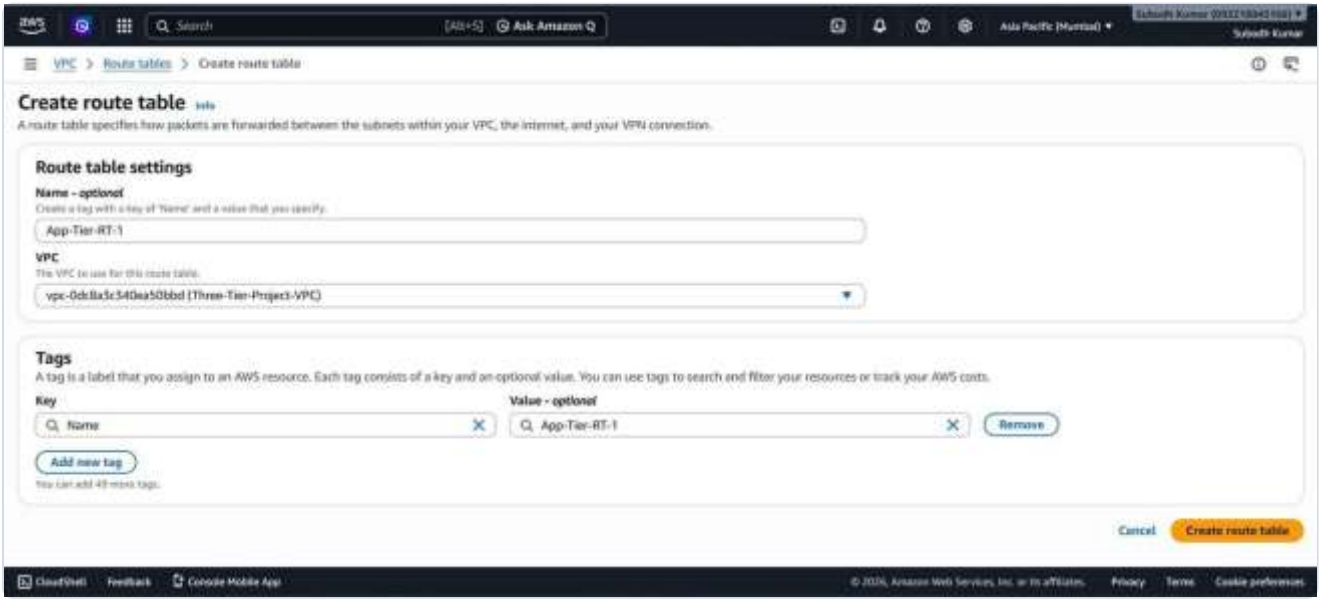


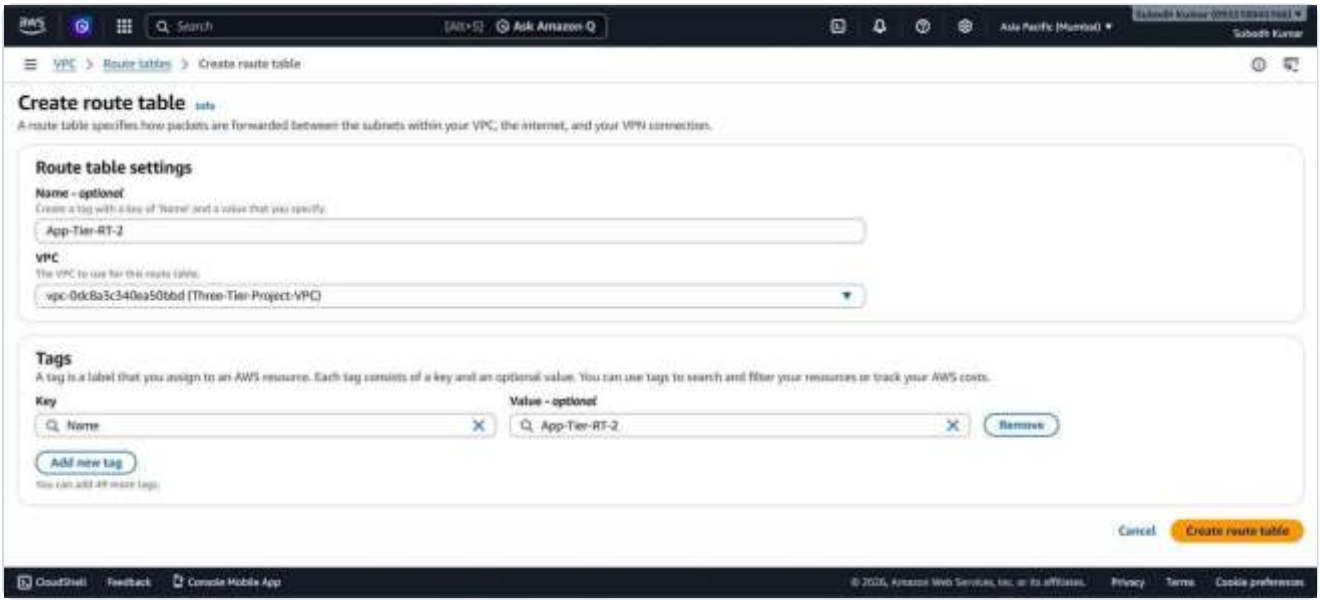
Create Route Tables

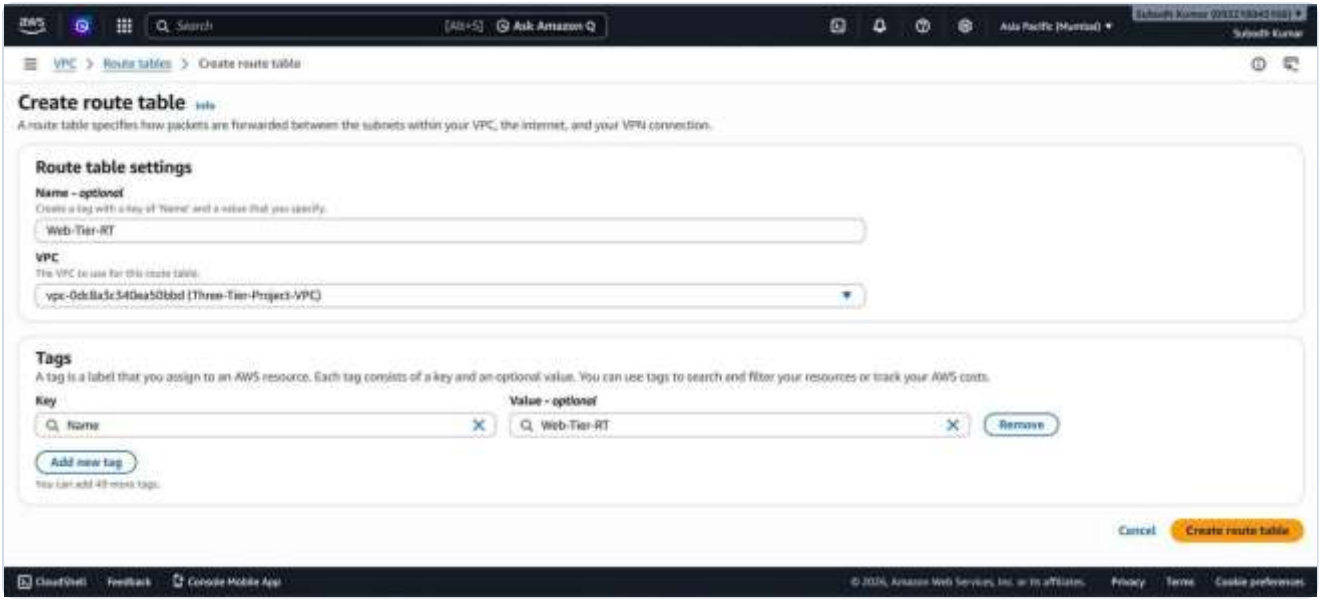
Now Let's create **Route tables**. According to our diagram we have to create **three route tables**.

- **App-Tier-RT-1**
- **App-Tier-RT-2 Web-Tier-**
- **RT**

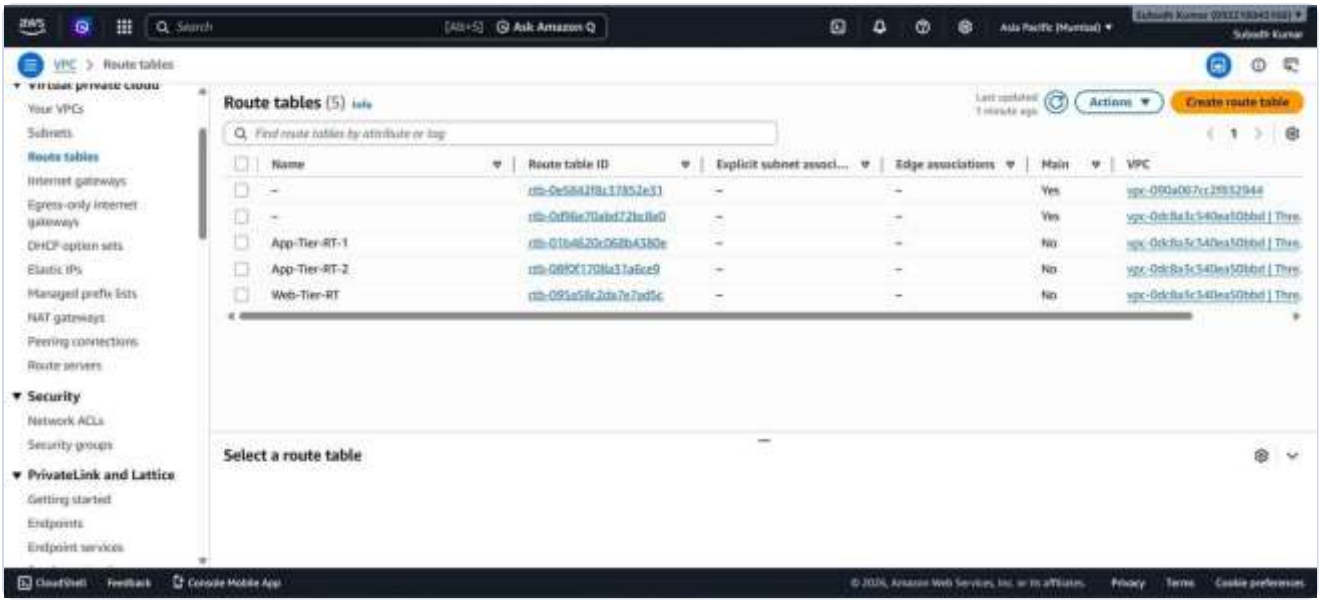
Now got to VPC dashboard **Route tables > Create route table**. Create Route tables as follows.



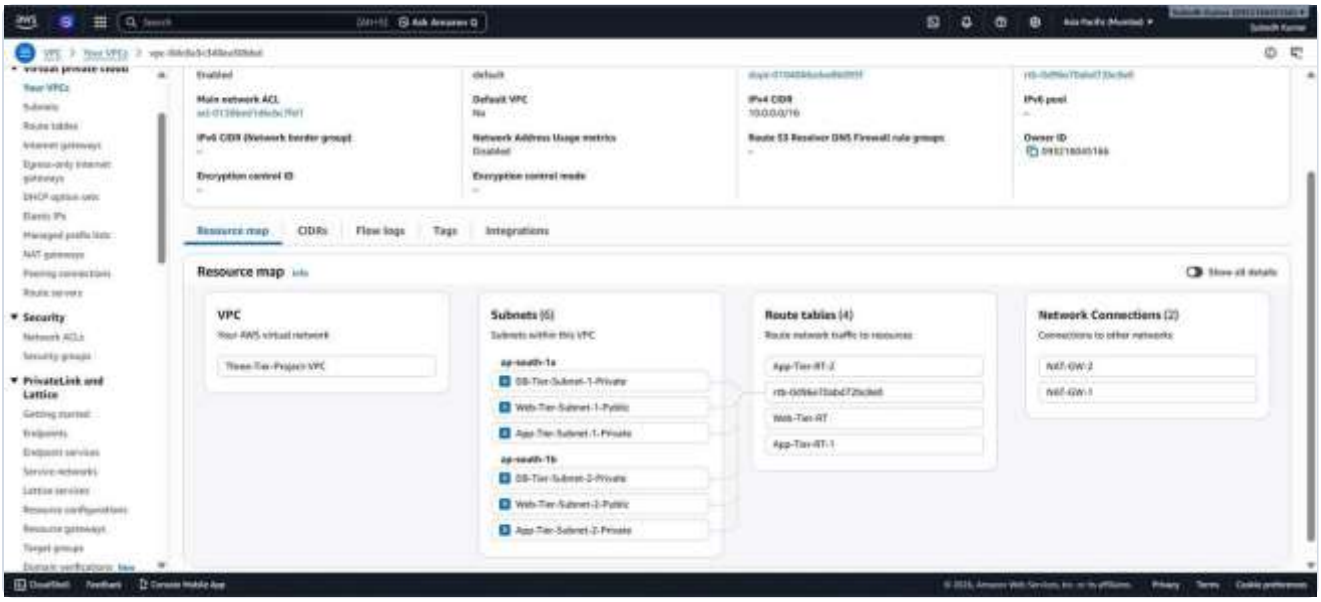

Search
[Alt+S]
Ask Amazon Q
Asia Pacific (Mumbai)
Subodh Kumar
VPC > Route tables > Create route table
Create route table
Info
A route table specifies how packets are forwarded between the subnets within your VPC, the internet, and your VPN connection.
Route table settings
Name - optional
Create a tag with a key of 'Name' and a value that you specify.
App-Tier-RT-2
VPC
The VPC to use for this route table.
vpc-0dc8a3c340ea50bbd (Three-Tier-Project-VPC)
Tags
A tag is a label that you assign to an AWS resource. Each tag consists of a key and an optional value. You can use tags to search and filter your resources or track your AWS costs.
Key
Value - optional
Name
App-Tier-RT-2
Remove
Add new tag
You can add 49 more tags.
Cancel
Create route table
CloudShell
Feedback
Console Mobile App
© 2026, Amazon Web Services, Inc. or its affiliates.
Privacy
Terms
Cookie preferences


Search
[Alt+S]
Ask Amazon Q
Asia Pacific (Mumbai)
Subodh Kumar
VPC > Route tables > Create route table
Create route table
Info
A route table specifies how packets are forwarded between the subnets within your VPC, the internet, and your VPN connection.
Route table settings
Name - optional
Create a tag with a key of 'Name' and a value that you specify.
Web-Tier-RT
VPC
The VPC to use for this route table.
vpc-0dc8a3c340ea50bbd (Three-Tier-Project-VPC)
Tags
A tag is a label that you assign to an AWS resource. Each tag consists of a key and an optional value. You can use tags to search and filter your resources or track your AWS costs.
Key
Value - optional
Name
Web-Tier-RT
Remove
Add new tag
You can add 49 more tags.
Cancel
Create route table
CloudShell
Feedback
Console Mobile App
© 2026, Amazon Web Services, Inc. or its affiliates.
Privacy
Terms
Cookie preferences

Now you will see all the route tables in the menu.

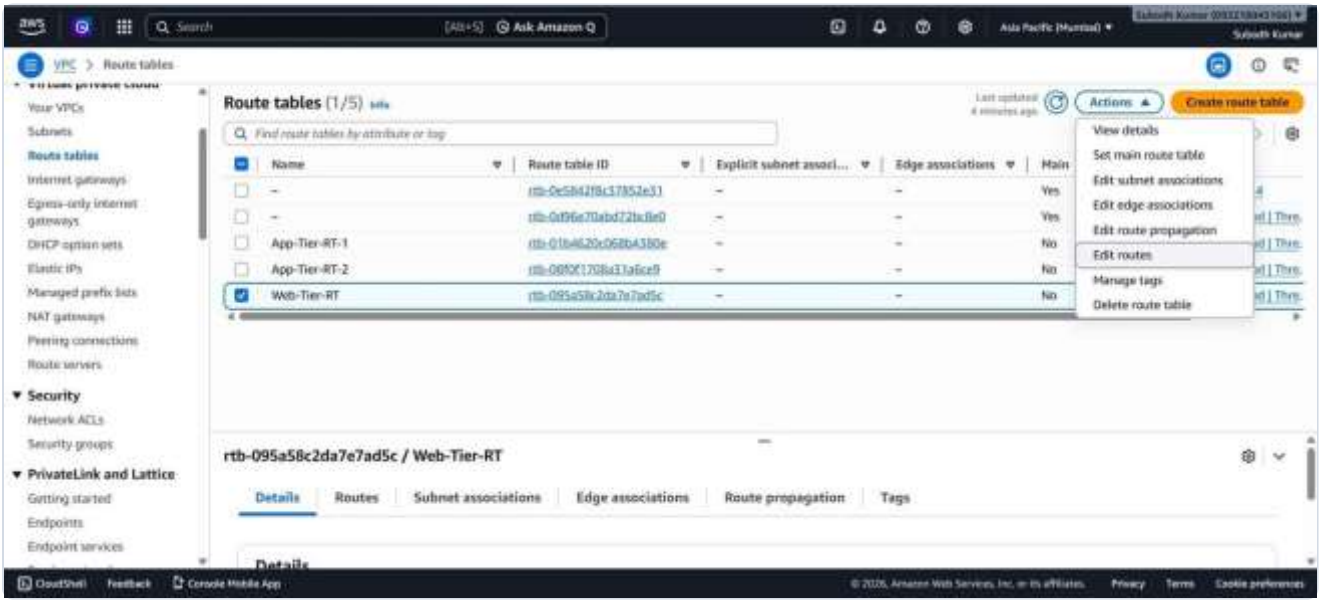


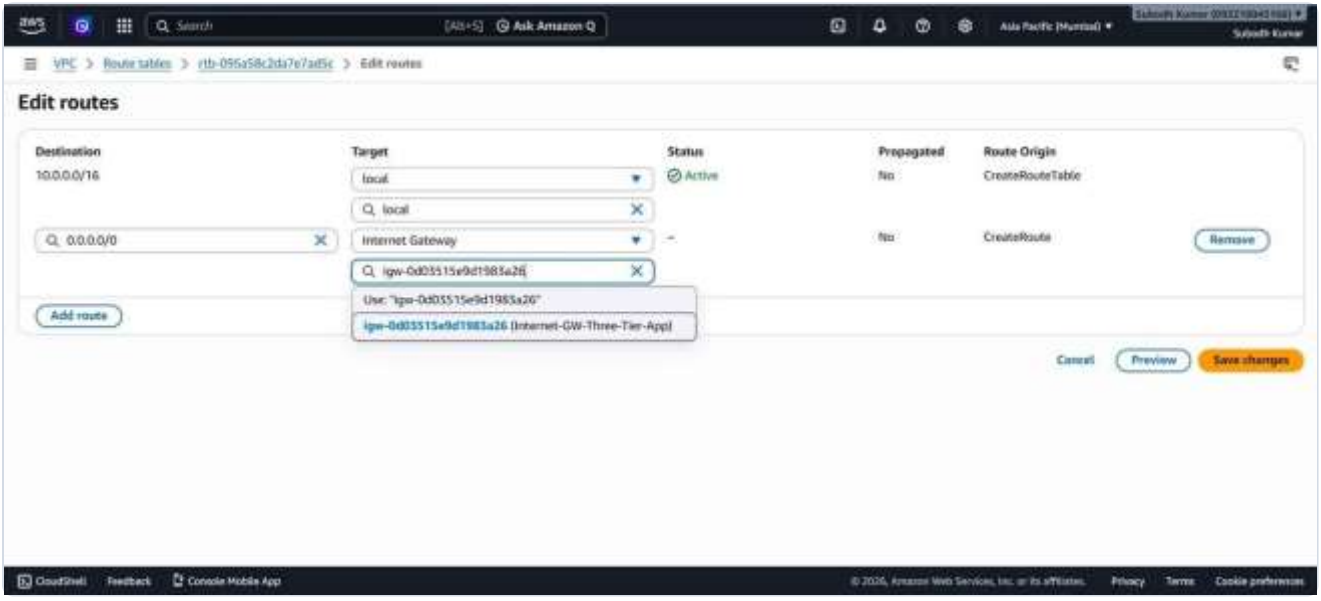
If you go to **VPC** dashboard **> Your VPCs**. And select **Three-Tier-Project-VPC**. Then if you go to **Resource map** you can see **Route tables** are **created** but there are **no connections.**



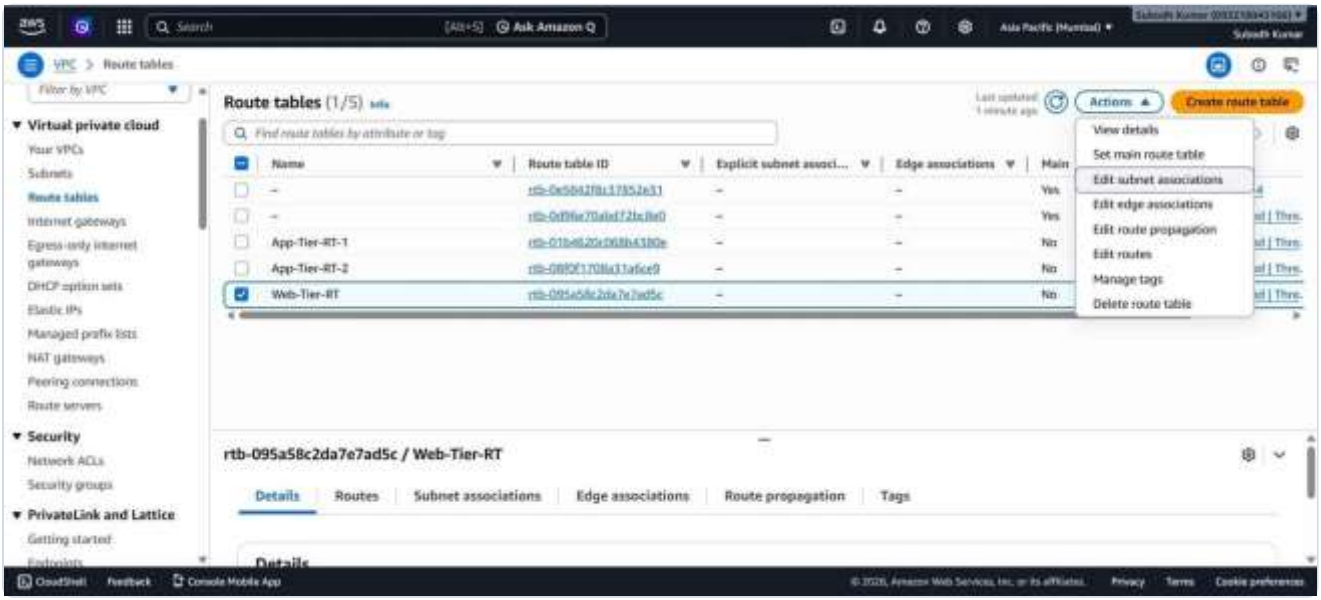
Edit Routes

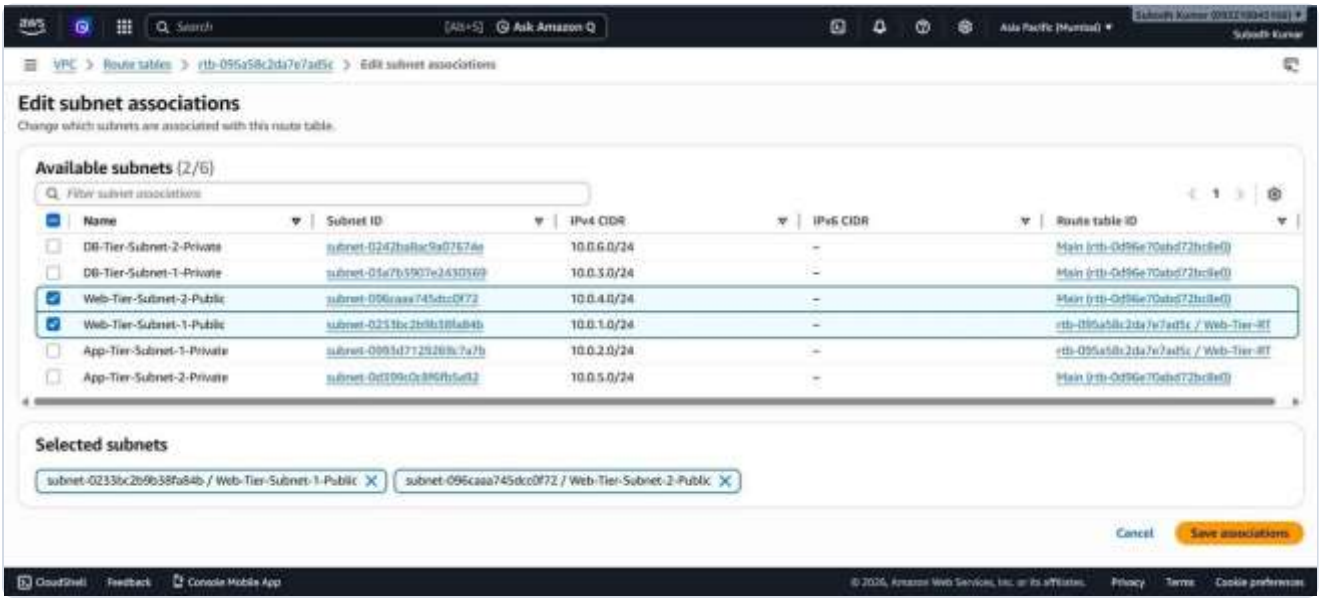
- Now Select Web-Tier-RT Go to **Actions > Edit routes**. Add route as follows. Destination 0.0.0.0/0, target internet gateway > select our igw. Then Save changes.


Route tables (1/5)
Actions
Create route table
View details
Set main route table
Edit subnet associations
Edit edge associations
Edit route propagation
Edit routes
Manage tags
Delete route table
App-Tier-RT-1
App-Tier-RT-2
Web-Tier-RT
rtb-095a58c2da7e7ad5c / Web-Tier-RT
Details
Routes
Subnet associations
Edge associations
Route propagation
Tags

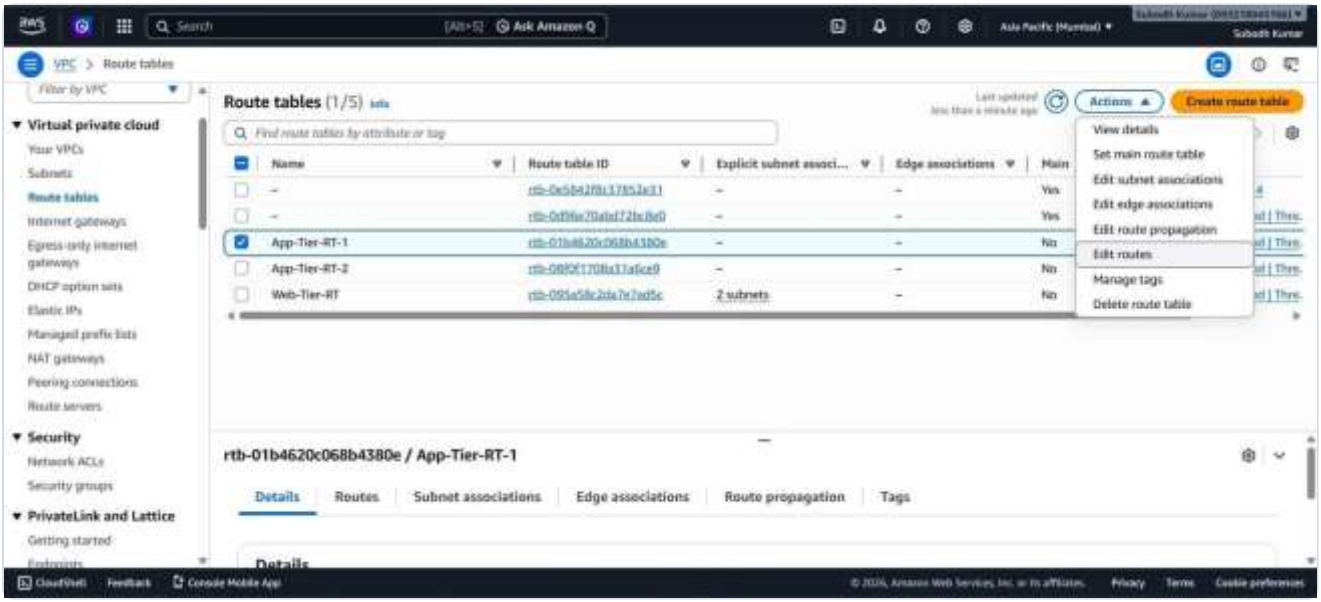

Edit routes
Destination
Target
Status
Propagated
Route Origin
10.0.0.0/16
local
Active
No
CreateRouteTable
0.0.0.0/0
Internet Gateway
No
CreateRoute
Remove
Add route
Cancel
Preview
Save changes

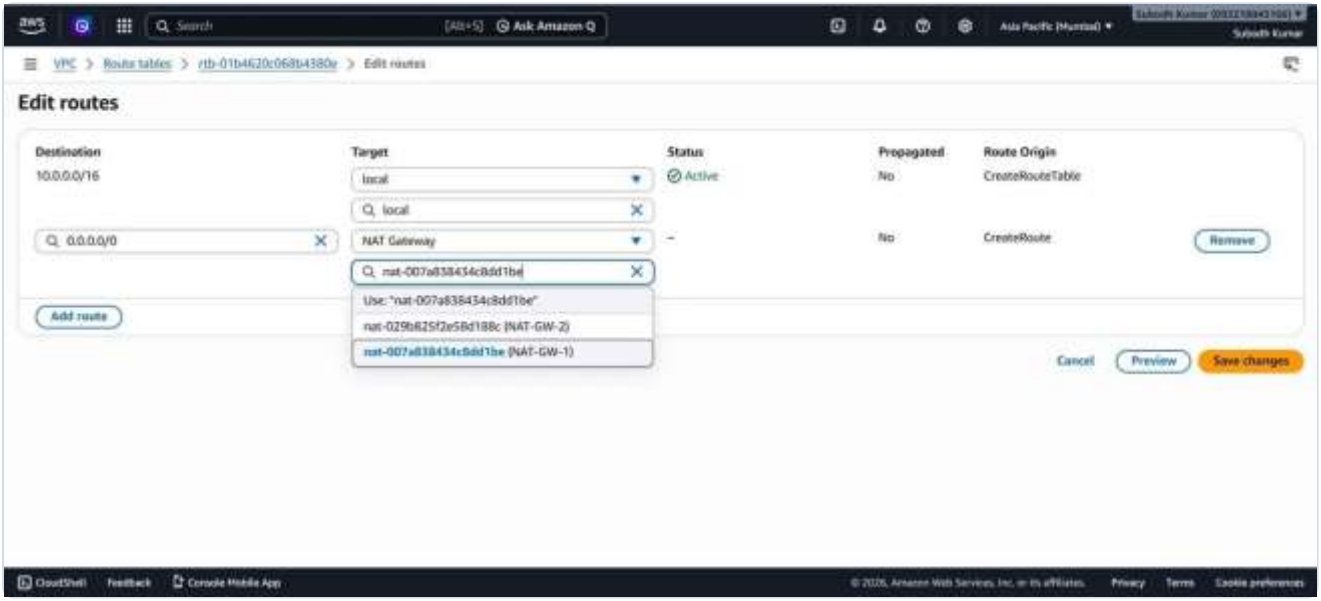
Again, select the Web-Tier-RT and go to **Actions > Edit subnet associations.** Select Web-Tier-Subnet-1-Public and Web-Tier-Subnet-2-Public and Save associations.





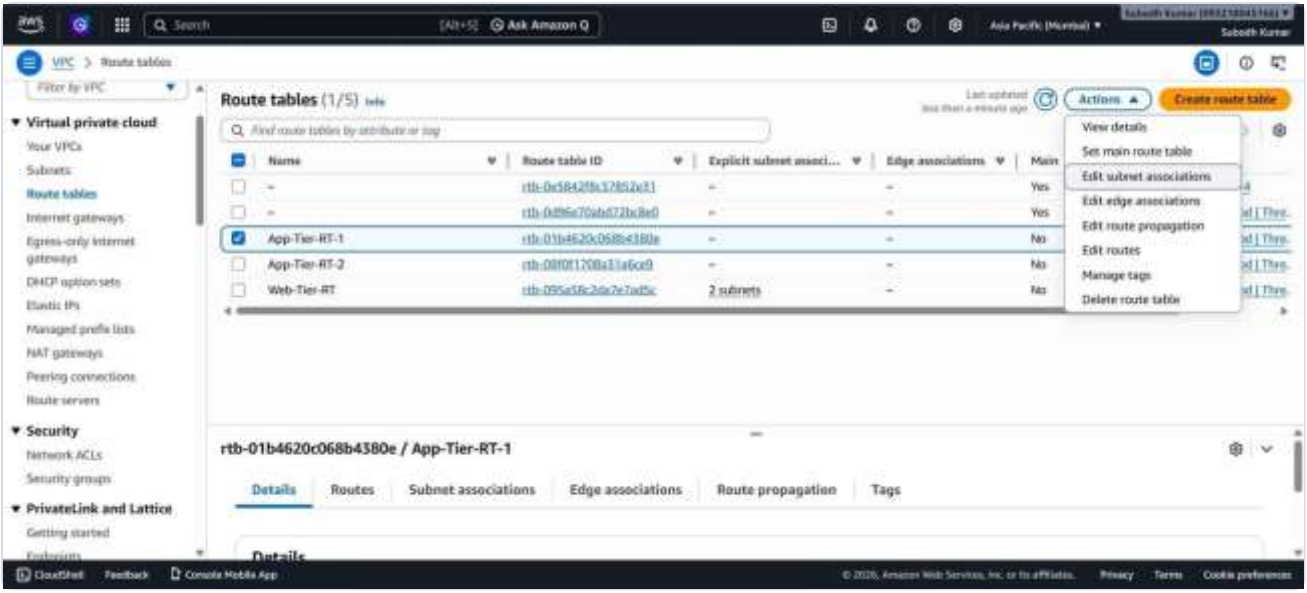
- Now Select **App-Tier-RT-1** Go to **Actions > Edit routes** Add route as follows. Destination 0.0.0.0/0, target NAT gateway > select our NAT-GW-1. Then Save changes.

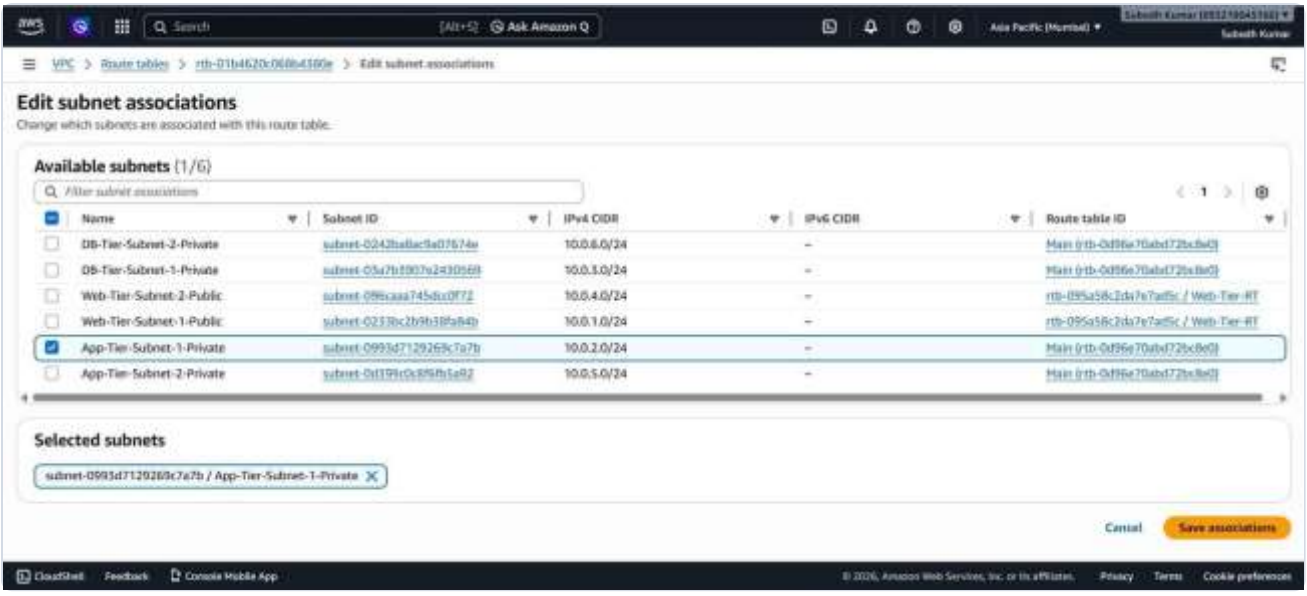




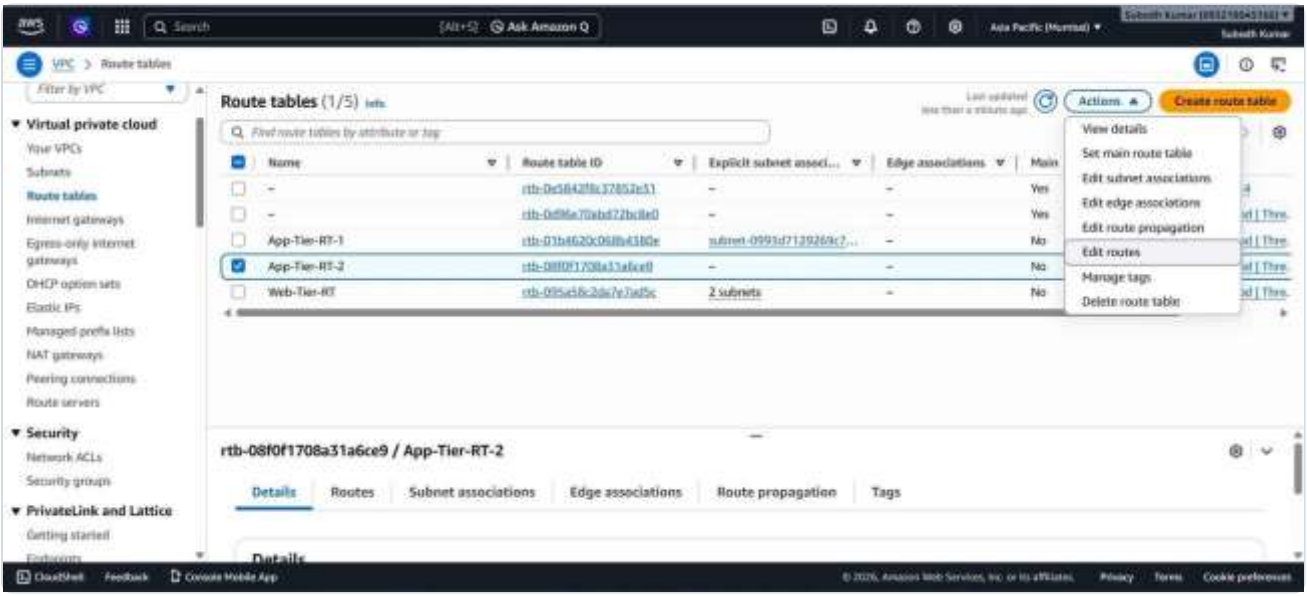
Again, select the App-Tier-RT-1 and go to **Actions > Edit subnet associations.**

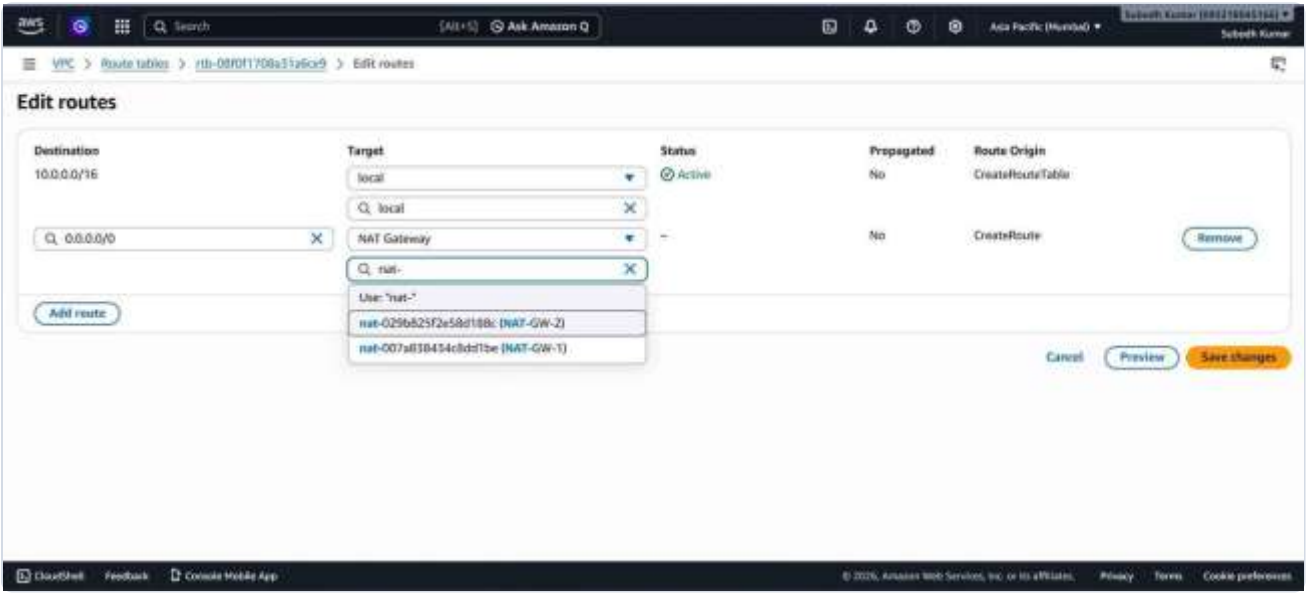
Select App-Tier-Subnet-1-Private and Save associations.





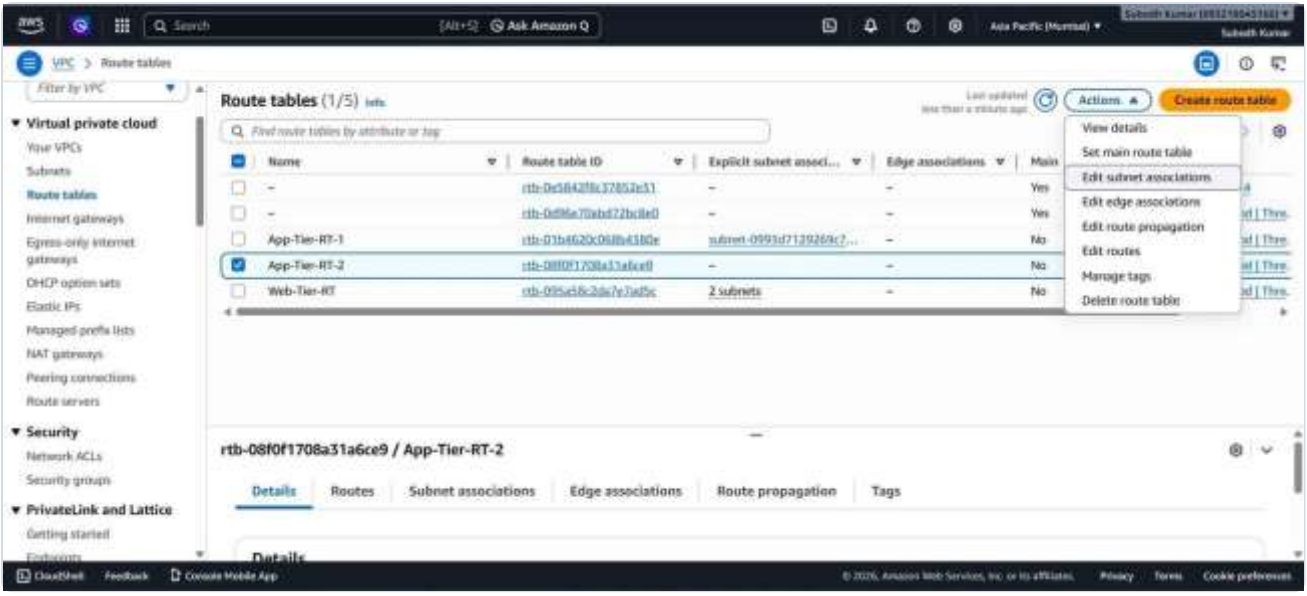
- Now Select **App-Tier-RT-2** Go to **Actions > Edit routes** Add route as follows. Destination 0.0.0.0/0, target NAT gateway > select our NAT-GW-2. Then Save changes.

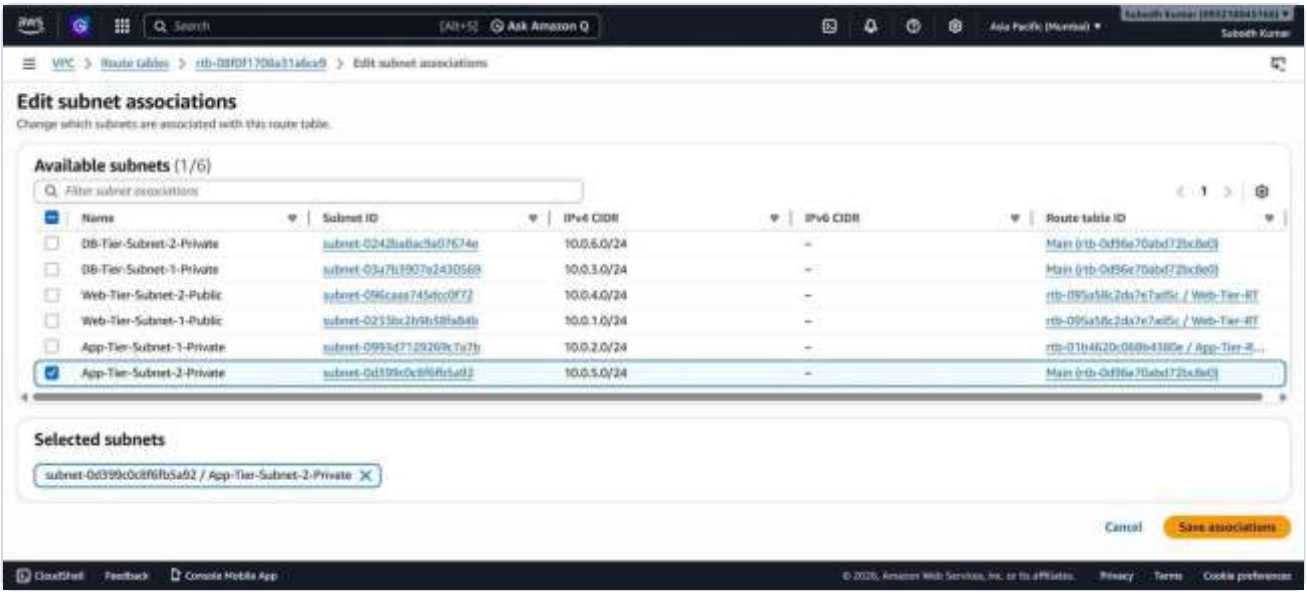




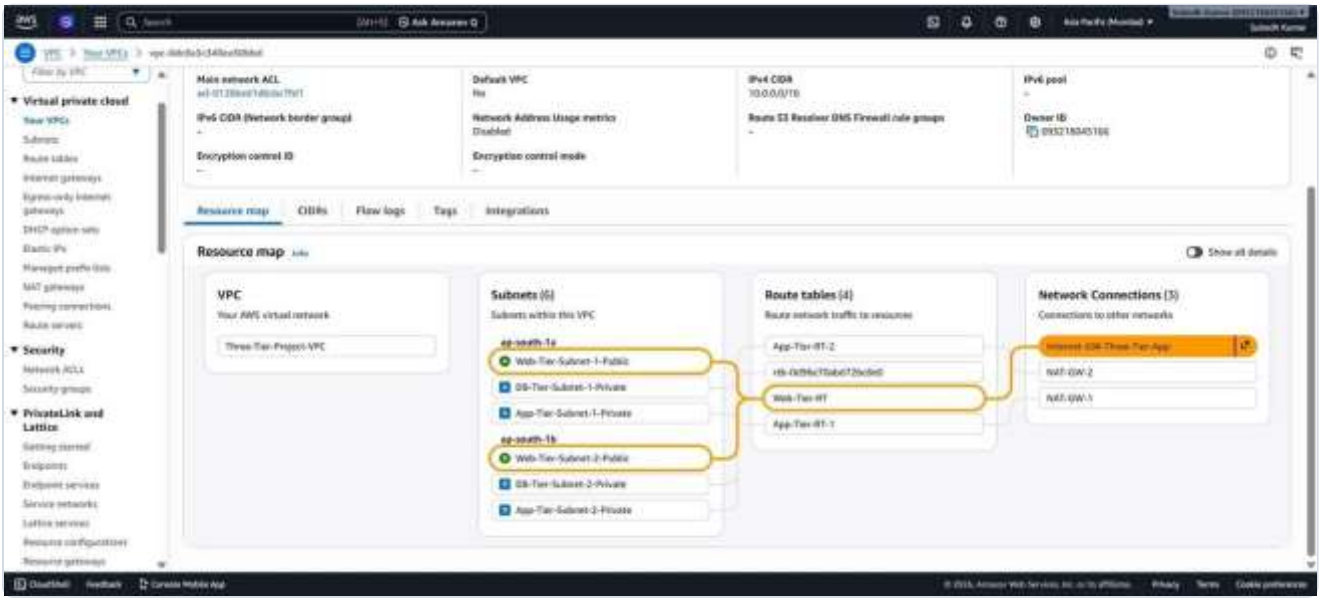
Again, select the App-Tier-RT-2 and go to **Actions > Edit subnet associations.**

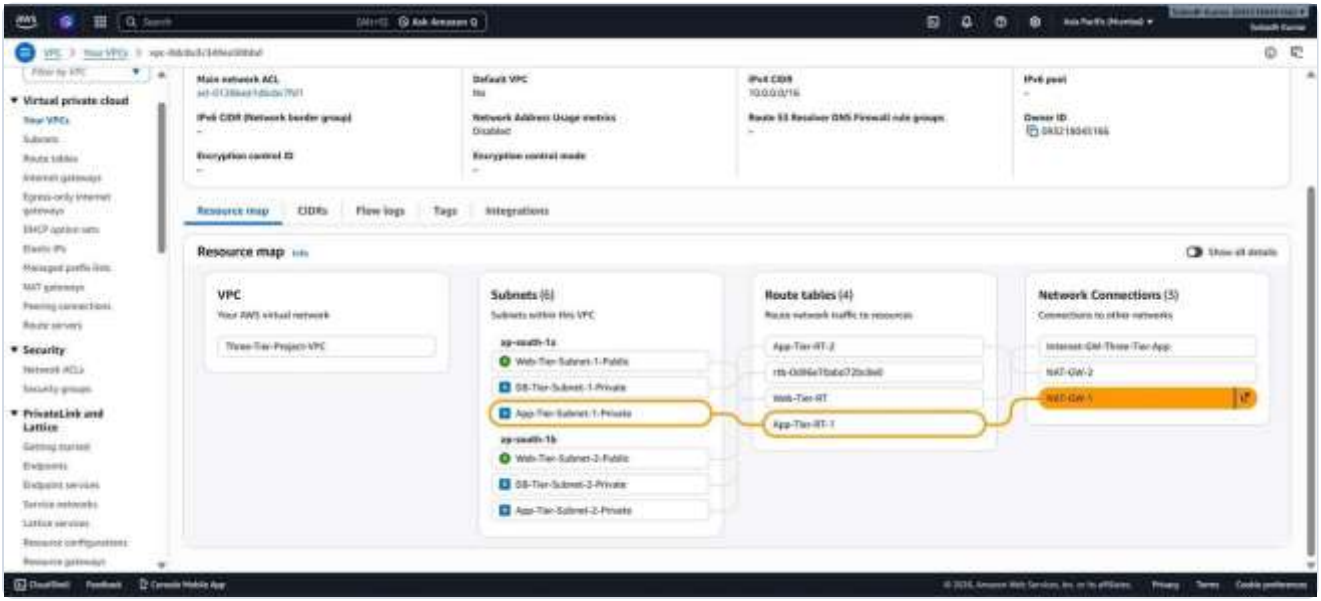
Select App-Tier-Subnet-2-Private and Save associations.

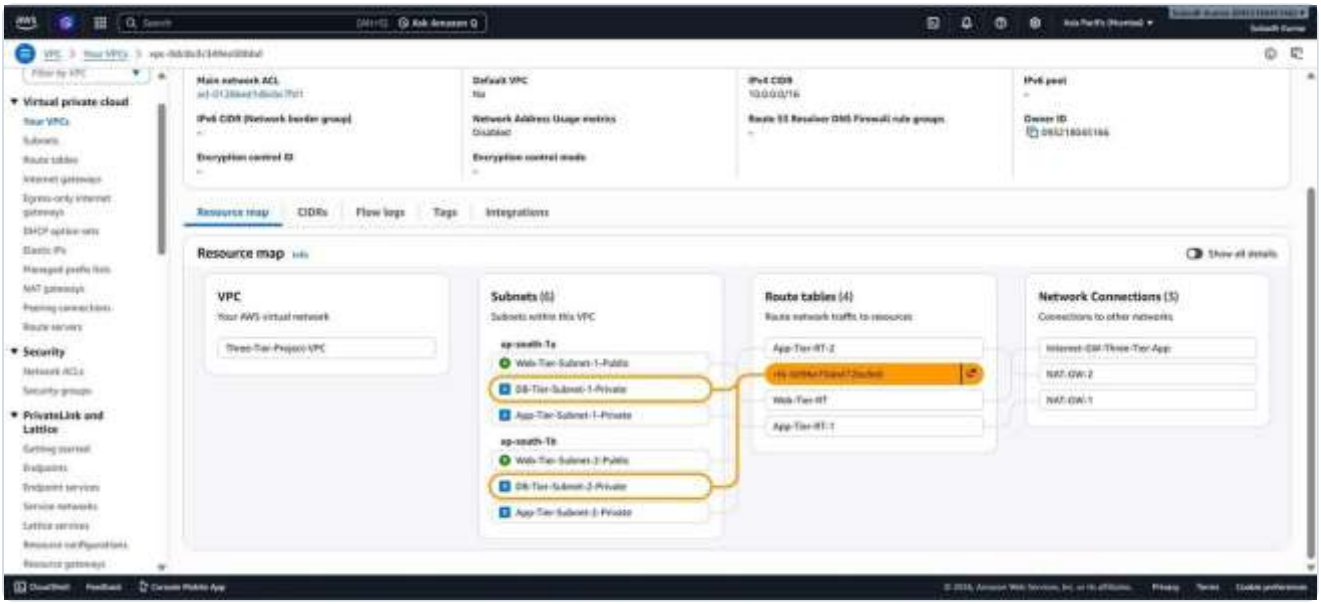




Now if you **check** the **Resource map** you can see that connections have been made. **Check** if there are **any mistakes** according to our architecture diagram by hovering over each element.







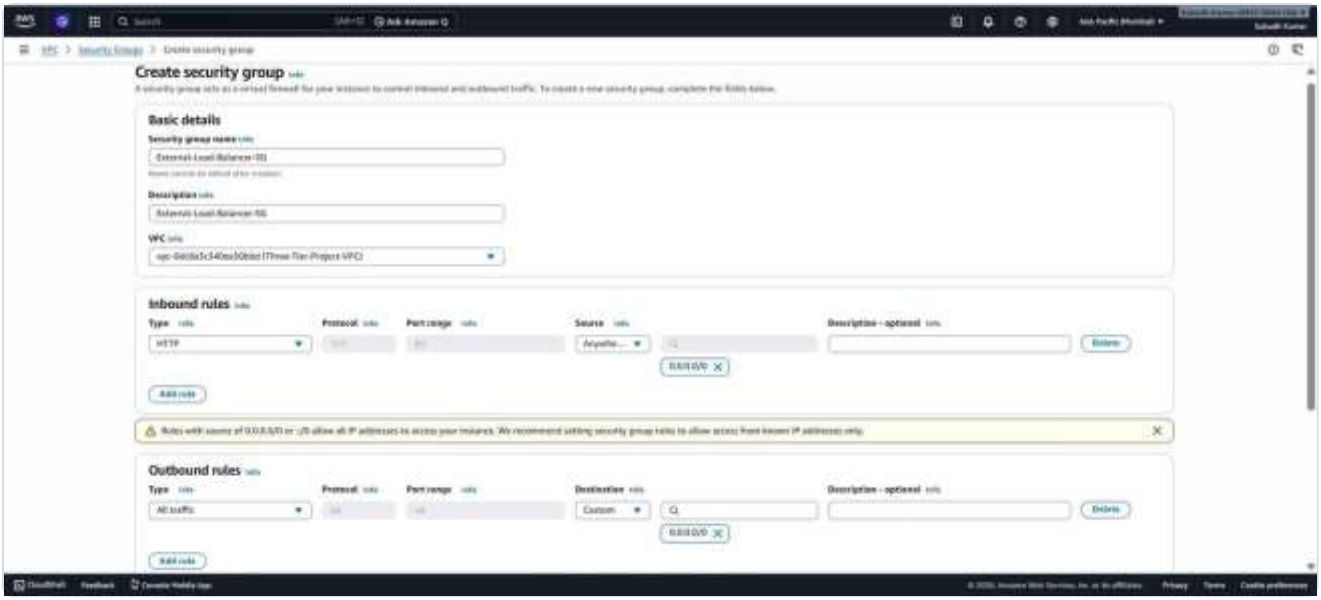
Good job! Now let's create the Security Groups.

Step 3 Create Security Groups

We have to create the following Security Groups.

Security Group	Type (Port Range)	Source
External-Load-Balancer-SG	HTTP (80)	0.0.0.0/0
Web-Tier-SG	HTTP (80)	External-Load-Balancer-SG
Internal-Load-Balancer-SG	HTTP (80)	Web-Tier-SG
App-Tier-SG	Custom TCP (4000)	Internal-Load-Balancer-SG
DB-Tier-SG	MYSQL (3306)	App-Tier-SG

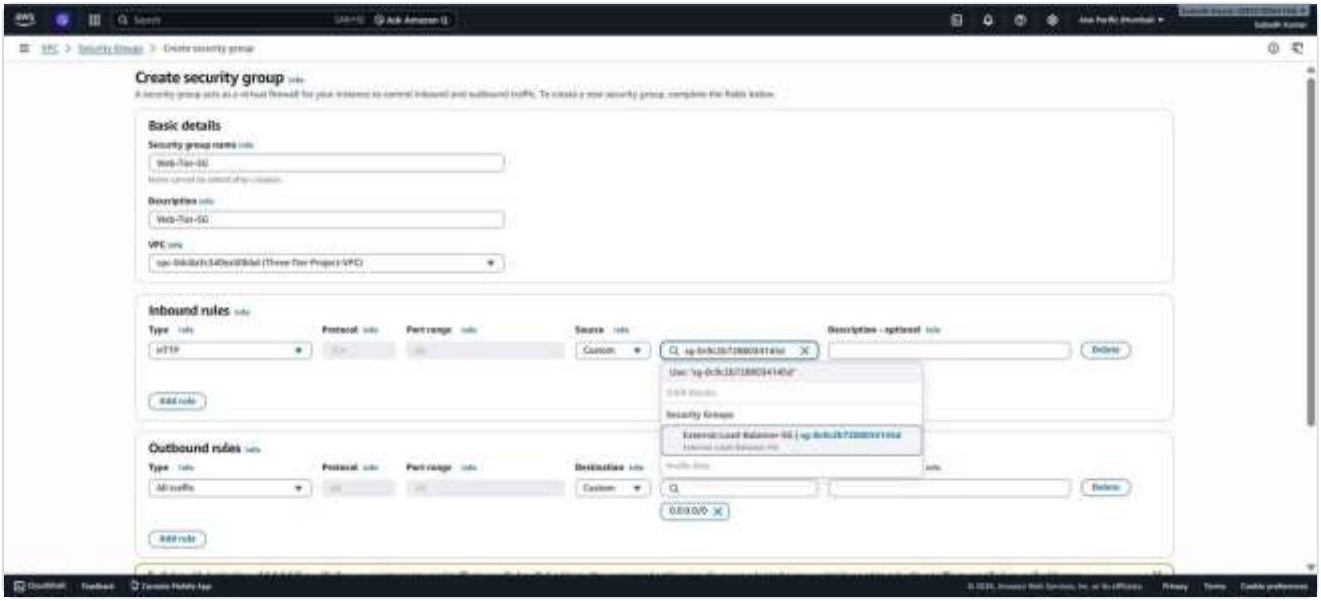
Go to VPC dashboard **Security Groups > Create security group**. Create **External-Load-Balancer-SG** and add inbound rule as follows by referring the above table. Then, Click **Create security group** icon.





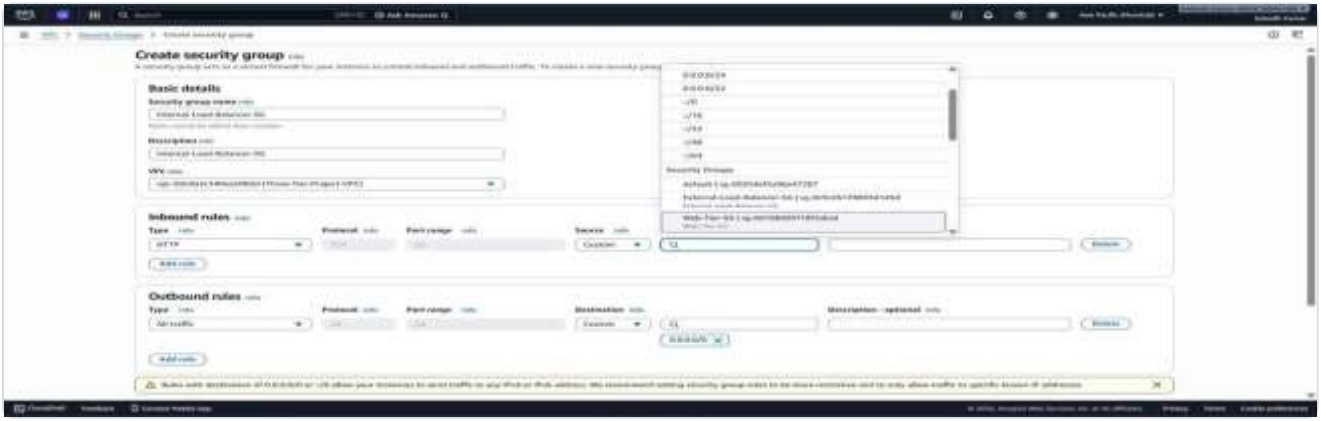
Creating other Security groups as well as shown in the table. The process is same.

Go to VPC dashboard **Security Groups > Create security group**. Create **Web-Tier-SG** and add inbound rule as follows by referring the above table. Then, Click **Create security group** icon.



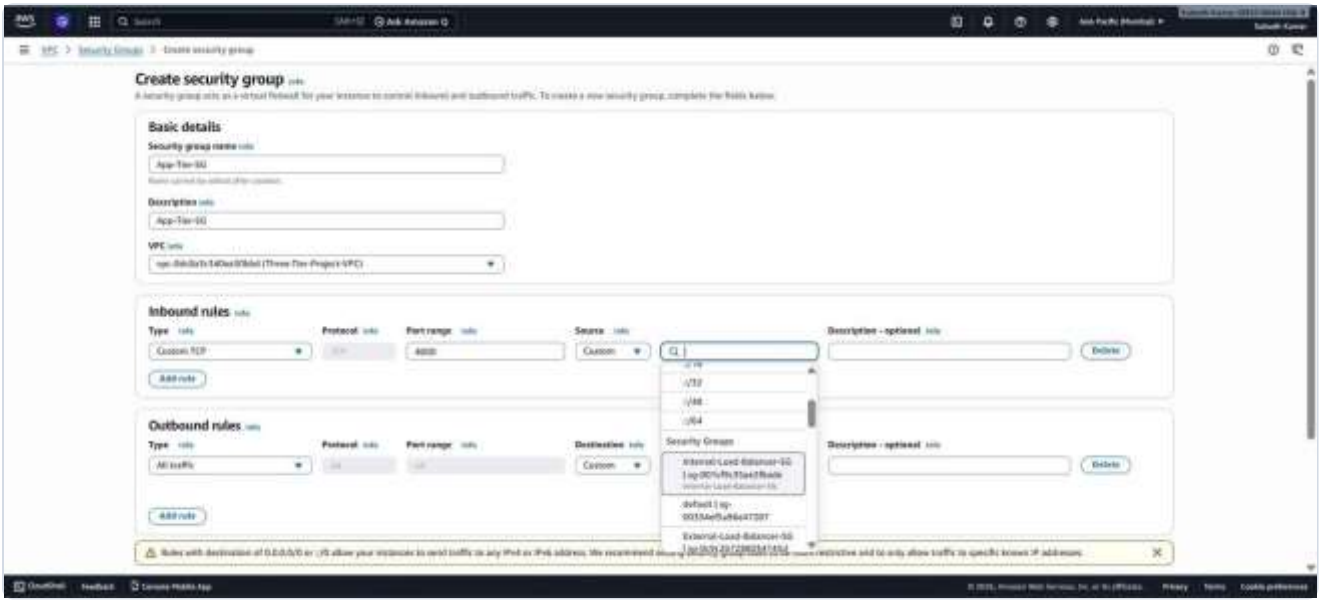


Go to VPC dashboard **Security Groups > Create security group**. Create **Internal- Load-Balancer-SG** and add inbound rule as follows by referring the above table. Then, Click **Create security group** icon.



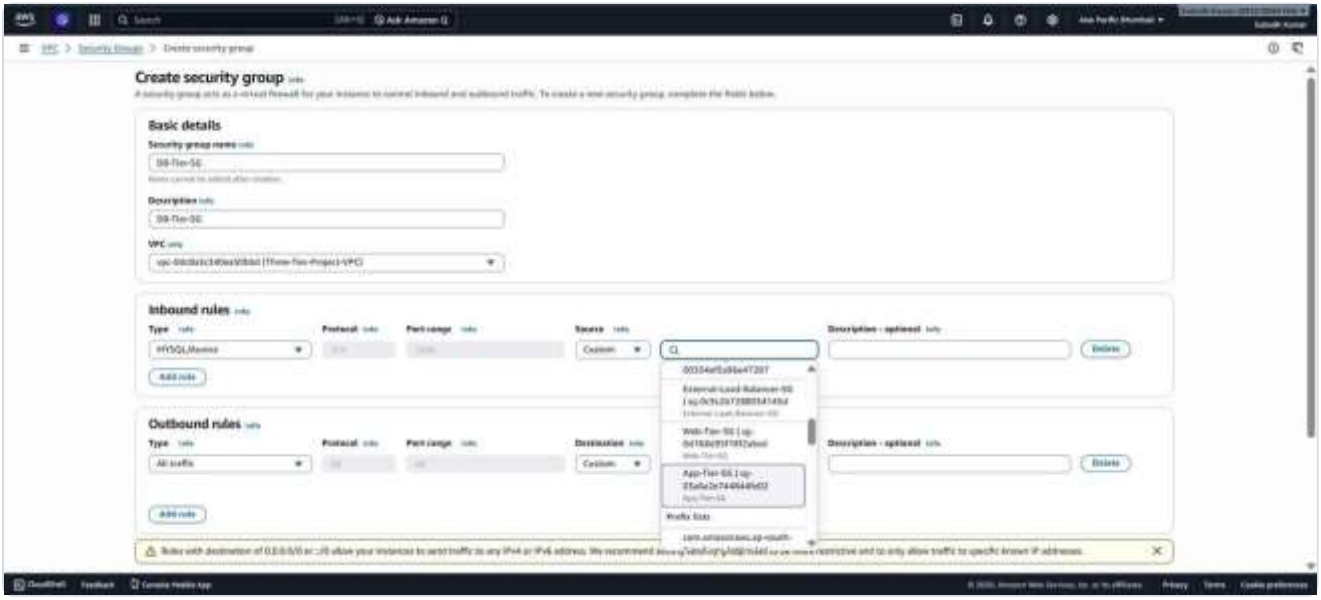


Go to VPC dashboard **Security Groups > Create security group**. Create **App-Tier-SG** and add inbound rule as follows by referring the above table. Then, Click on **Create security group** icon.



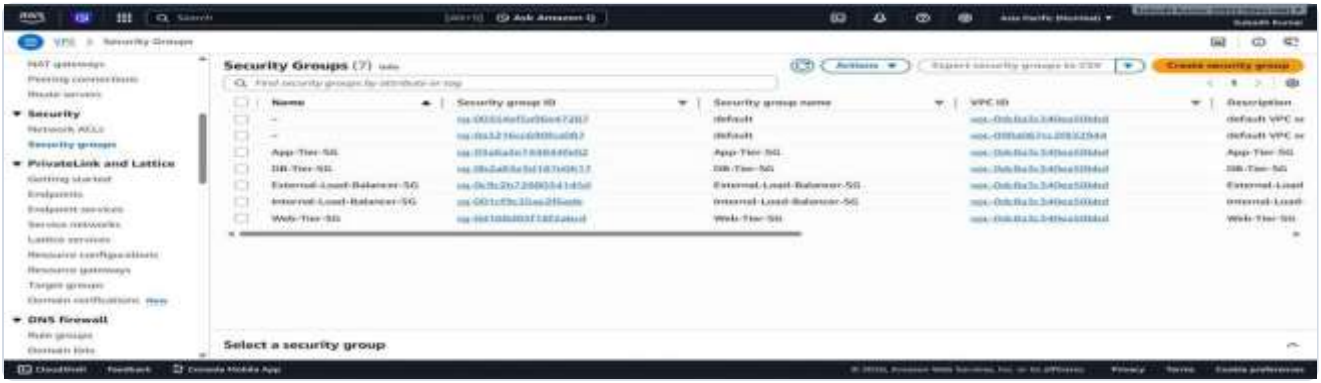


Go to VPC dashboard **Security Groups > Create security group**. Create **DB-Tier-SG** and add inbound rule as follows by referring the above table. Then, Click on **Create security group** icon.





After creating all Security groups you can see them like this.



Well done! Now let’s move on to setting up the database servers.

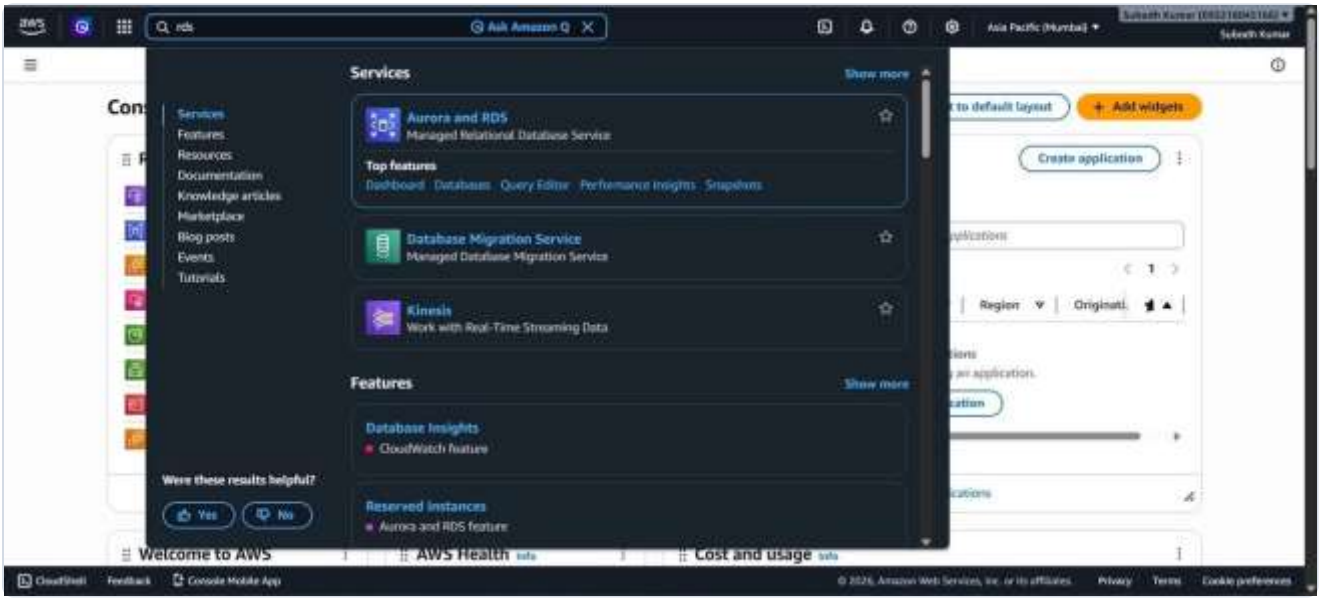
Step 4 Create Database Subnet Group and RDS

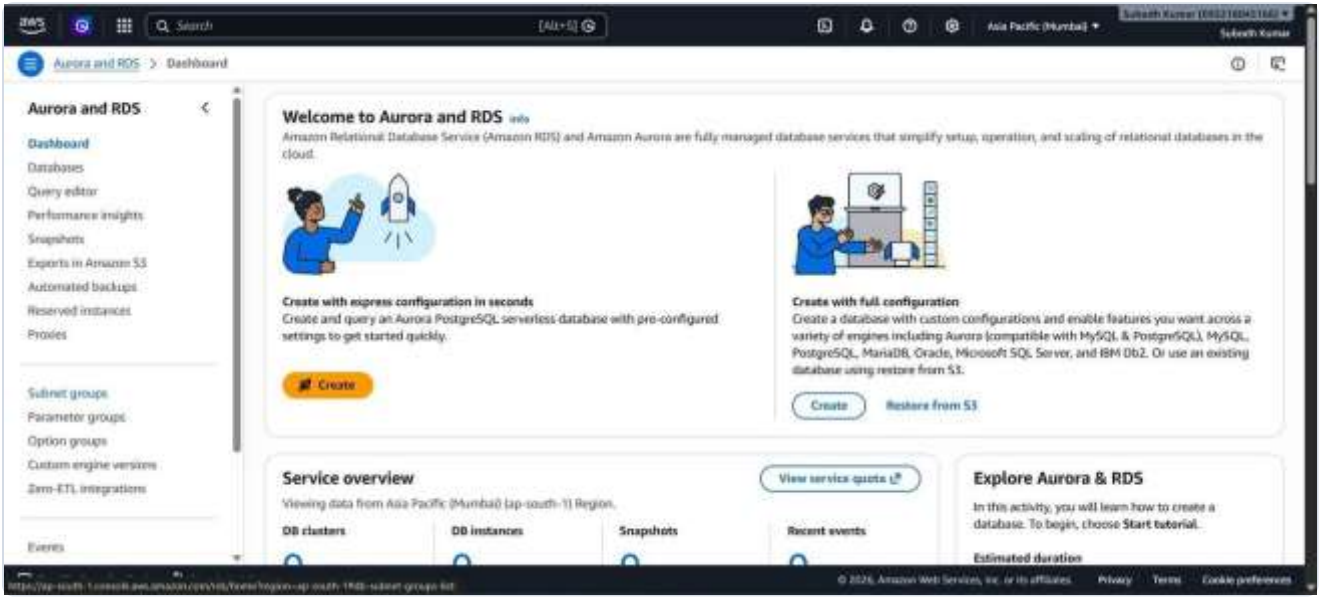
First, we need to **create** a **subnet group**. But **why?**

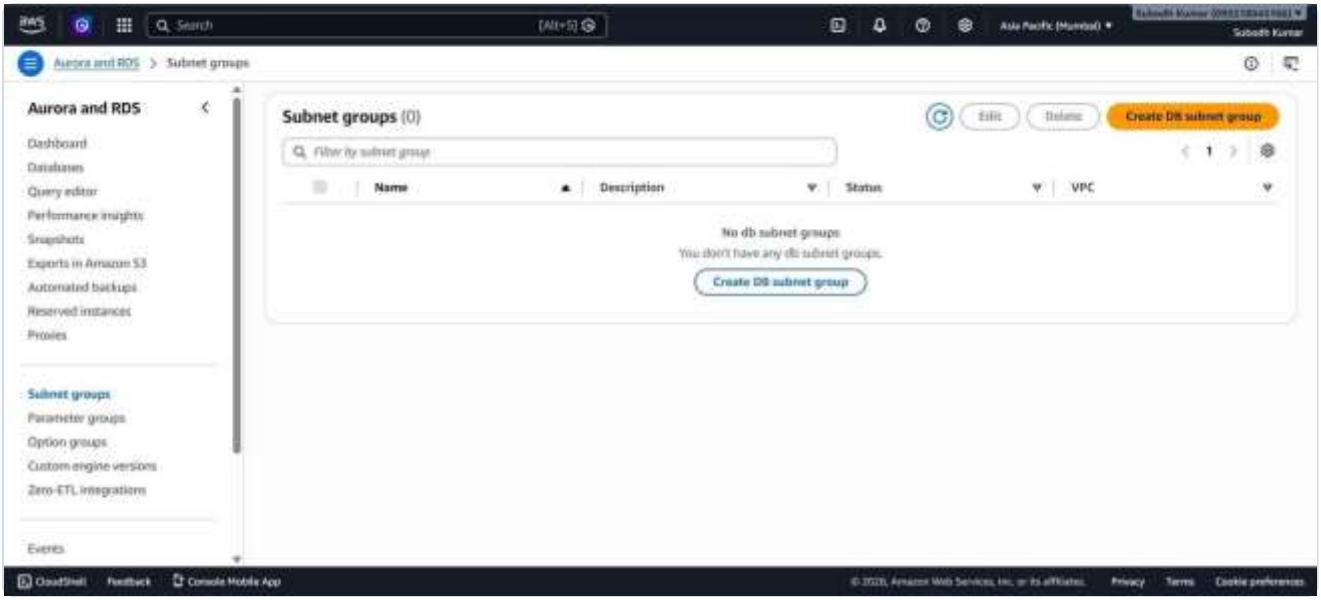
Because, when we **deploy** an **RDS instance** in a **Multi-AZ configuration**, AWS **automatically creates** a **primary database instance** in **one Availability Zone (AZ)** and **a standby replica** in **another AZ** within the **same region.** This setup **requires subnet groups** because **each subnet** in the **group** must belong to a **different AZ**, **ensuring** that your **database instances** are **spread across multiple AZs** for **redundancy** and **failover protection**.

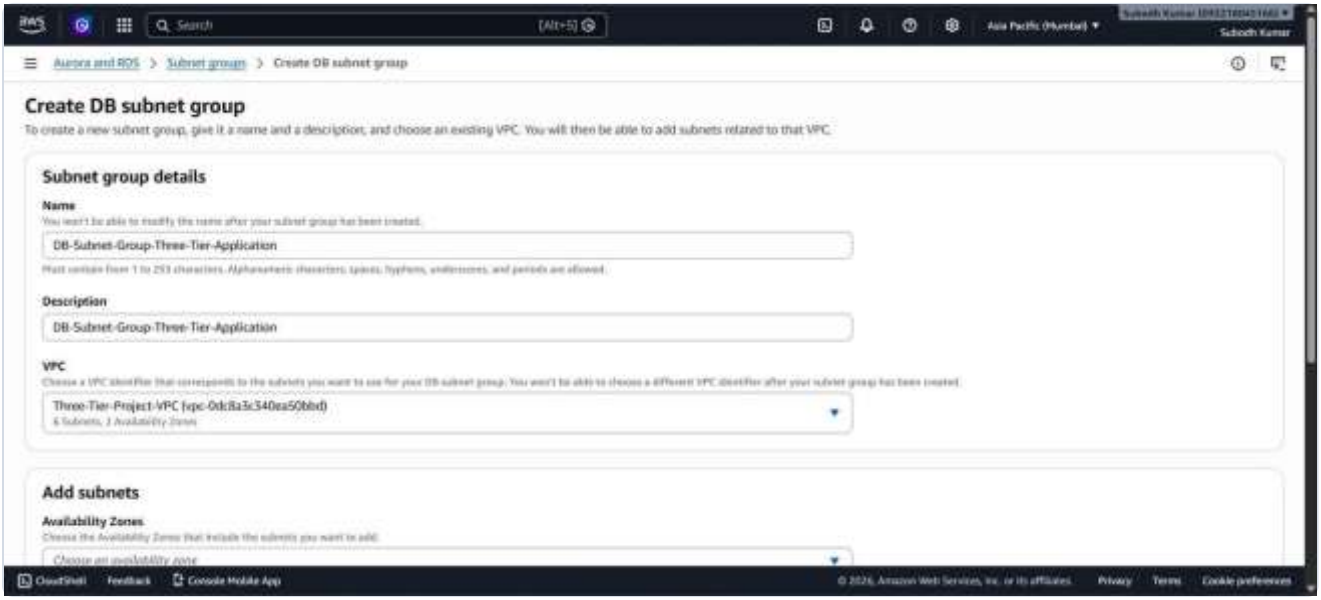
Cool!

Now search for **RDS** in the **search bar**. In the Amazon RDS go to **Subnet groups > Create DB subnet group.** Setup it as follows.



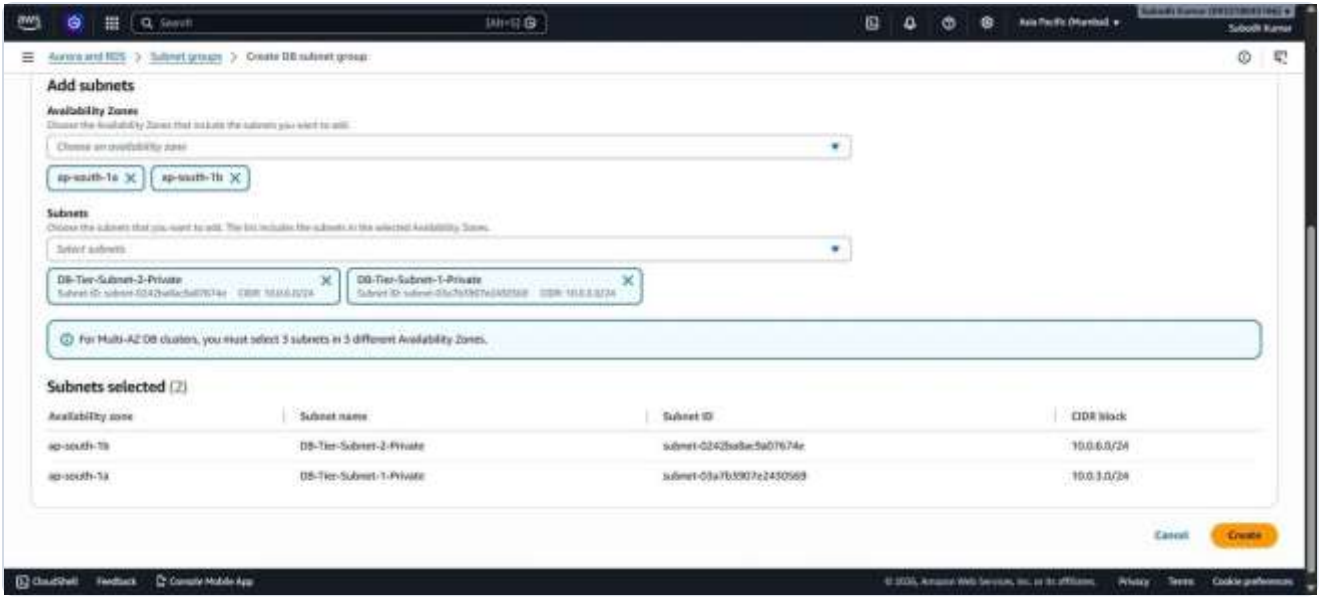






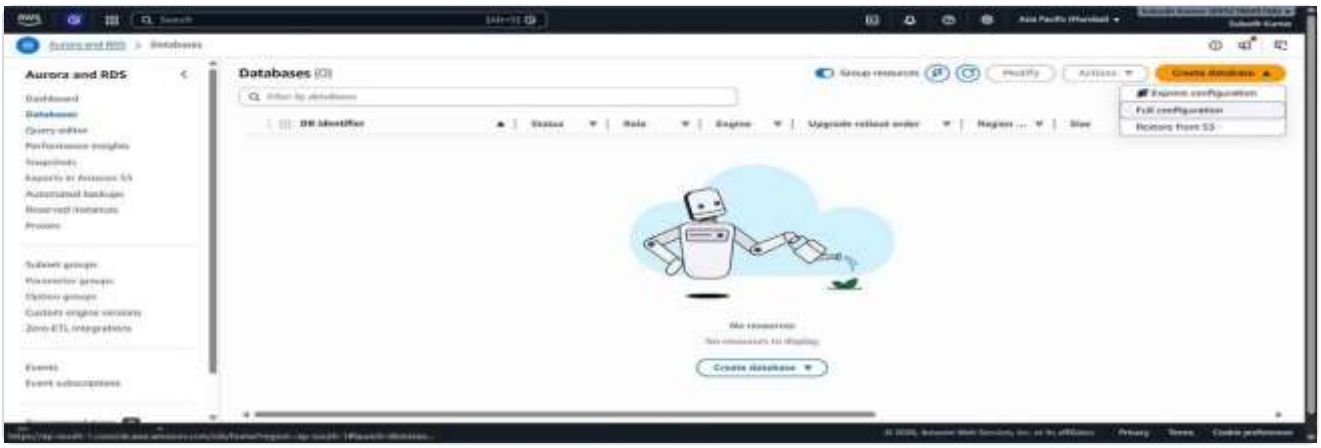
Choose **Availability Zones** that show in the diagram (**1a, 1b**).

Choose **Subnet** which are **assigned** to the **DB** (**10.0.3.0/24, 10.0.6.0/24**).

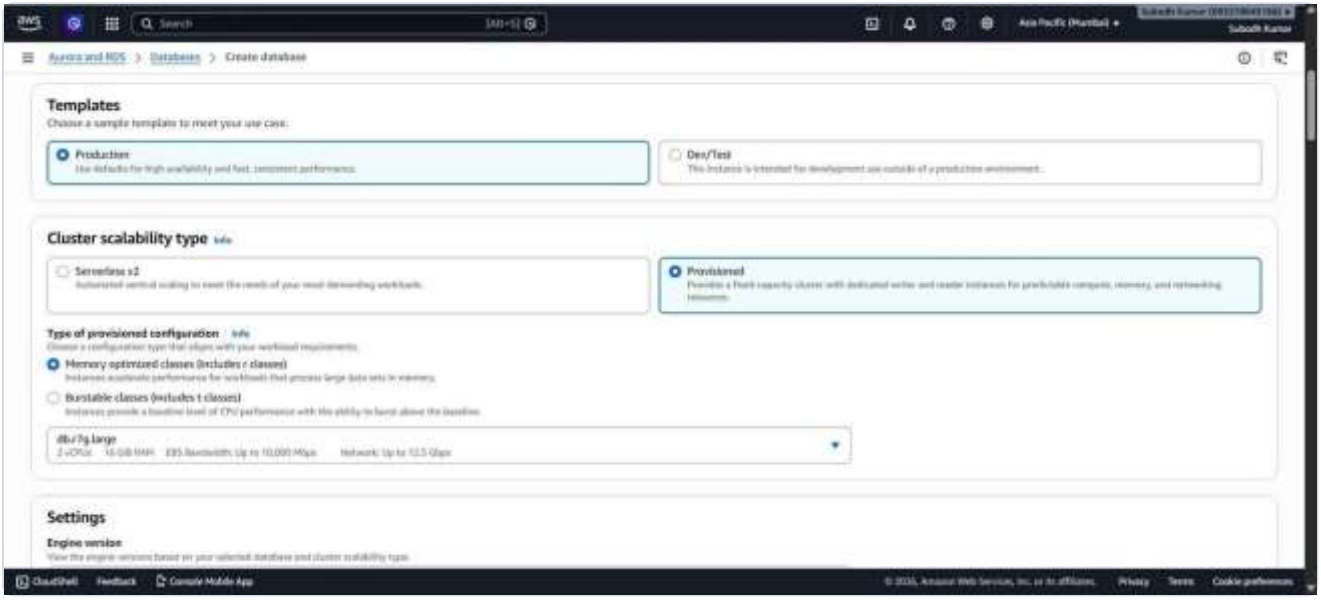


Now go to Amazon RDS **Databases > Create database**. Setup it as follows (In our application we have use MySQL therefore we use **Aurora (MySQL Compatible)** option. We can use MySQL as well).



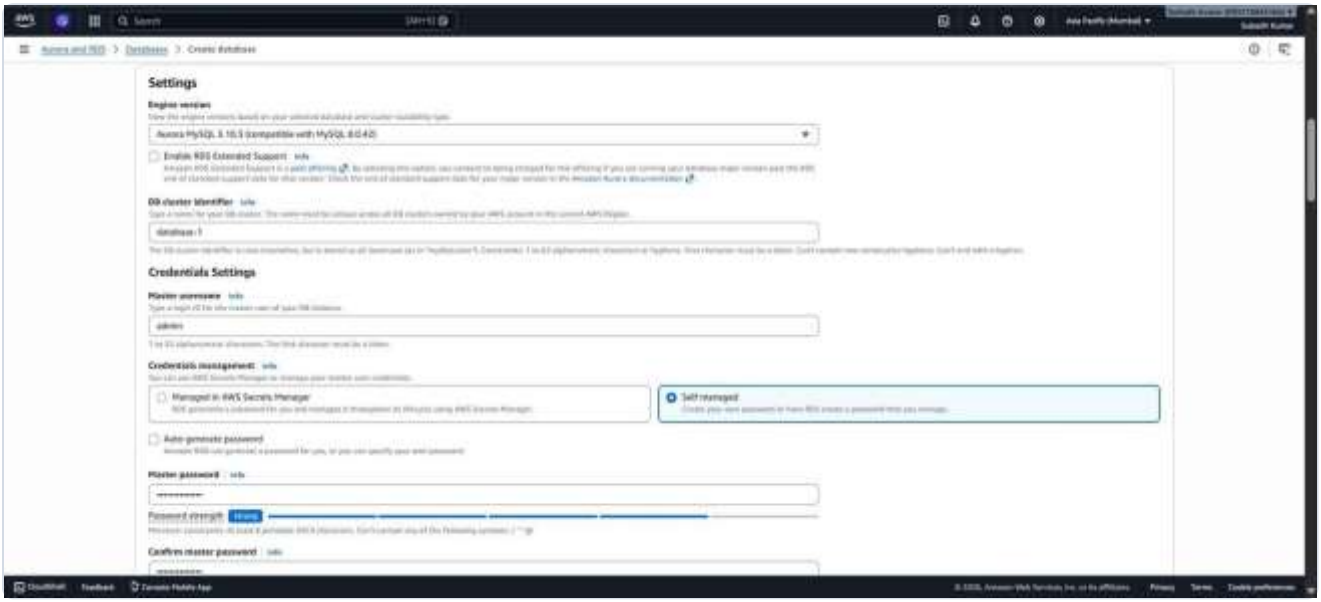


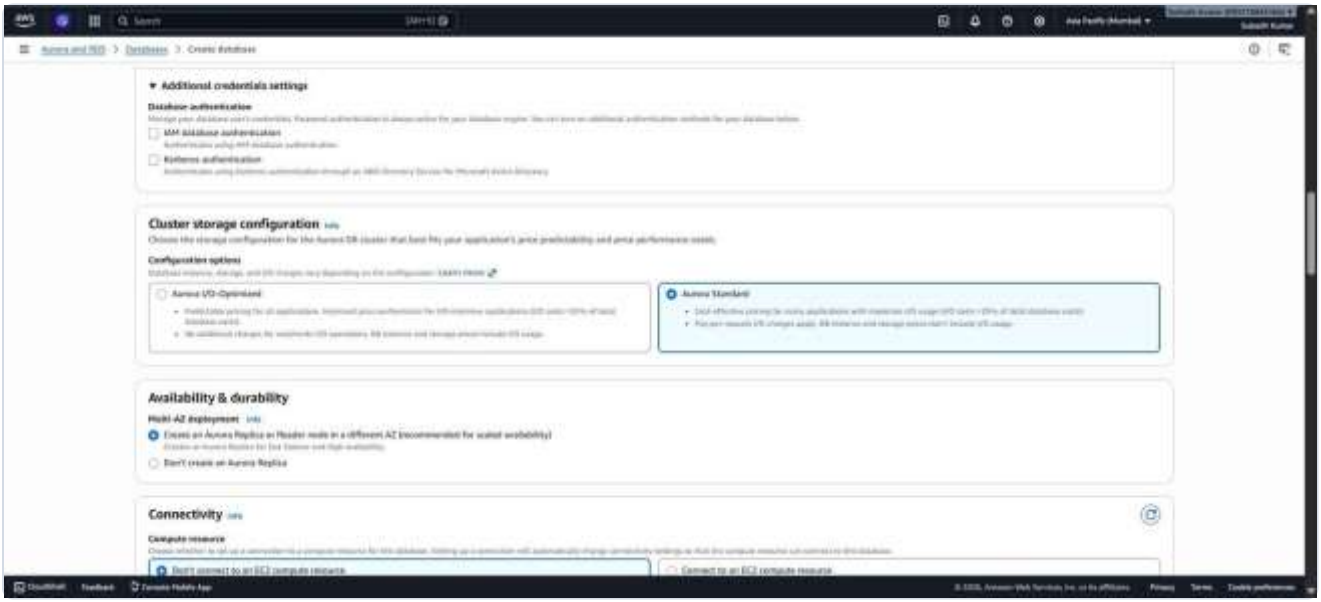


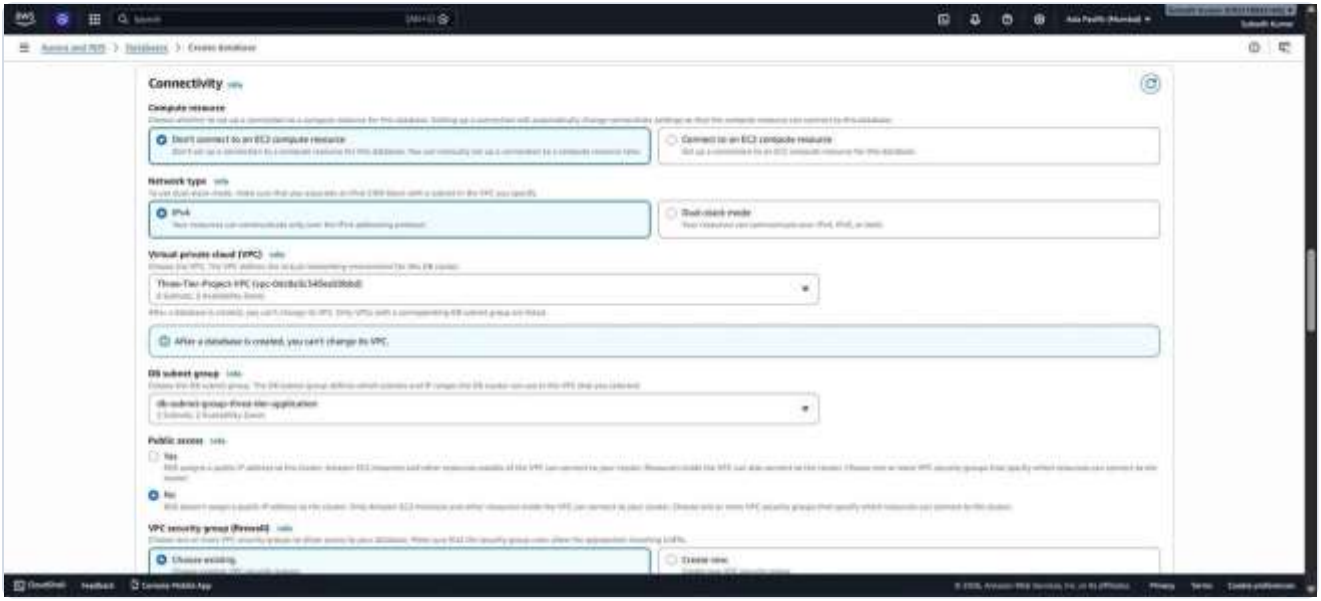


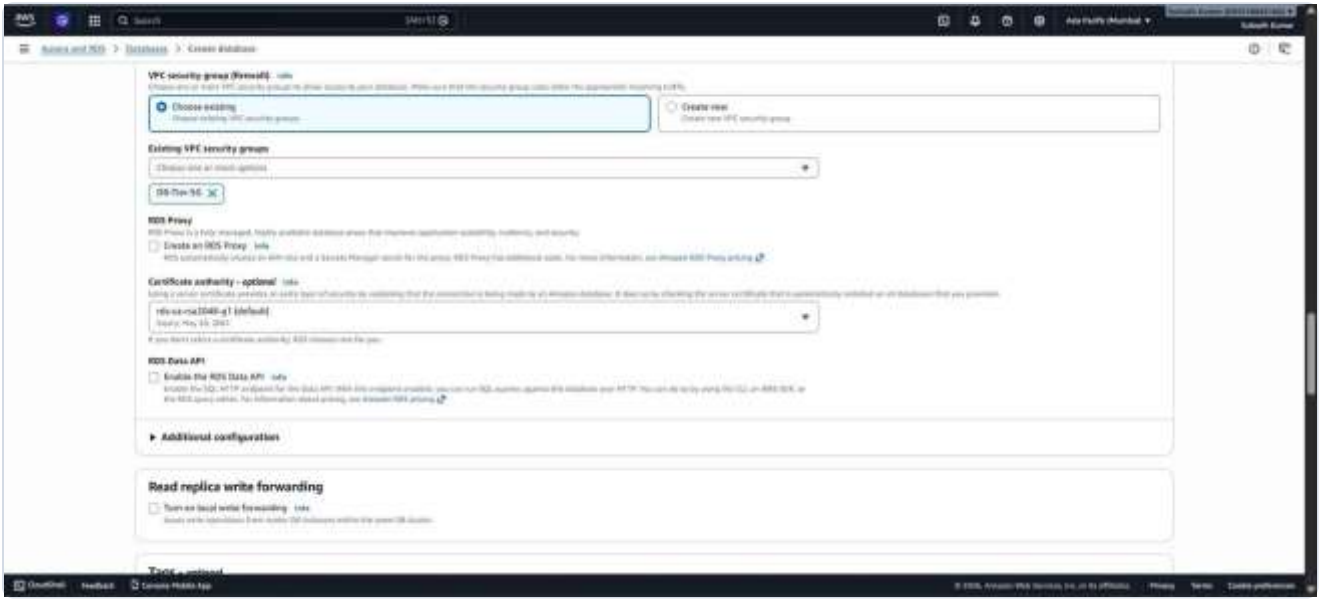
In the following step, remember the username and password. We require them when we access the DB server in the upcoming steps.

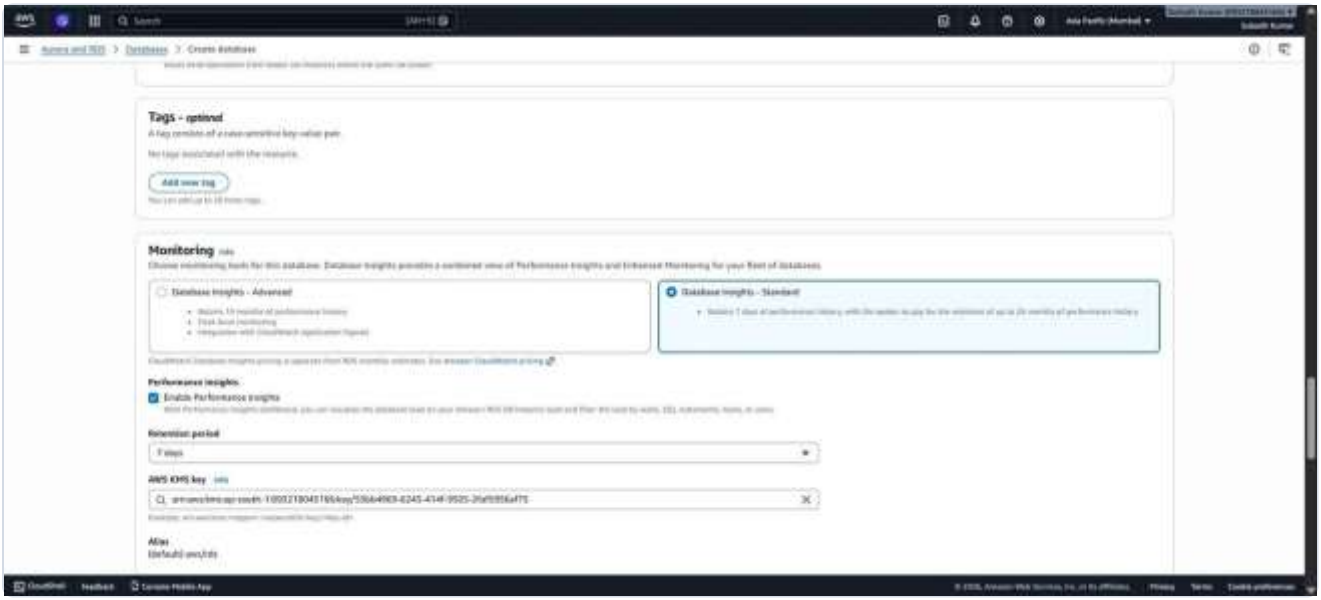
(Note down your master username and master Password)




Additional credentials settings
Cluster storage configuration
Availability & durability
Connectivity


Connectivity
Network type
Virtual private cloud (VPC)
DB subnet group
Public access
VPC security group (firewall)

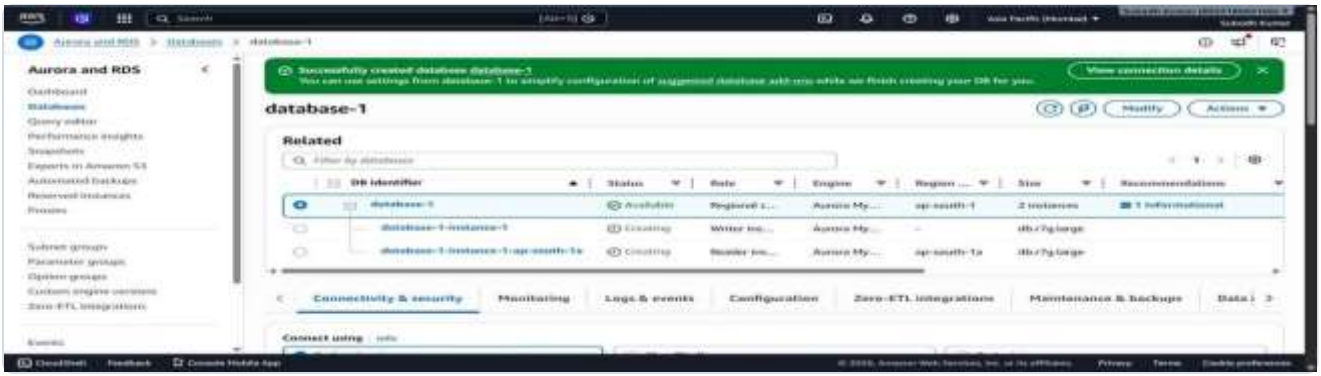

VPC security group (firewall)
Existing VPC security groups
RDS Proxy
Certificate authority - optional
RDS Data API
Additional configuration
Read replica write forwarding





Keep other settings as default. This DB Server creation will cost you some money. But since we delete as soon as project is implemented, it will be few dollars.

Now Click **Create database**. It will take some time to create the server.



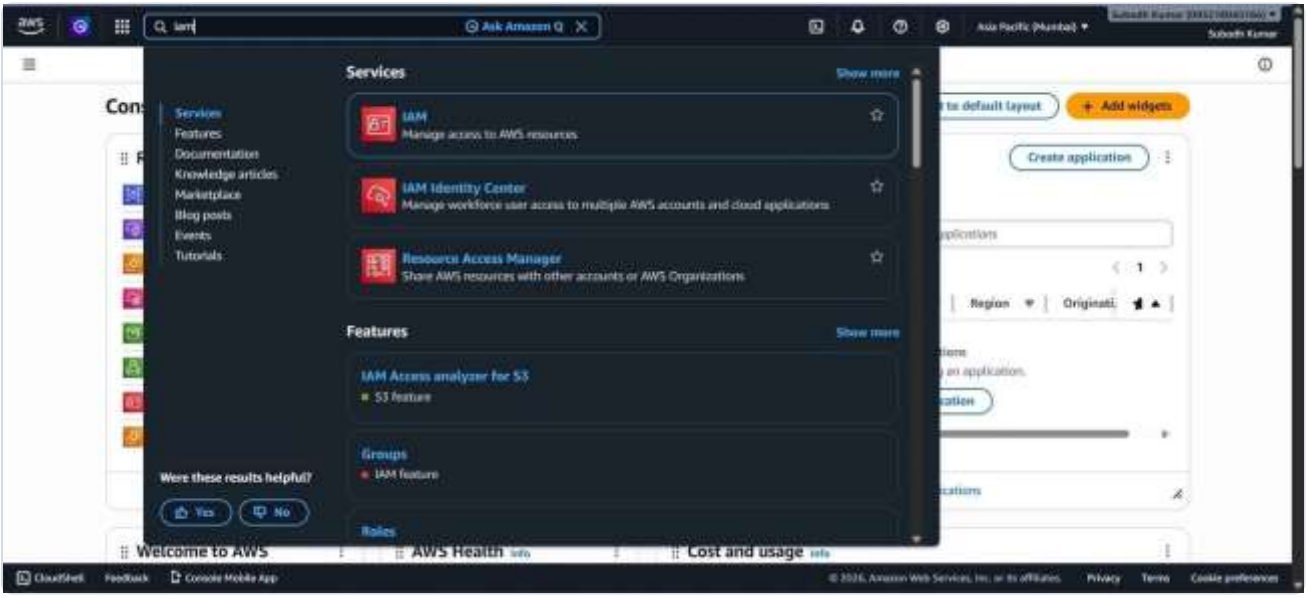
Great we created the database server

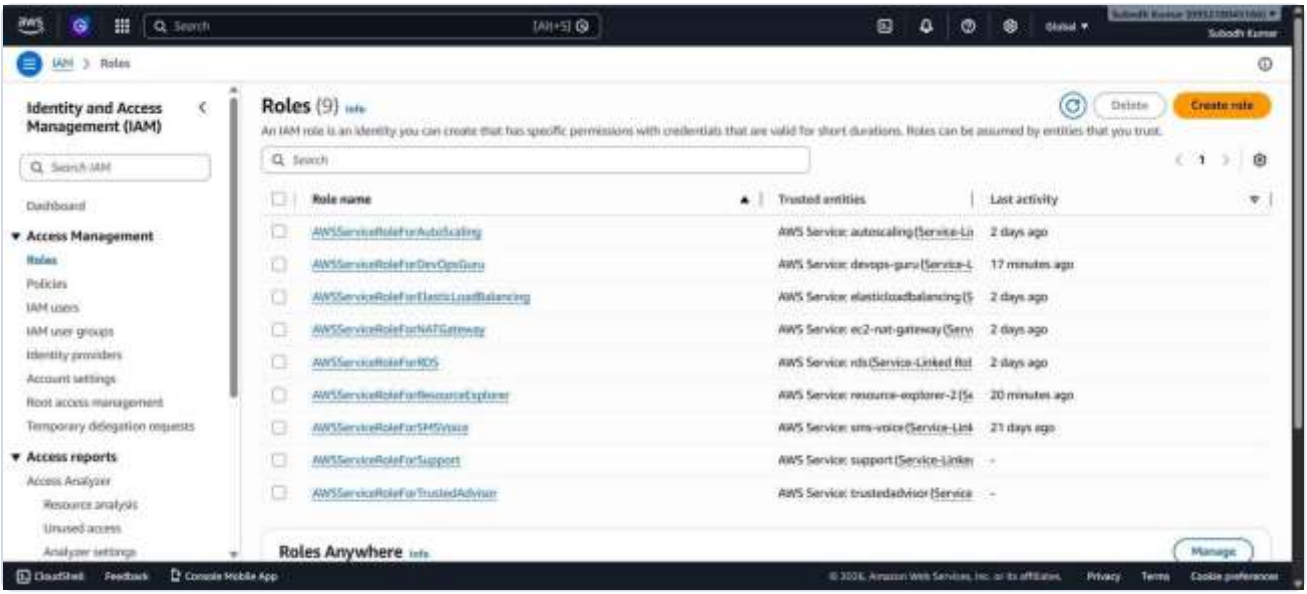
Step 5 Create IAM Role

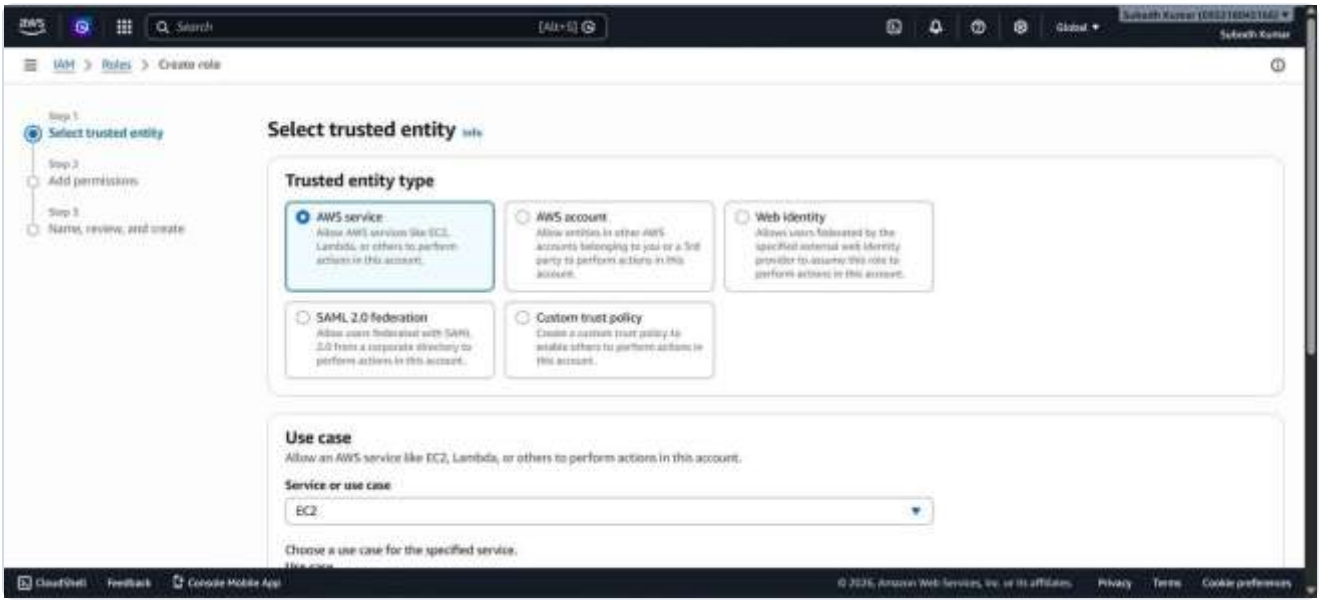
Our application should be Highly secured. In IAM Role we have to use two permission policies in upcoming steps. **S3 read only Access** for access the S3 bucket and **SSM managed instance core** for grants the necessary permissions for an EC2 instance to communicate with the Systems Manager service.

Let's create them

Search for IAM in the search bar. Go to **Roles > Create role**

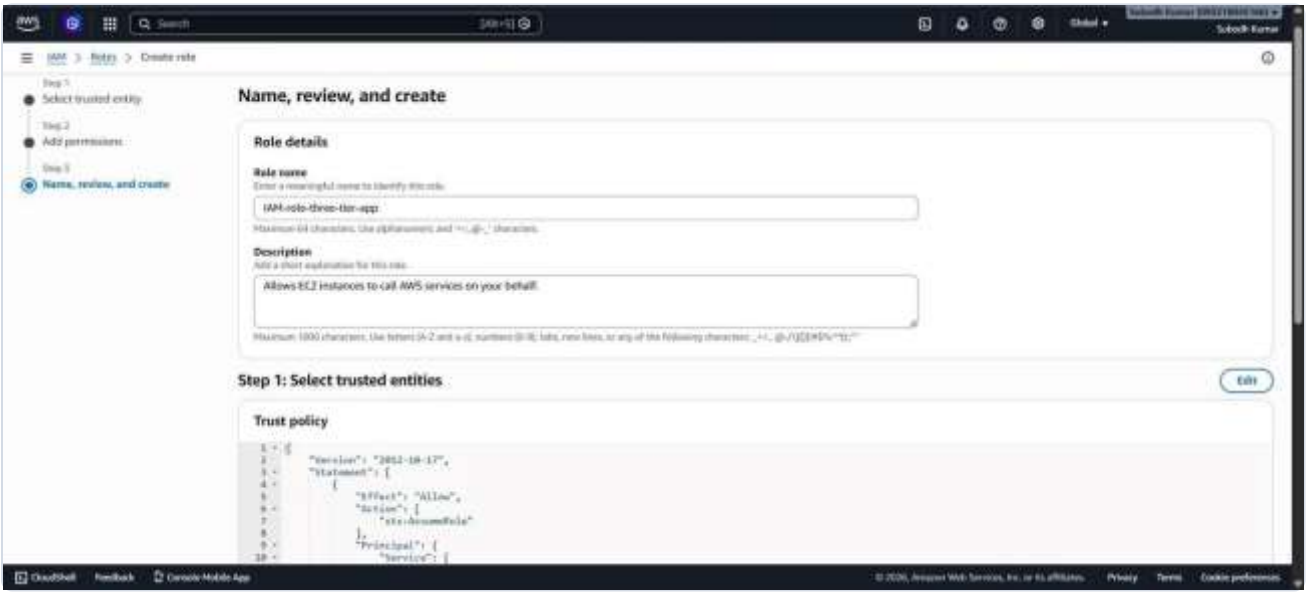


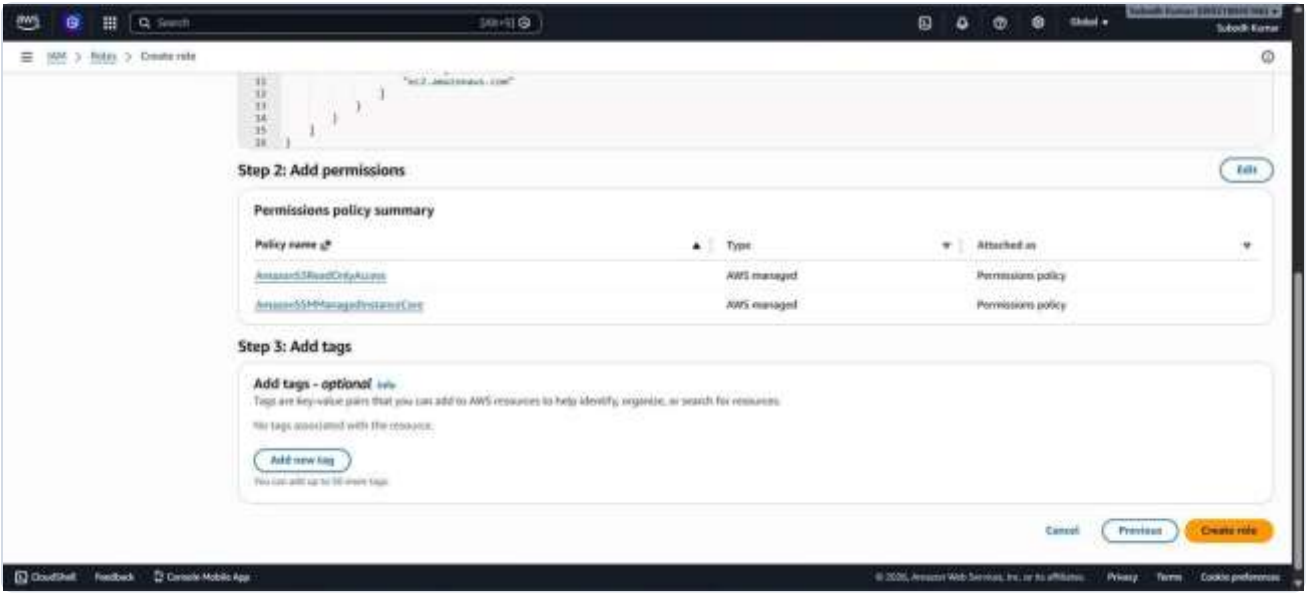



Select trusted entity
Trusted entity type
AWS service
AWS account
Web identity
SAML 2.0 federation
Custom trust policy
Use case
Allow an AWS service like EC2, Lambda, or others to perform actions in this account.
Service or use case
EC2
Choose a use case for the specified service.


Add permissions
Permissions policies (1/1147)
Filter by Type
Policy name
Type
AWS managed
Set permissions boundary - optional


Add permissions
Permissions policies (2/1147)
Filter by Type
Policy name
Type
AWS managed
Set permissions boundary - optional

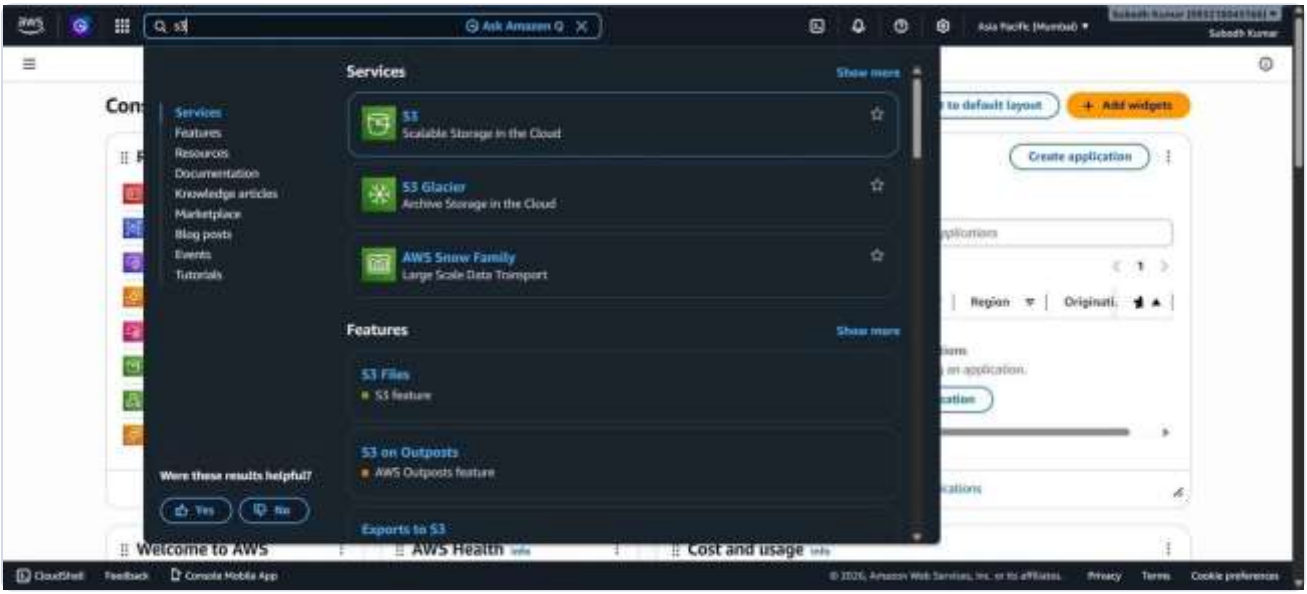


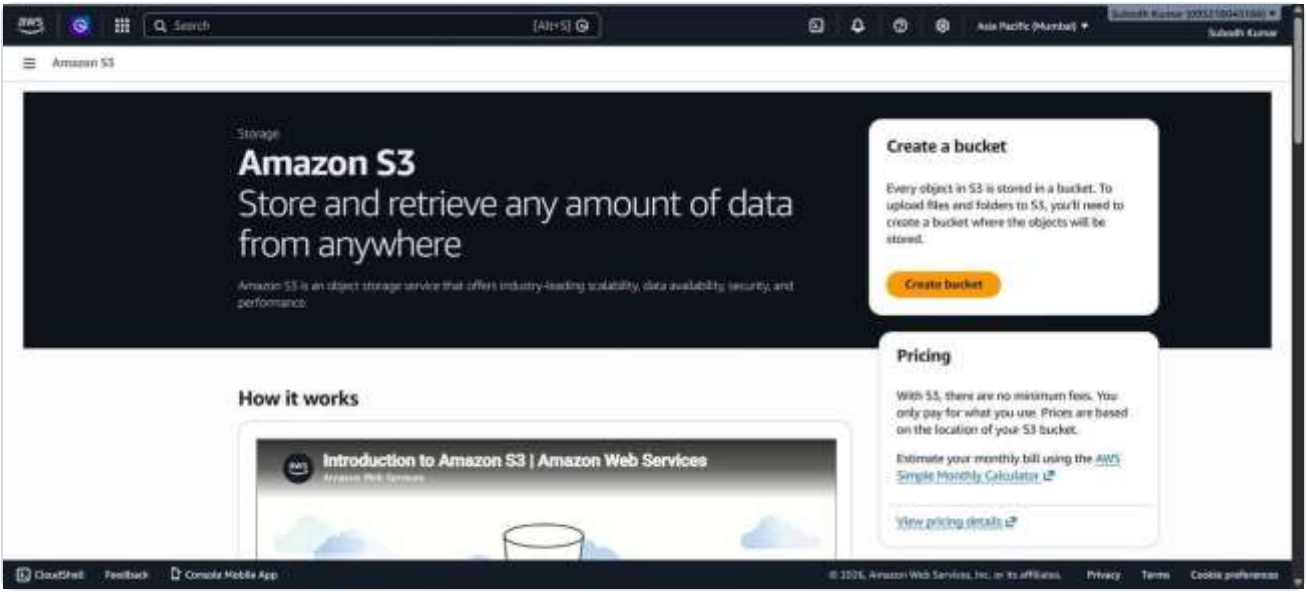


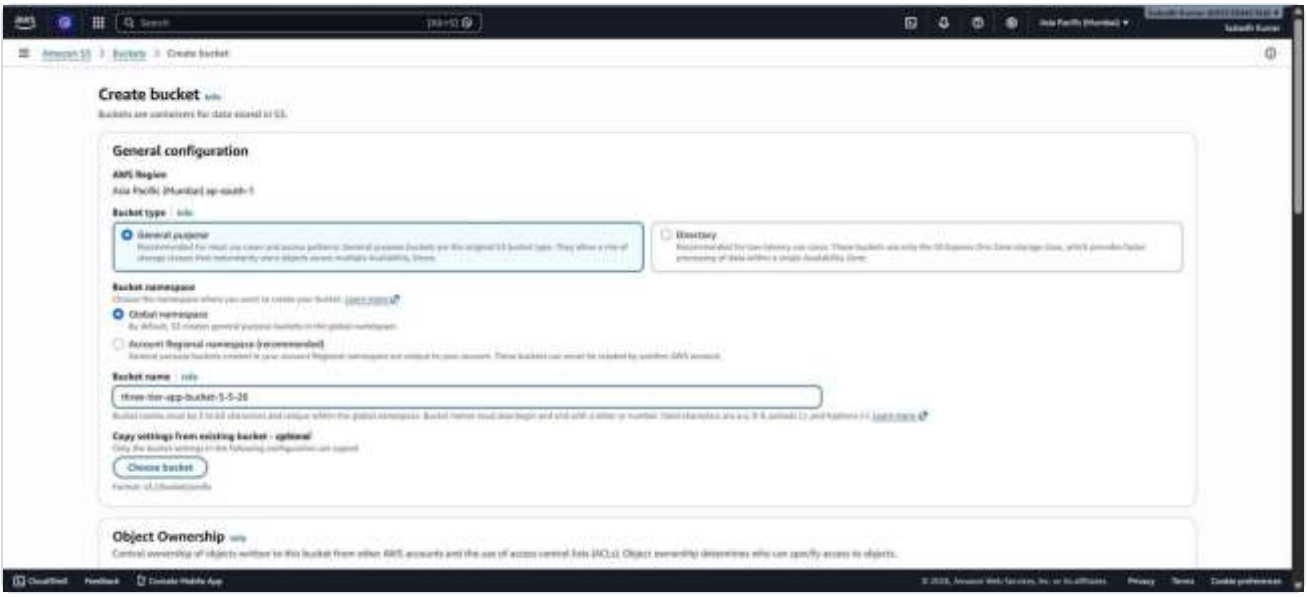
Step 6 Create an S3 Bucket and Upload the Source Code

When we setup application on servers it is easy to if the source codes are in a s3 bucket

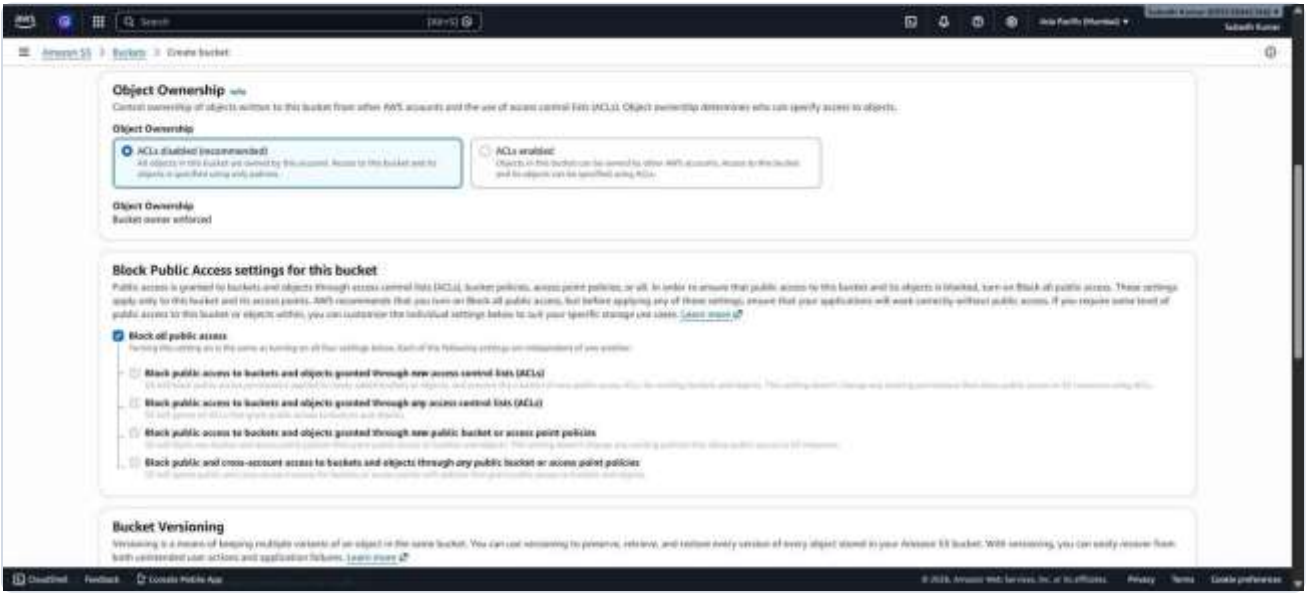
Go to **search bar** and search for **S3**. Then click **create bucket.** Bucket name should be unique one.

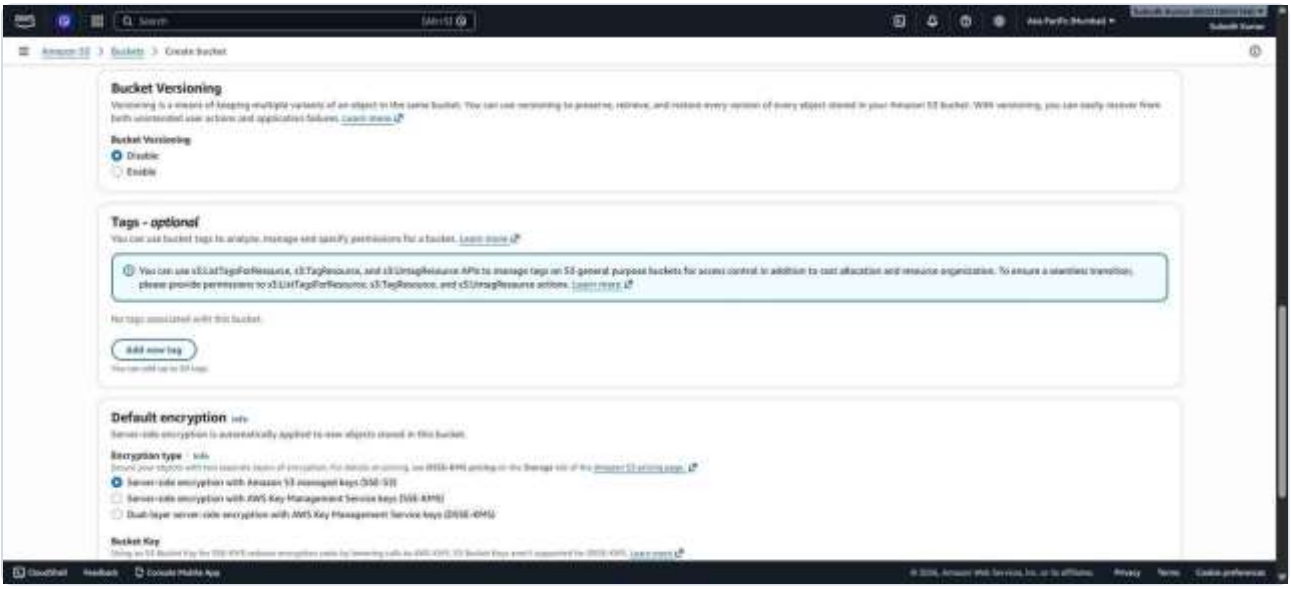


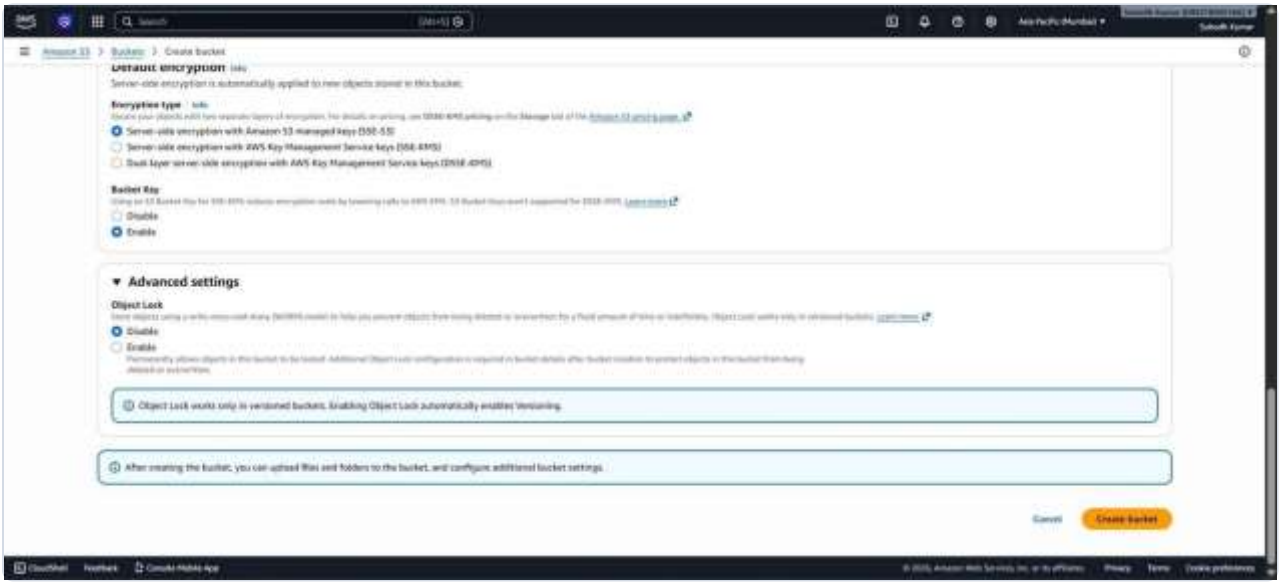




Let all other options default and click create bucket.



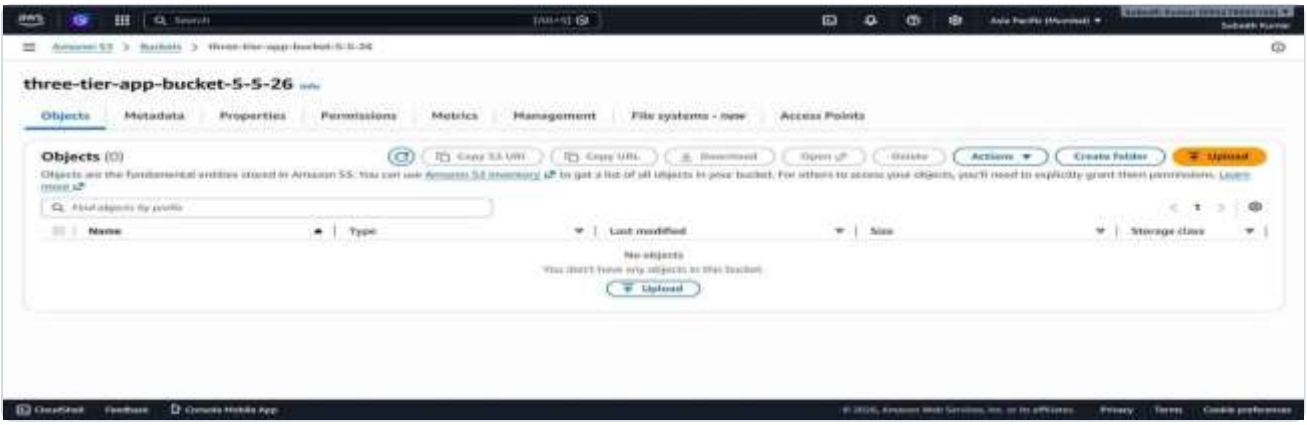


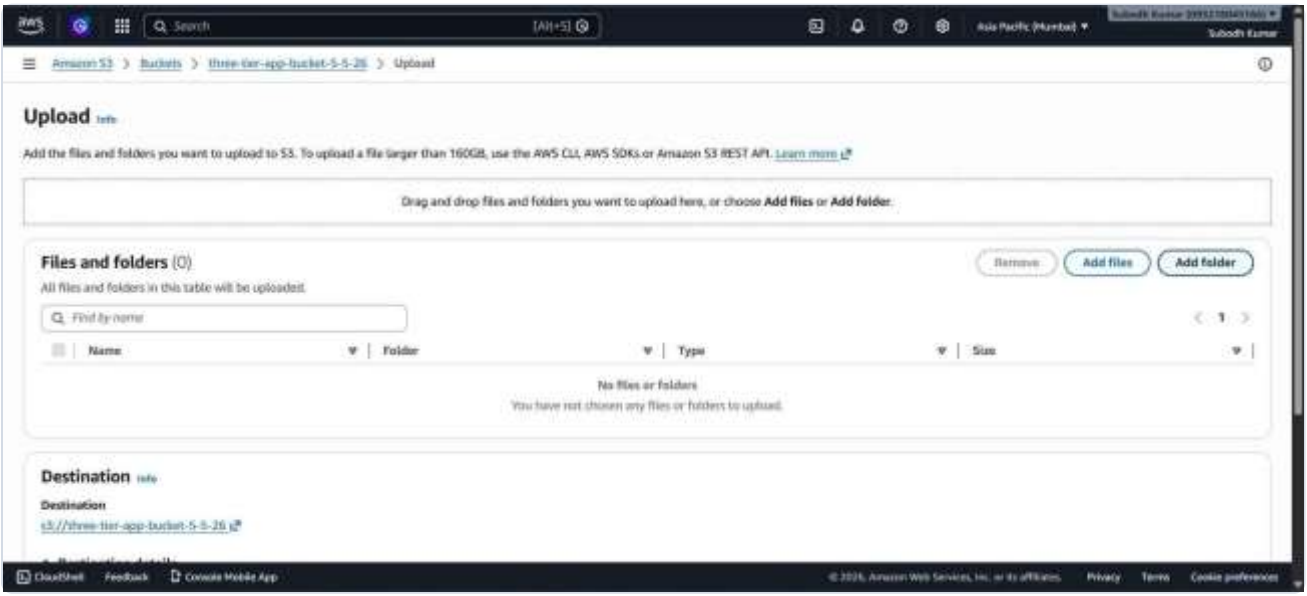


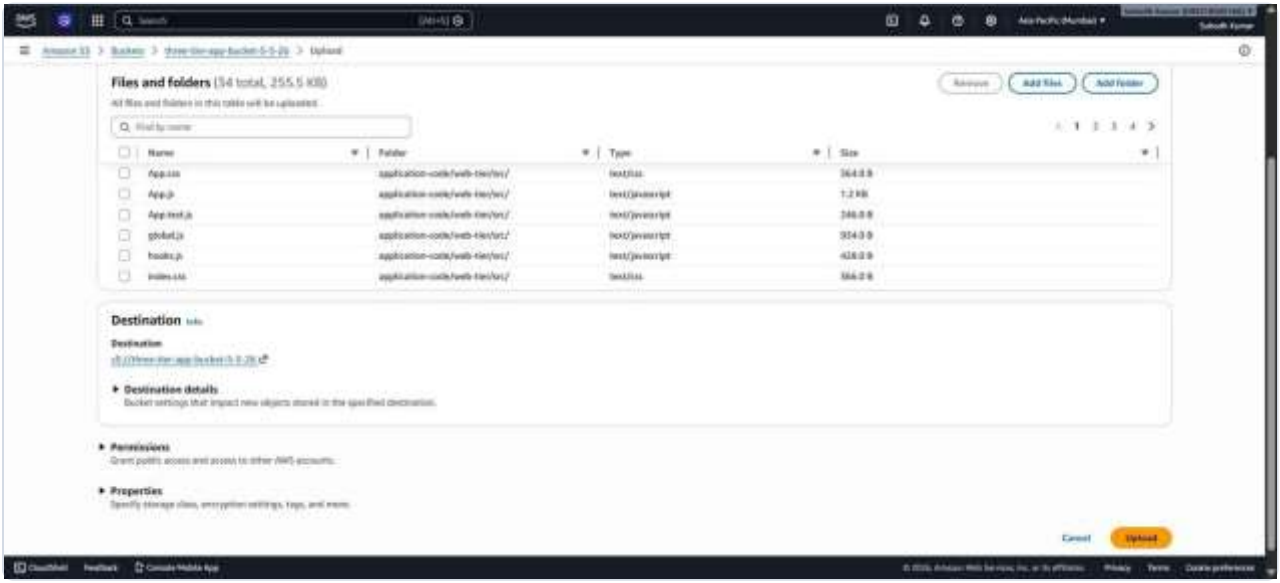
Now go to the following repository and clone it in your pc. It contains all source code.

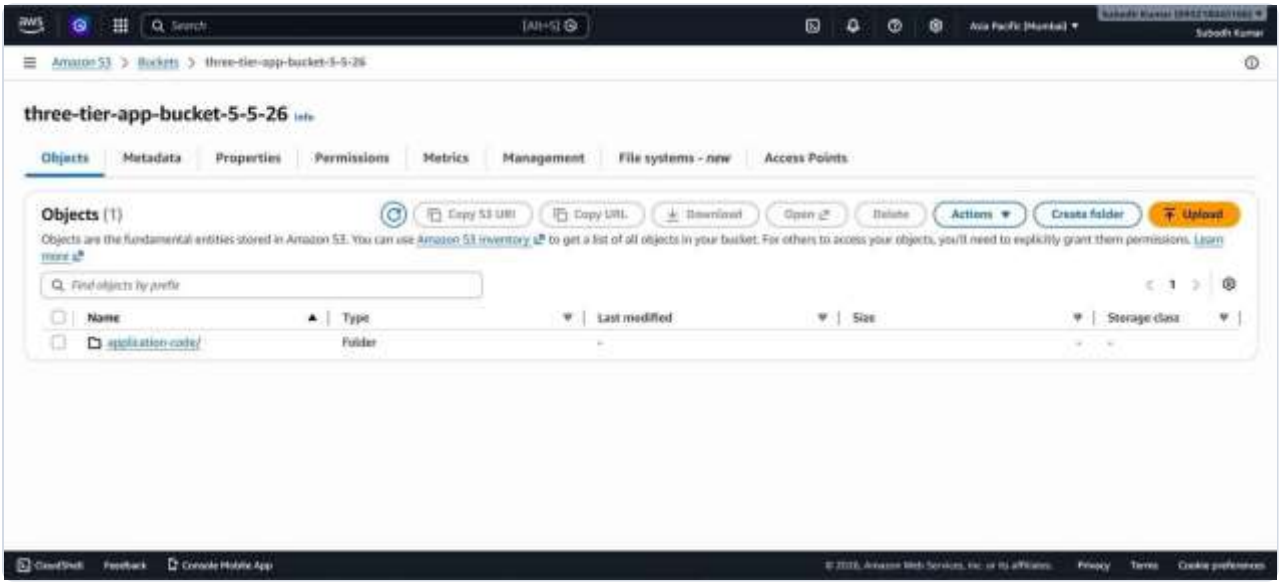
https://github.com/sukhada30github/Three-Tier-Project.gihttps://github.com/dhruvsonwane13-ship-it/Three-tier-project.git t

Now go to the bucket and upload the application-code folder. There are some changes to make in the files. We will do it in the upcoming steps.









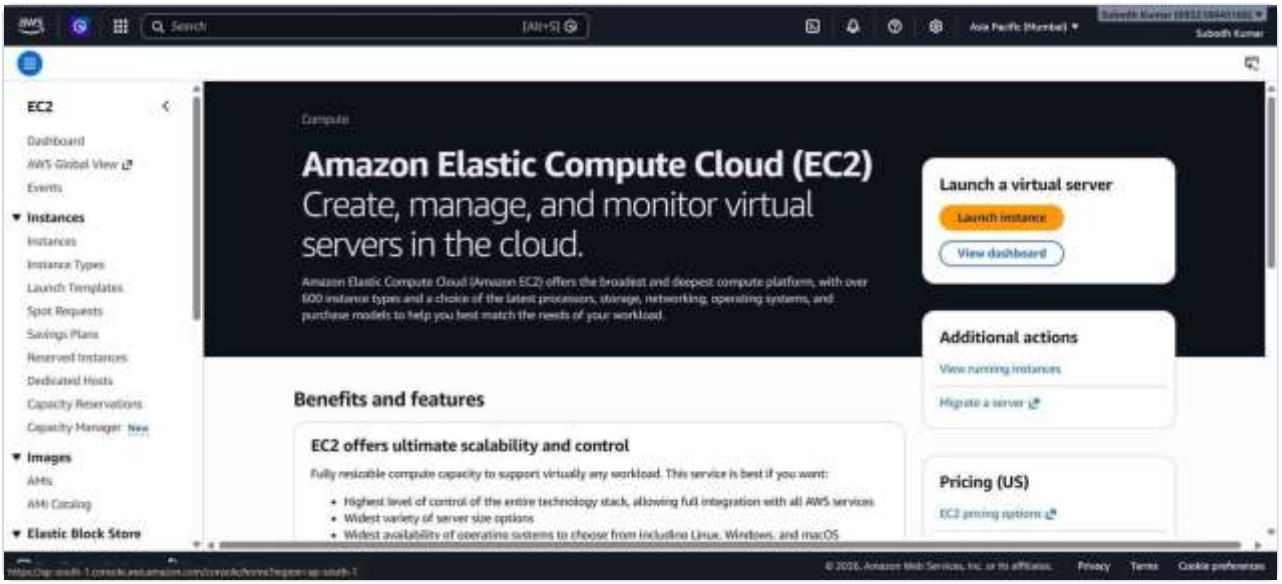
Our application should be highly scalable. To achieve this we use Auto Scaling Groups. To create servers and their configuration automatically we use Launch Templates, so we need Amazon Machine Images (AMIs) of our servers. For that we create a test server using an EC2 instance, configure the application on it and then create an AMI from it. In this way we implement our Web Servers and App Servers.

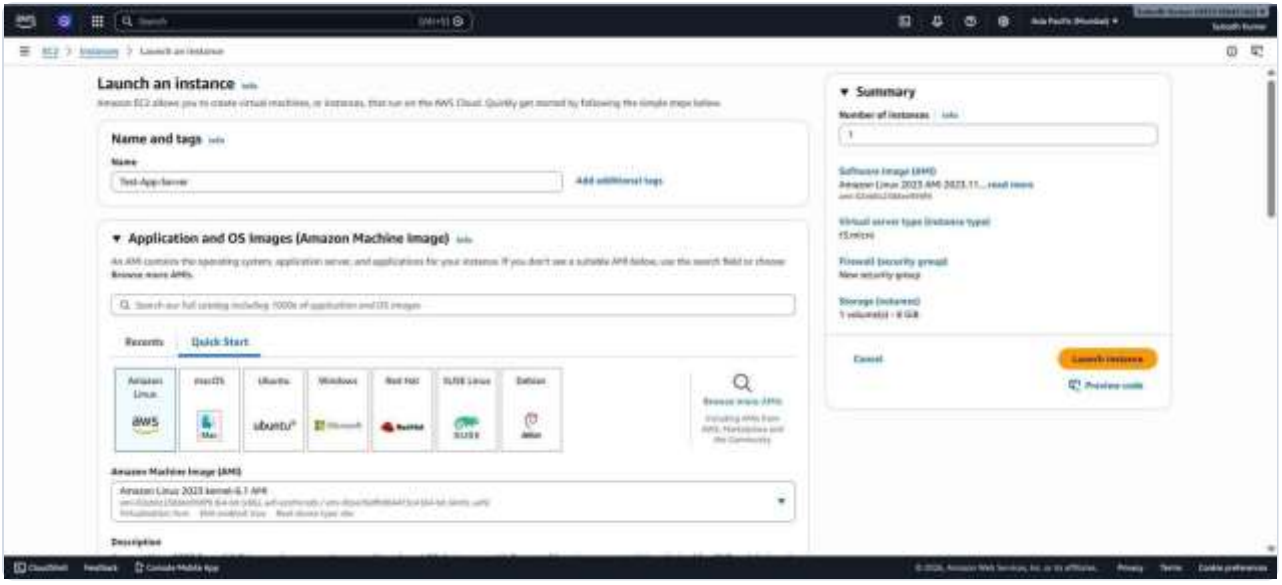
Let's do

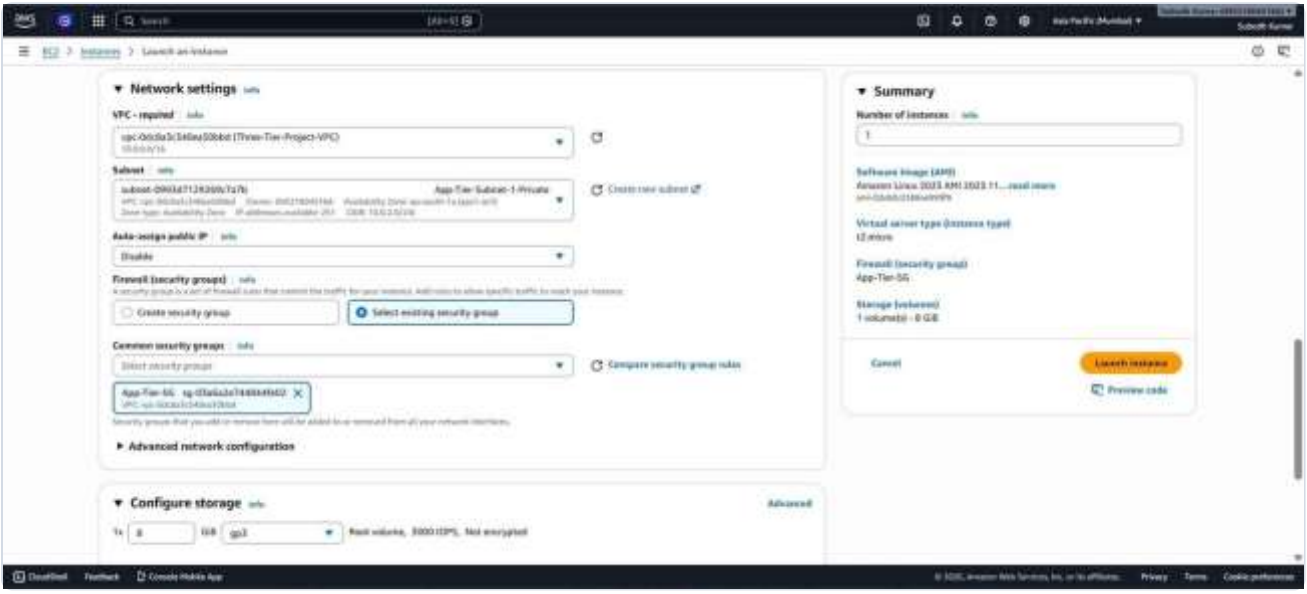
Step 7 Create Test App Server and Set It Up

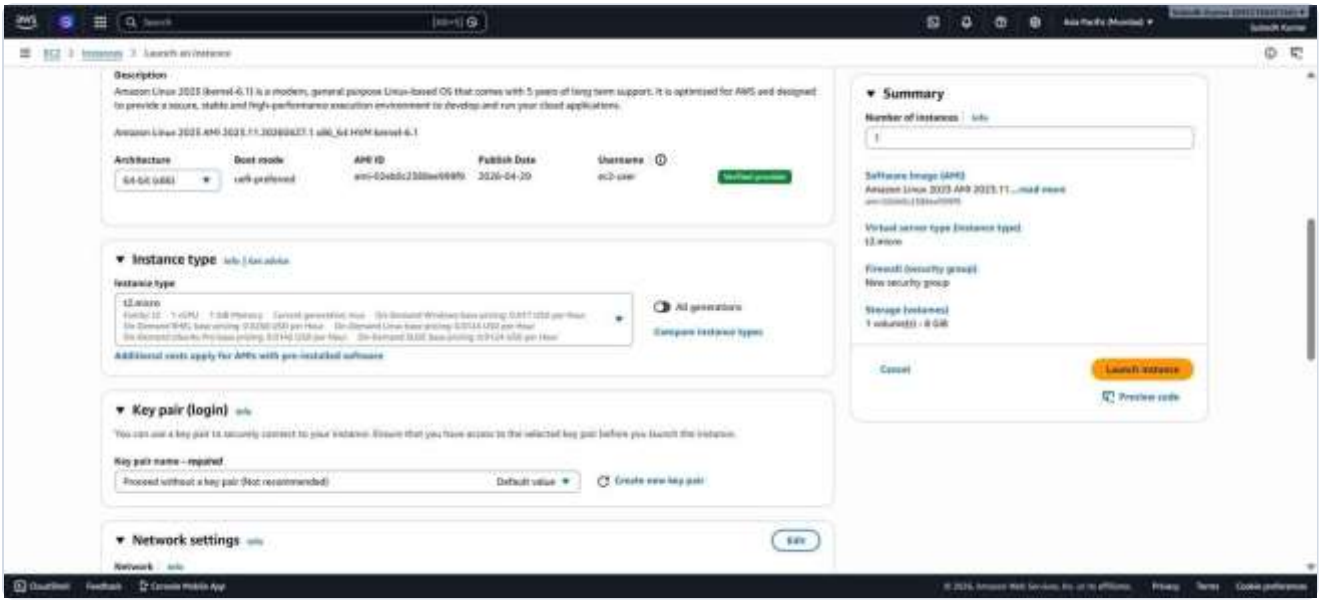
Search for EC2 in the search bar. Go to **Instances > Launch instances.** Setup it as follows.

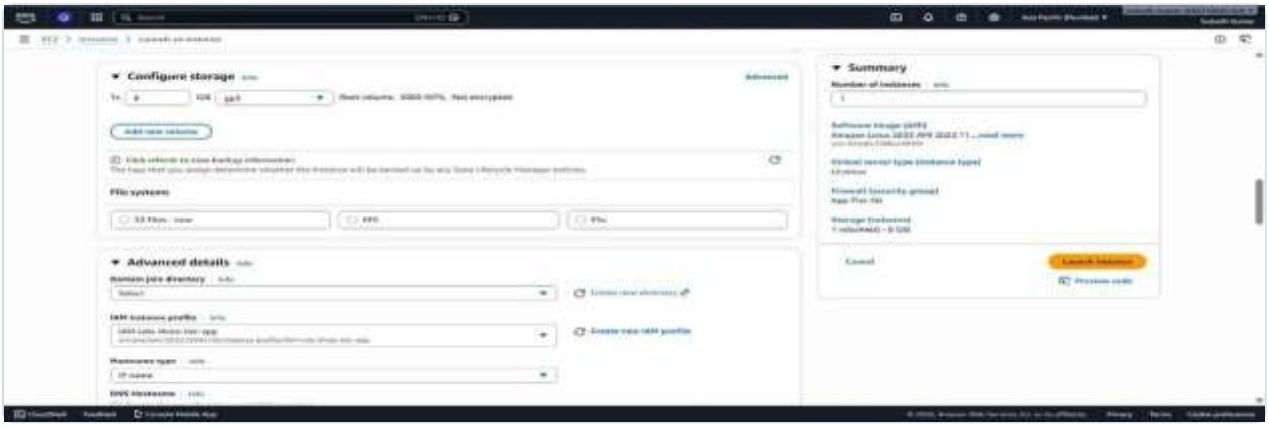



EC2
Dashboard
AWS Global View
Events
Instances
Instances
Instance Types
Launch Templates
Spot Requests
Savings Plans
Reserved Instances
Dedicated Hosts
Capacity Reservations
Capacity Manager
Images
AMIs
AMI Catalog
Elastic Block Store
Amazon Elastic Compute Cloud (EC2)
Create, manage, and monitor virtual servers in the cloud.
Launch a virtual server
Launch instance
View dashboard
Additional actions
View running instances
Migrate a server
Pricing (US)
Benefits and features
EC2 offers ultimate scalability and control


Launch an instance
Name and tags
Application and OS images (Amazon Machine Image)
Summary


Network settings
Summary
Configure storage
Advanced network configuration

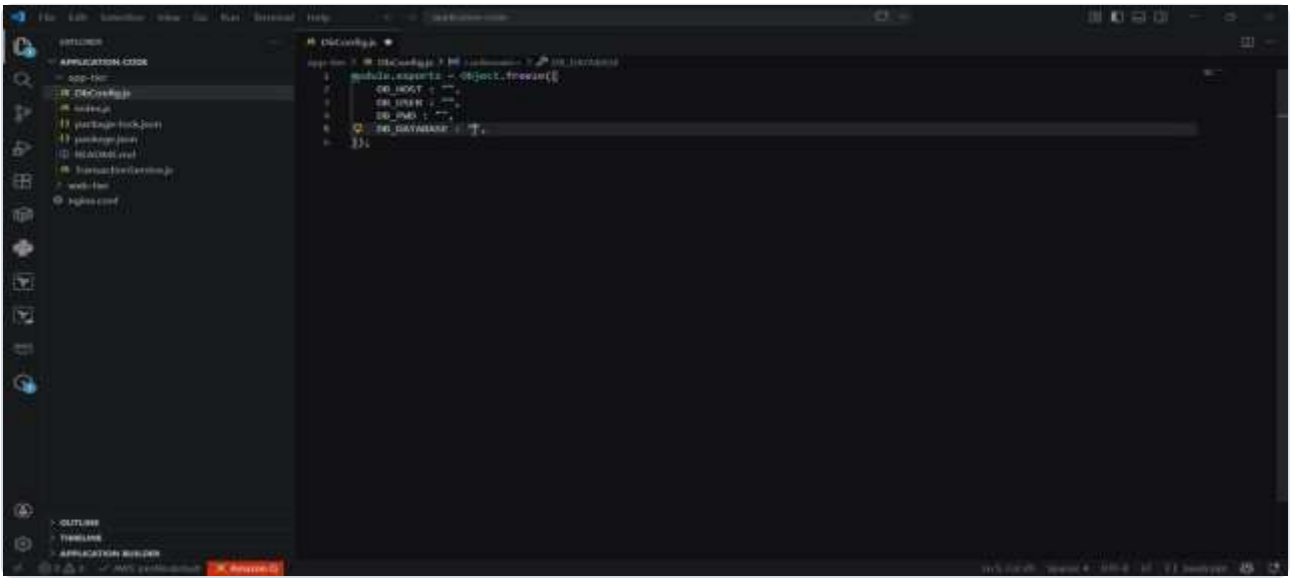




Keep other settings as default and click Launch Instance.

Now we have to connect to the server and configure the application tier.

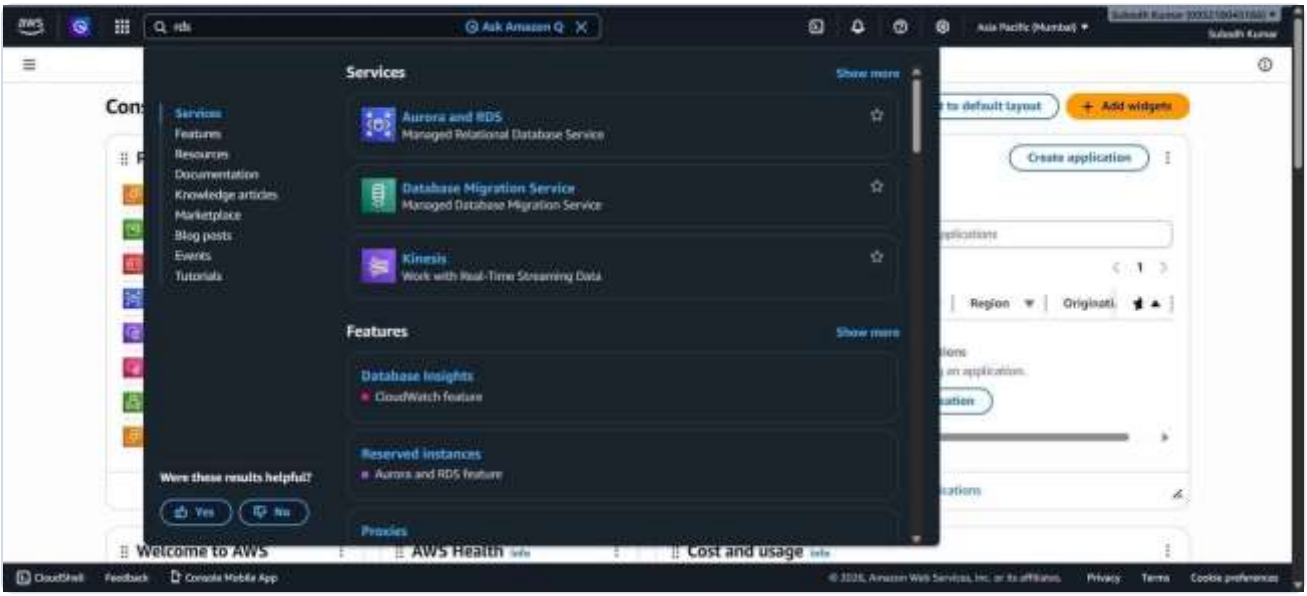
Before that let's do some changes in the source code. Go to the source code folder on your pc. Go to **application-code > app-tier.** Open **DbConfig.js** with your favorite editor. I use vs code.

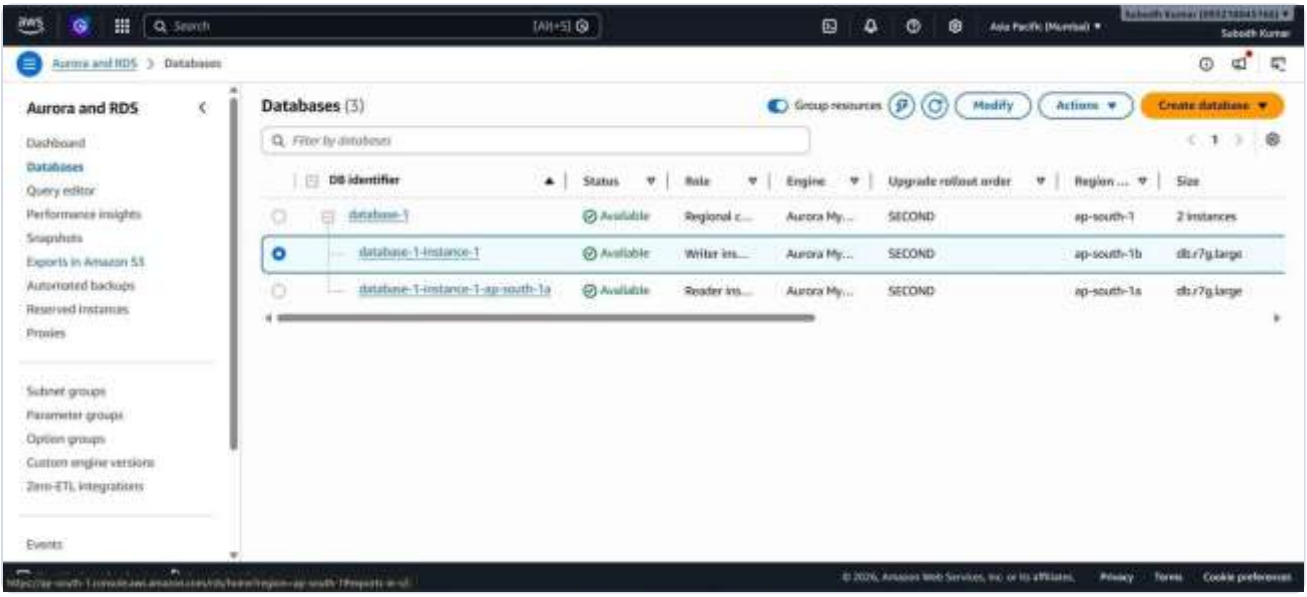


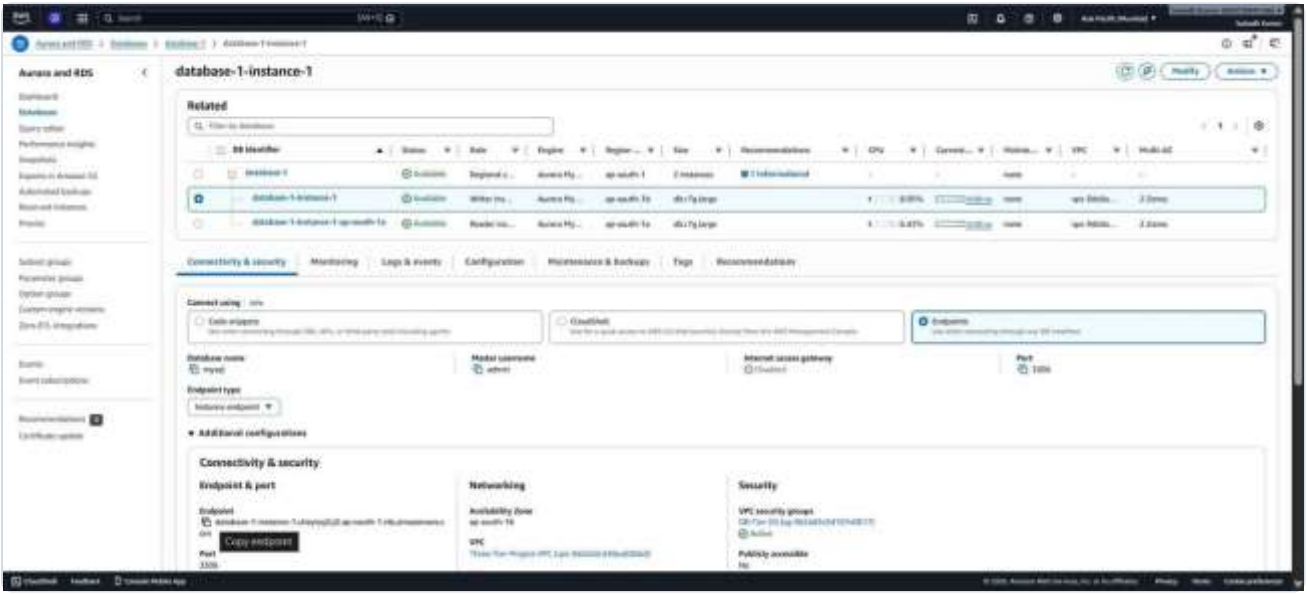
We have to change these values.

Let's see how to get the **DB_HOST**

Go to search and go to **RDS > Databases**. Select Writer instance. Under connectivity and security, you can find a value called Endpoint. Copy it and paste it as the value for DB_HOST.







DB_USER is the username for DB server (**admin**). **DB_PWD** is the password we assigned (**Your master password**). **DB_DATABASE** is **webappdb**.



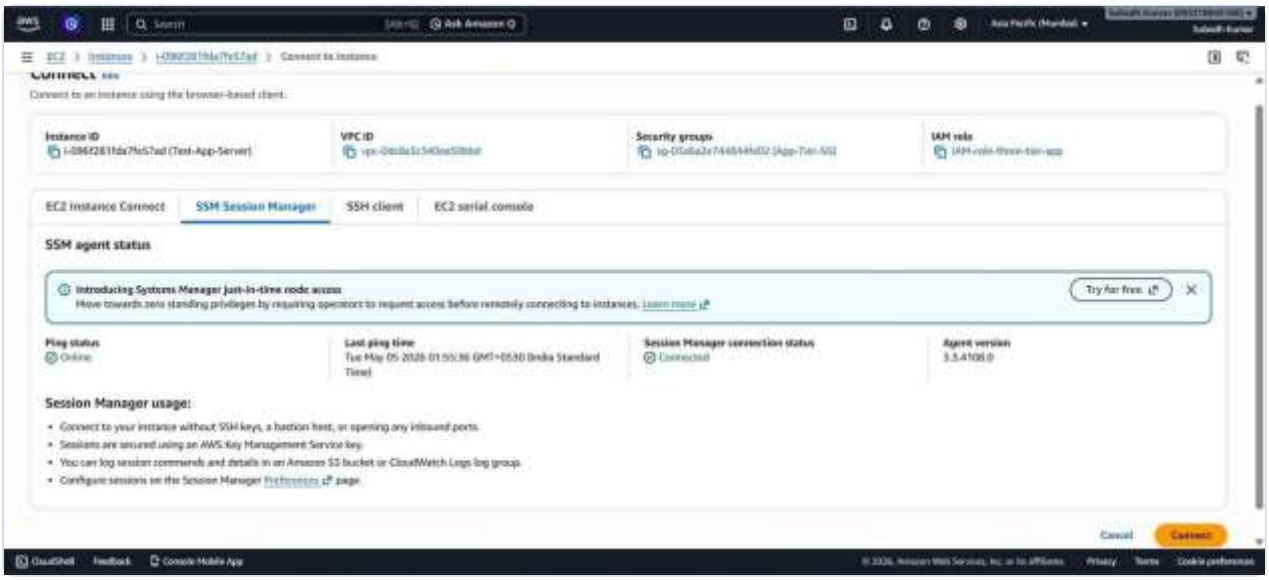
Now go to the S3 Bucket we created. Go to **application-code > app-tier** in the s3 bucket**.** Delete the DbConfig.js and then upload our newly updated DbConfig.js file.

Cool!

Now again go to EC2 instance and select Test-APP-Server we created click connect.



Go to Session Manager tab and click connect.



It will open a new tab like this. Which means you have connected to EC2 instance. IAM role we created let us connect to this EC2 instance even if it is in a private subnet.



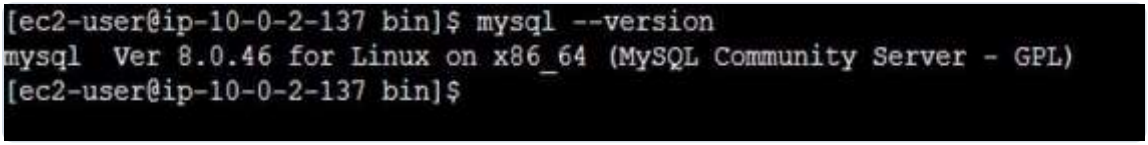
Now let’s configure the app-tier

First Switch user to the ec2-user by running sudo -su ec2-user

Now install the MySQL using following commands.

```
sudo wget https://dev.mysql.com/get/mysql80-community-release-el9-1.noarch.rpm
sudo dnf install mysql80-community-release-el9-1.noarch.rpm -y
sudo rpm --import https://repo.mysql.com/RPM-GPG-KEY-mysql-2023
sudo dnf install mysql-community-client -y
```

Run mysql --version to confirm it installed correctly



Now let's connect to Database server by running following command

```
mysql -h <RDS-Endpoint> -u <username> -p
```

RDS-Endpoint: Endpoint value of Writer instance copied from the RDS>Databases.

Username: admin

```
mysql -h database-1-instance-1.ctieyicq2cj3.ap-south-1.rds.amazonaws.com -u admin -p
```

Hit enter and input the password (**Your master password**). Enter. You will see the following output.



Let's Create the database. Run following commands.

```
create DATABASE webappdb;
show databases;
use webappdb;
```

```
mysql> create DATABASE webappdb;
Query OK, 1 row affected (0.00 sec)

mysql> show databases;
+--------------------+
| Database           |
+--------------------+
| information_schema |
| mysql              |
| performance_schema |
| sys                |
| webappdb           |
+--------------------+
5 rows in set (0.00 sec)

mysql> use webappdb;
Database changed
mysql>
```

Now Create a table by running following command

```
CREATE TABLE IF NOT EXISTS transactions(id INT NOT NULL AUTO_INCREMENT, amount DECIMAL(10,2),
description VARCHAR(100), PRIMARY KEY(id));
```

```
mysql> CREATE TABLE IF NOT EXISTS transactions(id INT NOT NULL
    -> AUTO_INCREMENT, amount DECIMAL(10,2), description
    -> VARCHAR(100), PRIMARY KEY(id));
Query OK, 0 rows affected (0.02 sec)

mysql> show tables;
+--------------------+
| Tables_in_webappdb |
+--------------------+
| transactions       |
+--------------------+
1 row in set (0.00 sec)

mysql>
```

Let’s create insert a sample value.

```
INSERT INTO transactions (amount, description) VALUES ('400', 'groceries');
```

```
SELECT * FROM transactions;
```

```
mysql> INSERT INTO transactions (amount,description) VALUES ('400','groceries');
Query OK, 1 row affected (0.00 sec)

mysql> SELECT * FROM transactions;
+----+--------+-------------+
| id | amount | description |
+----+--------+-------------+
|  1 | 400.00 | groceries   |
+----+--------+-------------+
1 row in set (0.00 sec)

mysql>
```

Type exit and hit Enter

Next we copy application tier codes from s3 bucket. Replace <S3_BUCKET_NAME> with your s3 bucket name.

```
cd /home/ec2-user
sudo aws s3 cp s3://<YOUR-S3-BUCKET-NAME>/application-code/app-tier app-tier --recursive
```

```
cd app-tier
sudo chown -R ec2-user:ec2-user /home/ec2-user/app-tier
sudo chmod -R 755 /home/ec2-user/app-tier
```

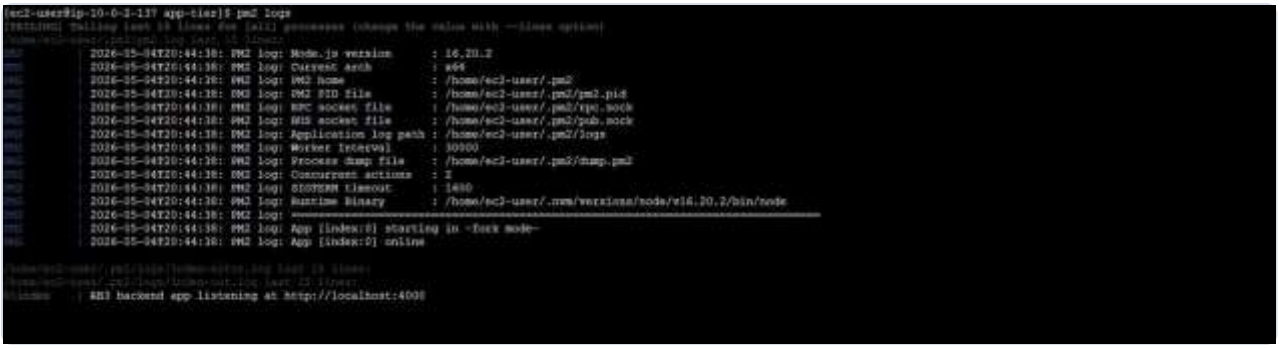
```
[ec2-user@ip-10-0-2-137 bin]$ cd /home/ec2-user
[ec2-user@ip-10-0-2-137 ~]$ sudo aws s3 cp s3://three-tier-app-bucket-5-5-26/application-code/app-tier app-tier --recursive
download: s3://three-tier-app-bucket-5-5-26/application-code/app-tier/README.md to app-tier/README.md
download: s3://three-tier-app-bucket-5-5-26/application-code/app-tier/DbConfig.js to app-tier/DbConfig.js
download: s3://three-tier-app-bucket-5-5-26/application-code/app-tier/package-lock.json to app-tier/package-lock.json
download: s3://three-tier-app-bucket-5-5-26/application-code/app-tier/package.json to app-tier/package.json
download: s3://three-tier-app-bucket-5-5-26/application-code/app-tier/index.js to app-tier/index.js
download: s3://three-tier-app-bucket-5-5-26/application-code/app-tier/TransactionService.js to app-tier/TransactionService.js
[ec2-user@ip-10-0-2-137 ~]$ cd app-tier
[ec2-user@ip-10-0-2-137 app-tier]$ sudo chown -R ec2-user:ec2-user /home/ec2-user/app-tier
[ec2-user@ip-10-0-2-137 app-tier]$ sudo chmod -R 755 /home/ec2-user/app-tier
[ec2-user@ip-10-0-2-137 app-tier]$
```

Now let’s install Node.js

```
curl -o- https://raw.githubusercontent.com/nvm-sh/nvm/v0.39.7/install.sh | bash
source ~/.bashrc
nvm install 16
nvm use 16
npm install -g pm2
npm install
npm audit fix
```

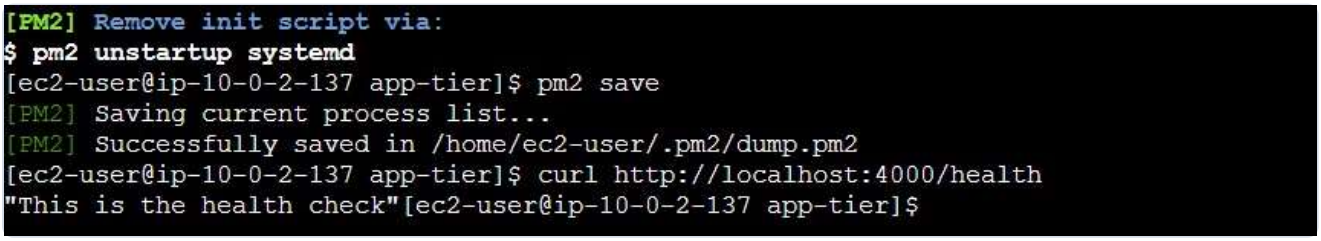
After that, let’s run the index.js file and then save the configuration.

```
pm2 start index.js
pm2 logs
```



If you got output like this. The backend is connected to port 4000. Press Ctrl+c to exit the loop. Then run the following commands.

```
pm2 startup
sudo env PATH=$PATH:/home/ec2-user/.nvm/versions/node/v16.20.2/bin /home/ec2-user/.nvm/versions/node/v16.20.2/lib/node_modules/pm2/bin/pm2 startup systemd -u ec2-user --hp /home/ec2-user
pm2 save
curl http://localhost:4000/health
```

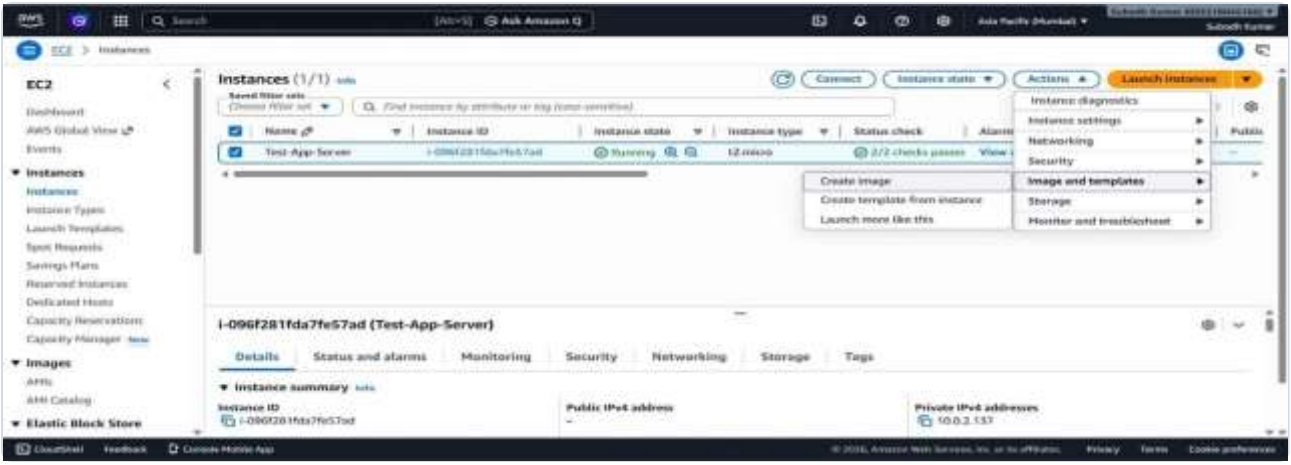


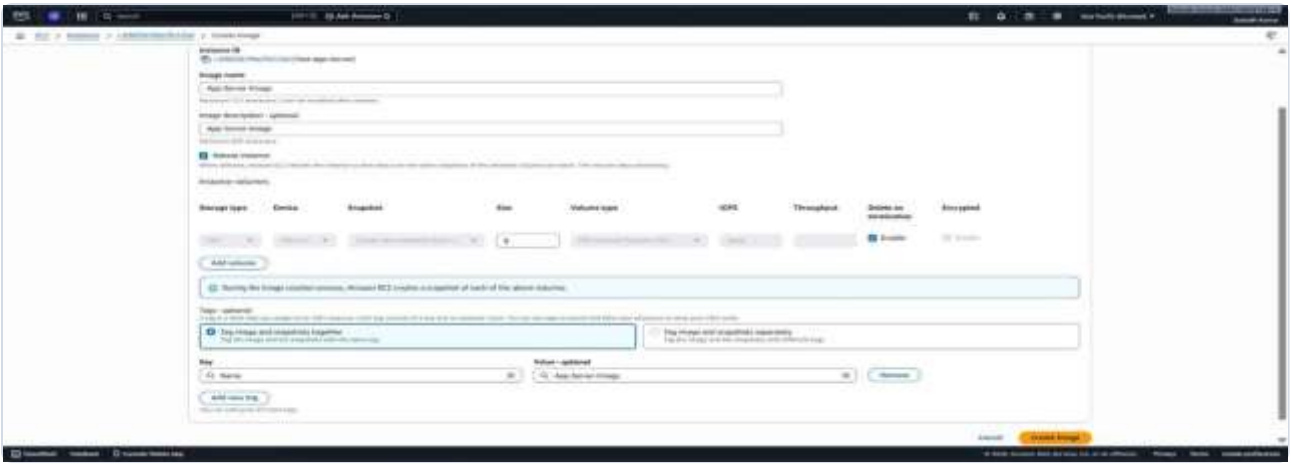
Cool! You successfully set up the app server.

Step 8 Create AMI of the App Server

Next Let’s create an image from this Test-App-Server instance.

Go to instances choose **Test-App-Server** go to **Actions > Image and templates > Create image.**





If you go to **AMIs section**, you can see that App-Server AMI is created

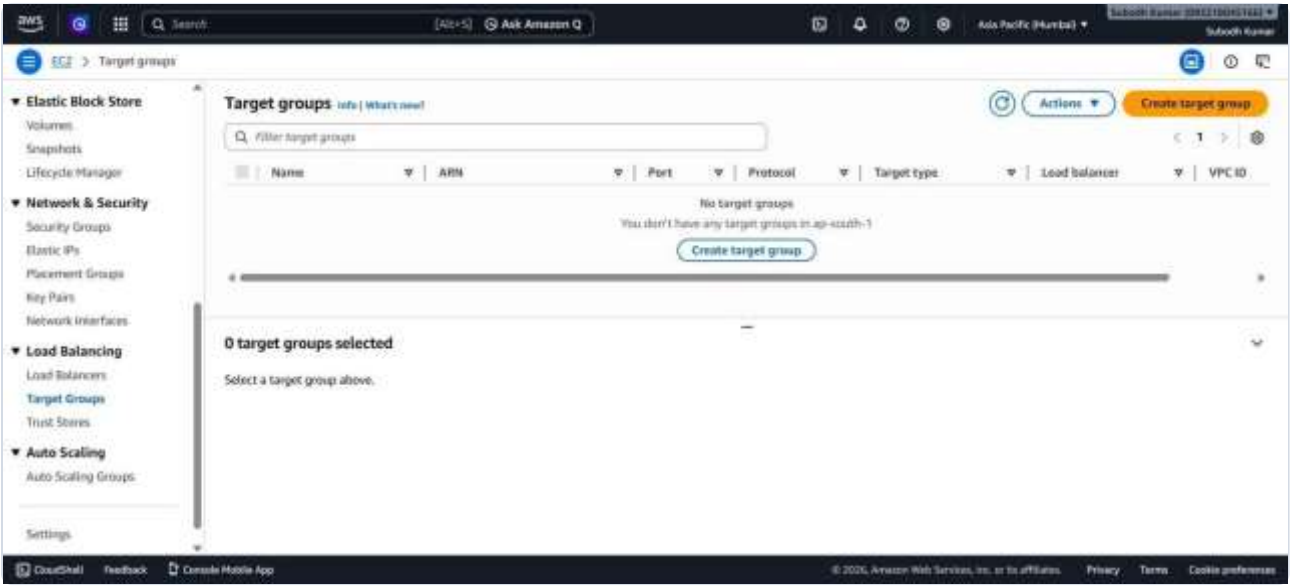


Step 9 Create Target Group, Launch Template and Internal Load Balancer for App Tier

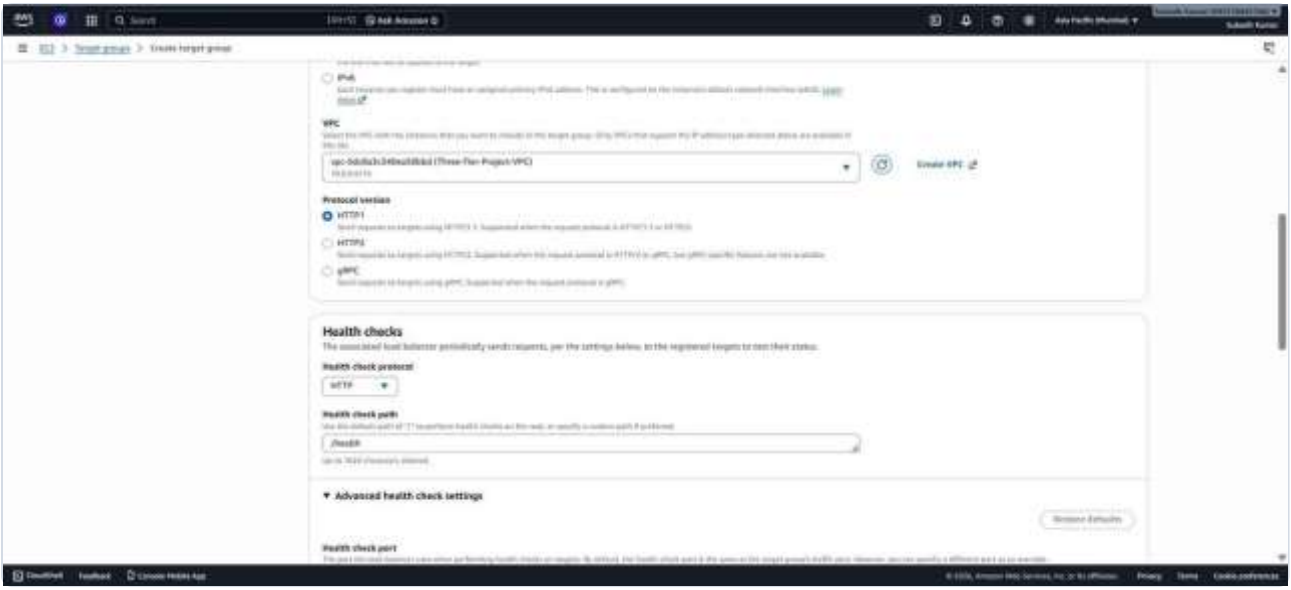
Create Target Group

Go to EC2 > Target Groups > Create Target Group change only following settings. Click Next.

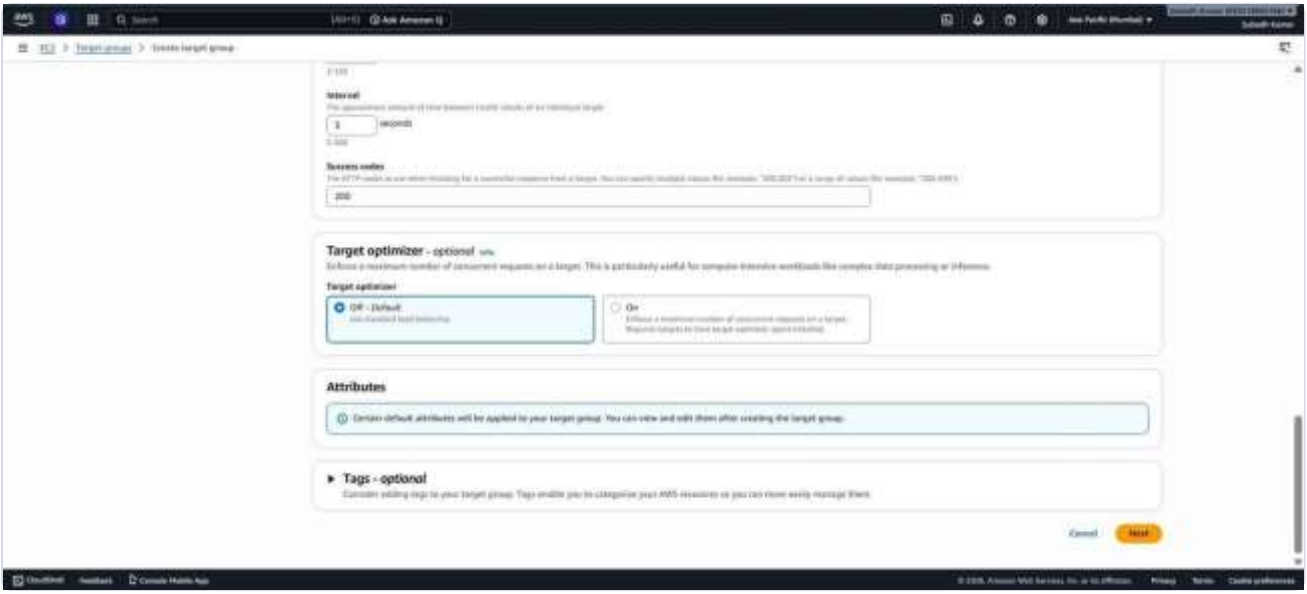
Click Create target group.

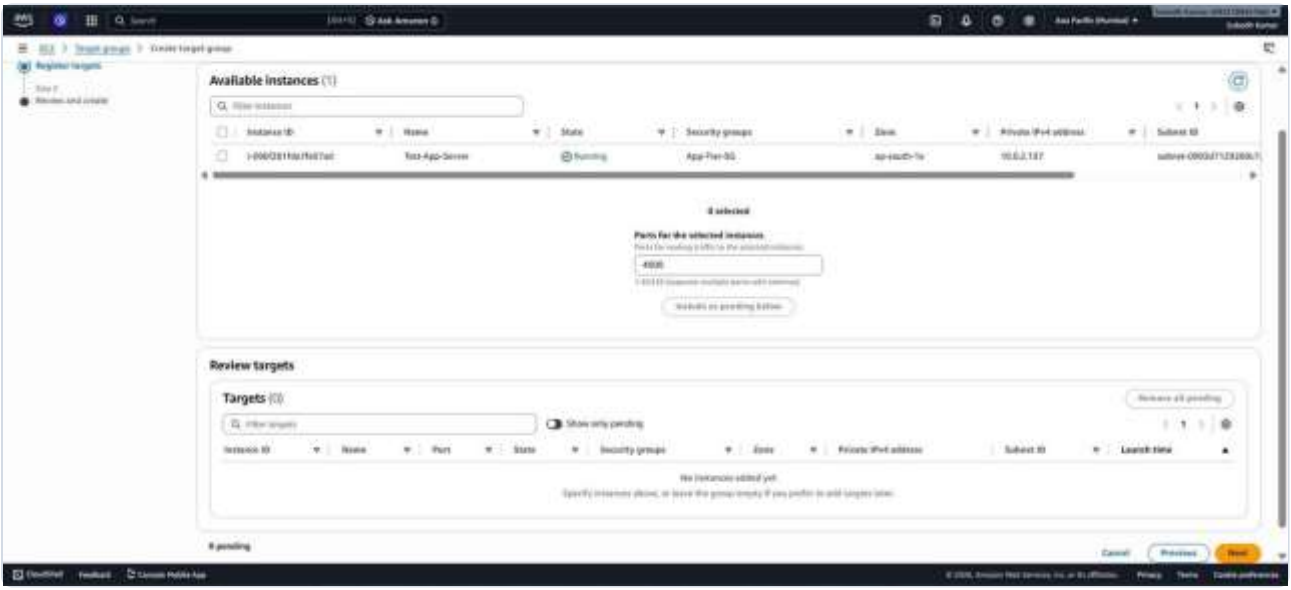


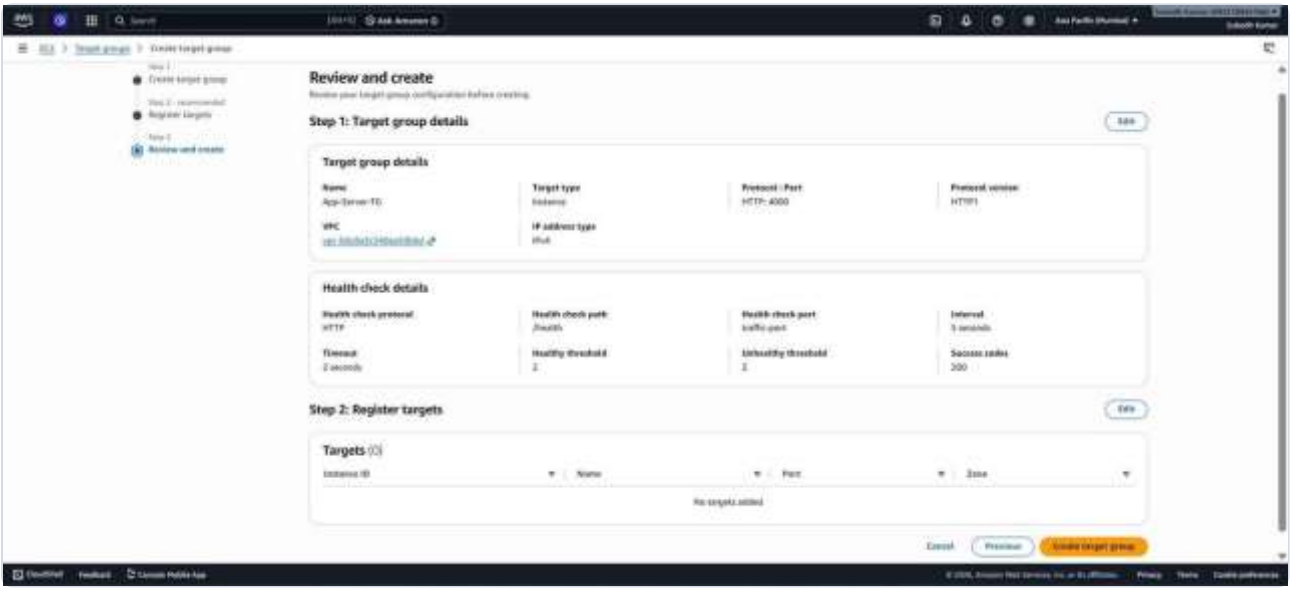



Health checks
Health check protocol
Health check path
Advanced health check settings


Advanced health check settings
Restore defaults
Health check port
Traffic port
Override
Healthy threshold
Unhealthy threshold
Timeout
Interval
Success codes
200
Target optimizer - optional

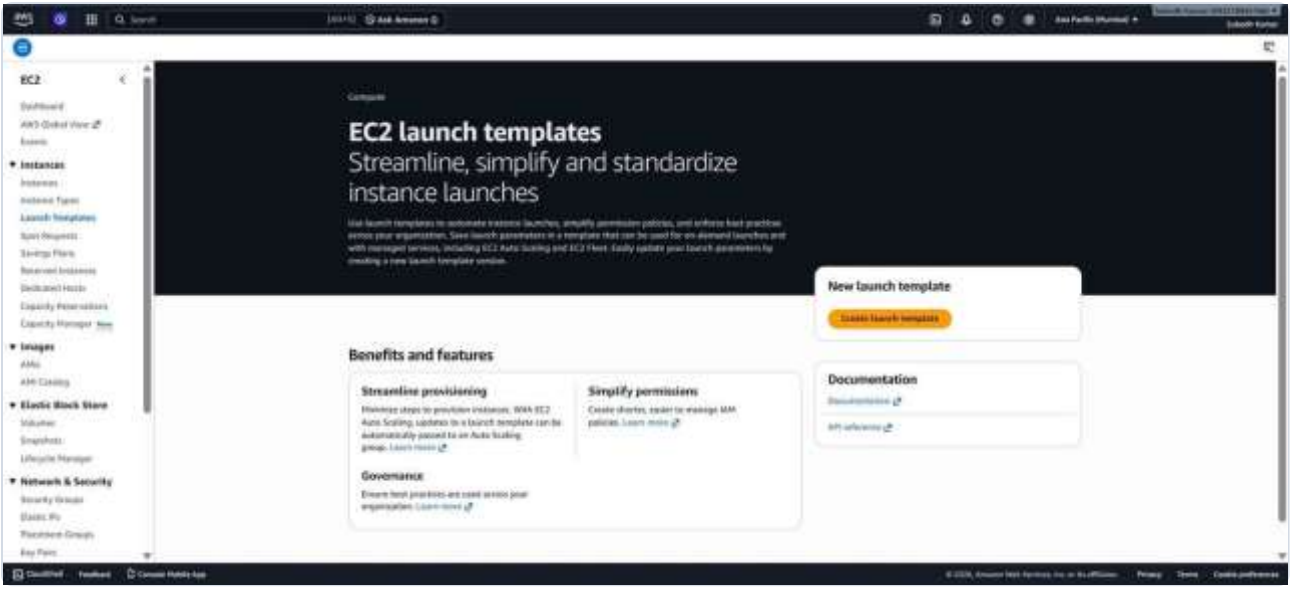

Target optimizer - optional
Target optimizer
Off - Default
On
Attributes
Tags - optional
Cancel
Next

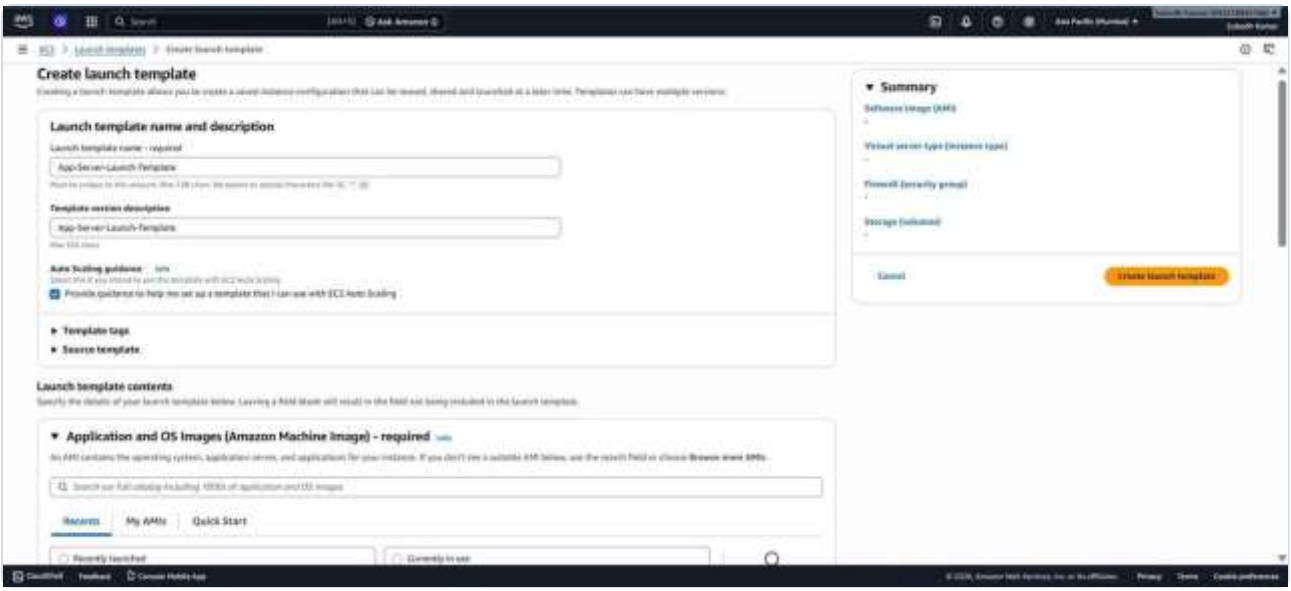

Available instances (1)
Review targets
Targets (0)

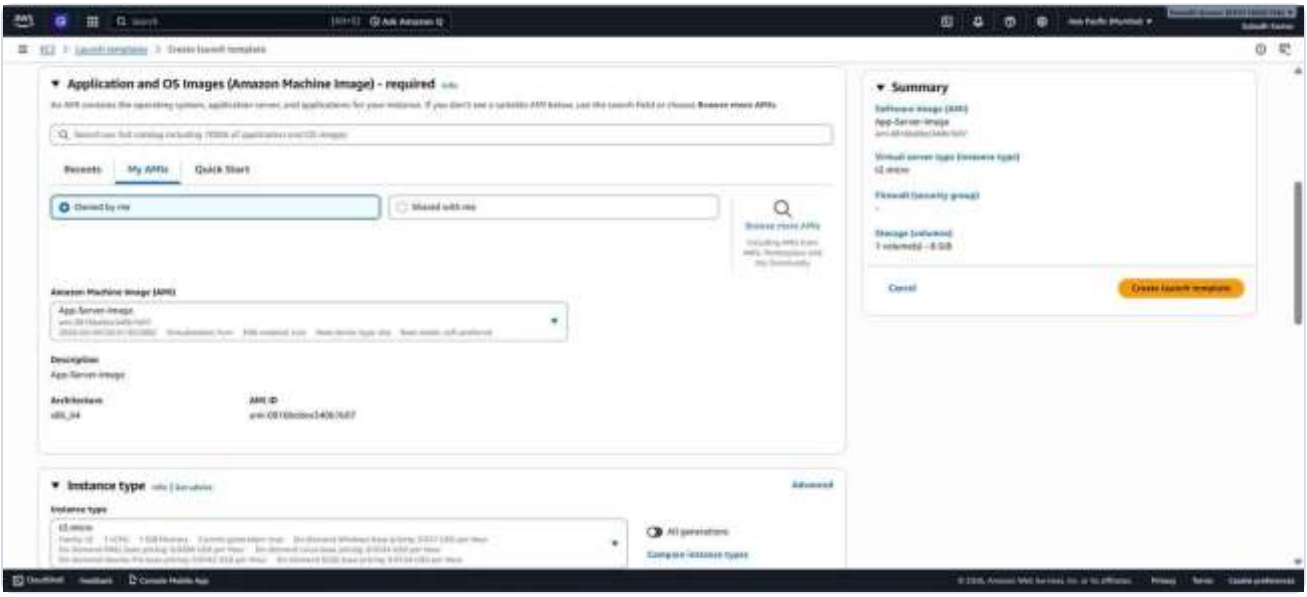

Review and create
Step 1: Target group details
Target group details
Health check details
Step 2: Register targets
Targets (0)

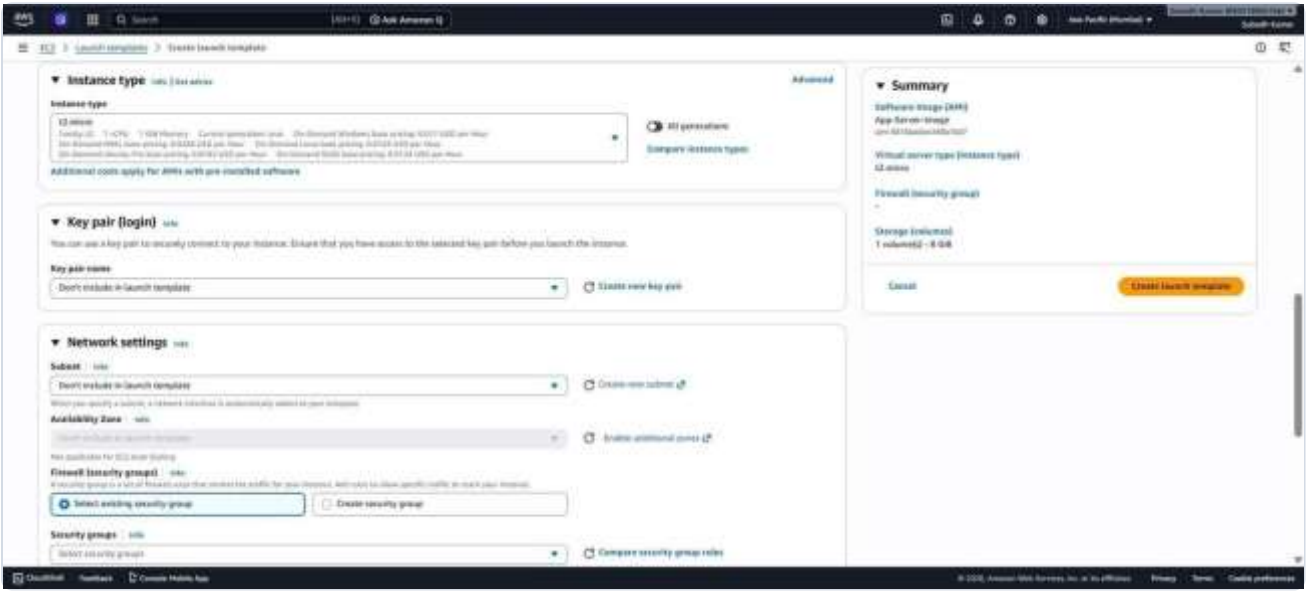
Create Launch Template

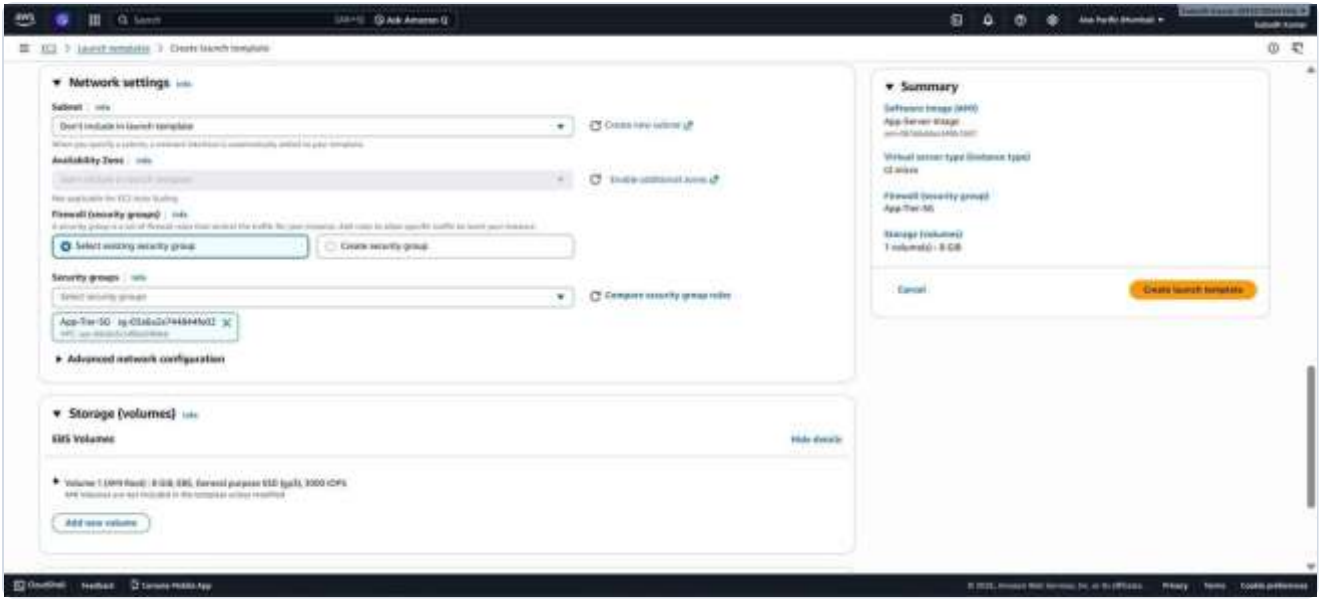
Go to EC2 > Launch Templates > Create launch template.

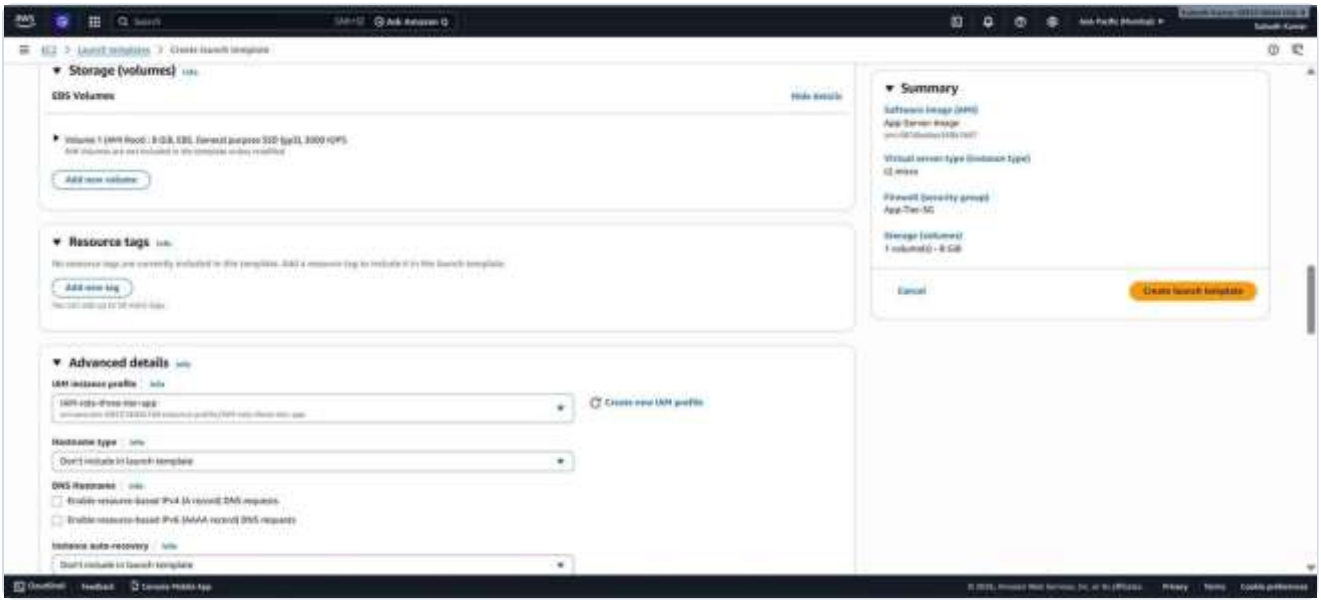





Application and OS Images (Amazon Machine Image) - required
Summary
Instance type
Cancel

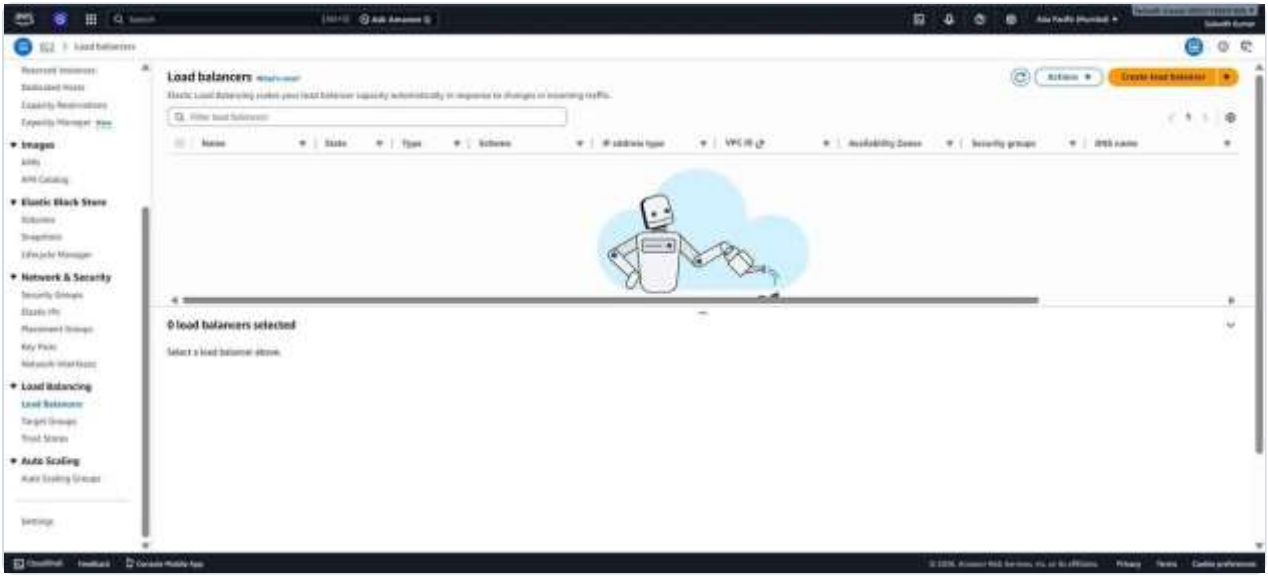

Instance type
Summary
Key pair (login)
Network settings
Cancel


Network settings
Summary
Storage (volumes)
Cancel

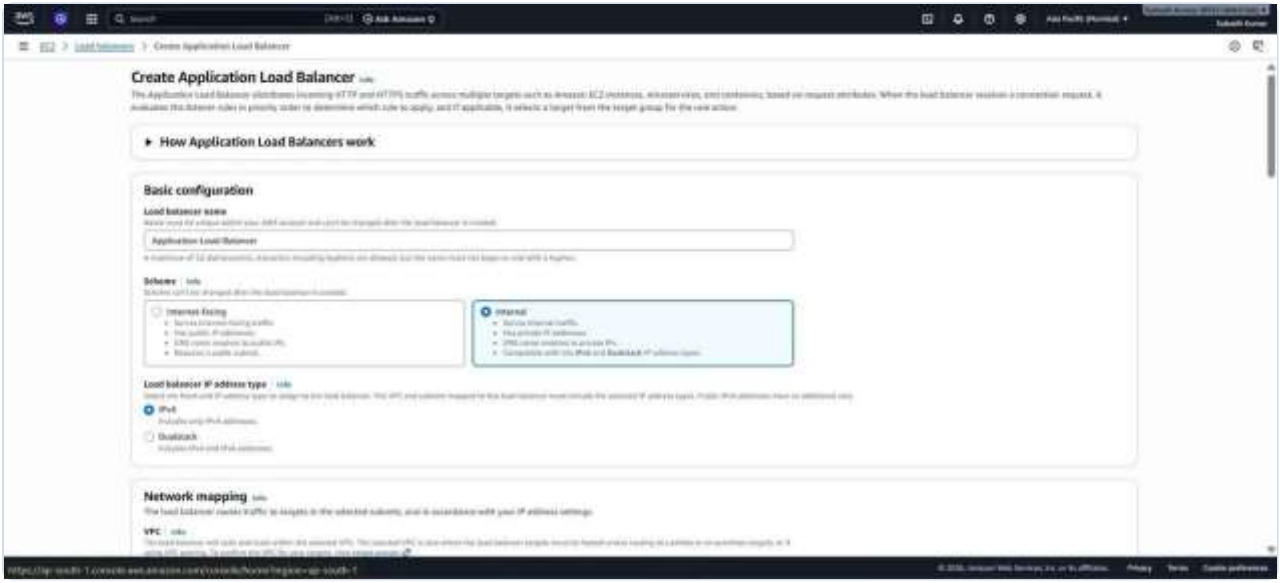


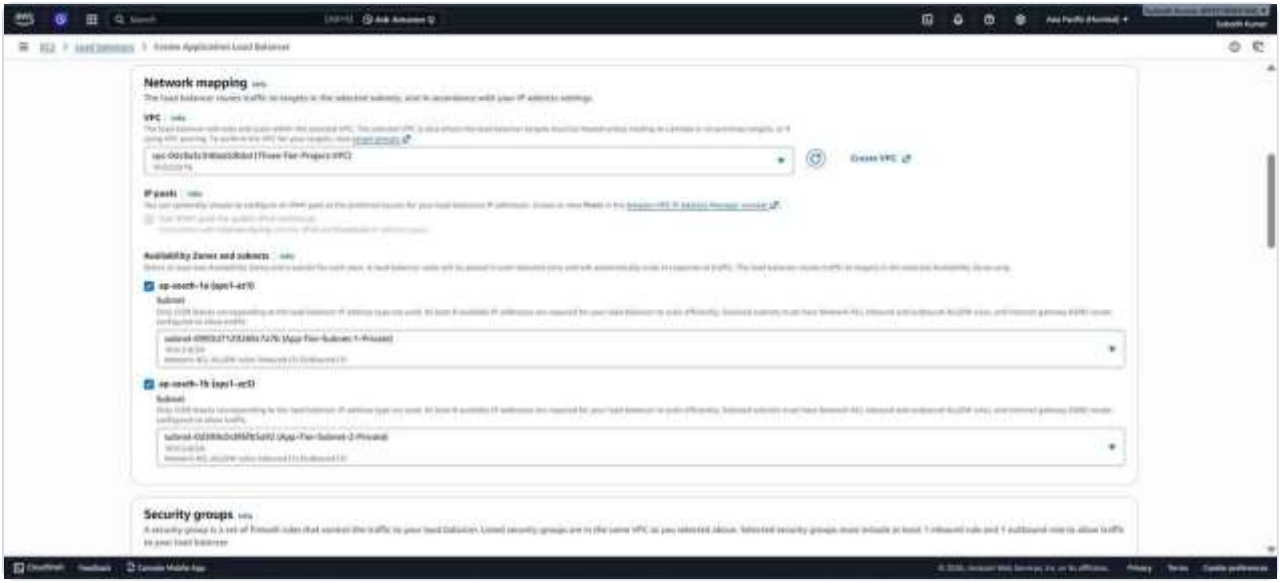
Create Application Load Balancer (Internal)

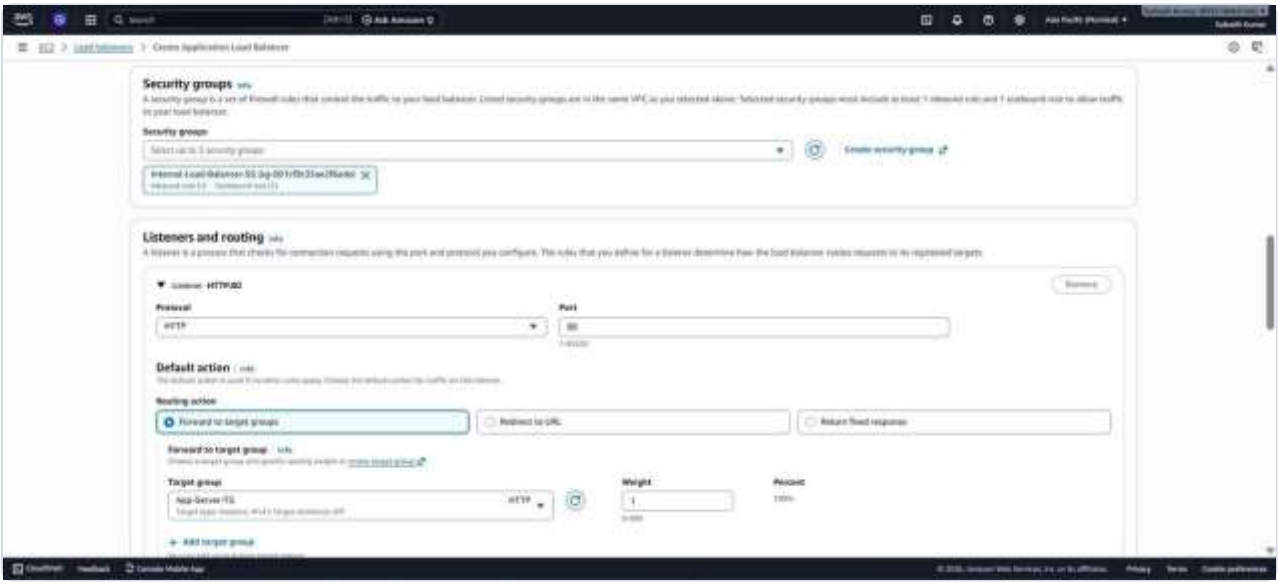
Go to EC2 > Load Balancer > Create Load Balancer create application load balancer.

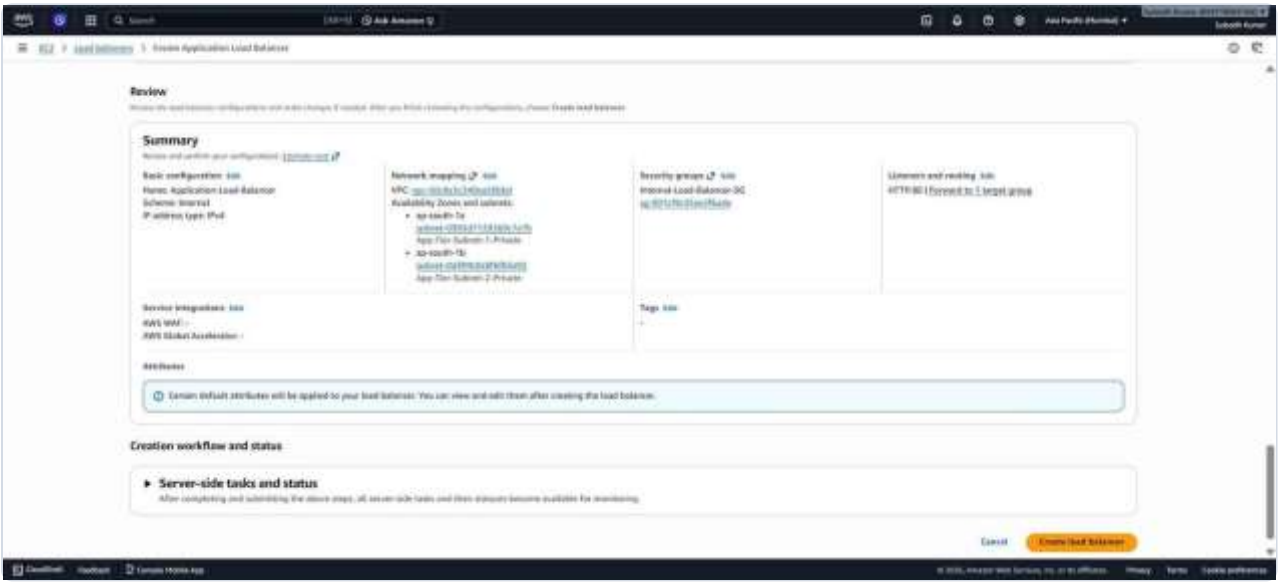



Load balancer types
Application Load Balancer
Network Load Balancer
Gateway Load Balancer
Classic Load Balancer – previous generation


Create Application Load Balancer
How Application Load Balancers work
Basic configuration
Network mapping


Network mapping
Security groups

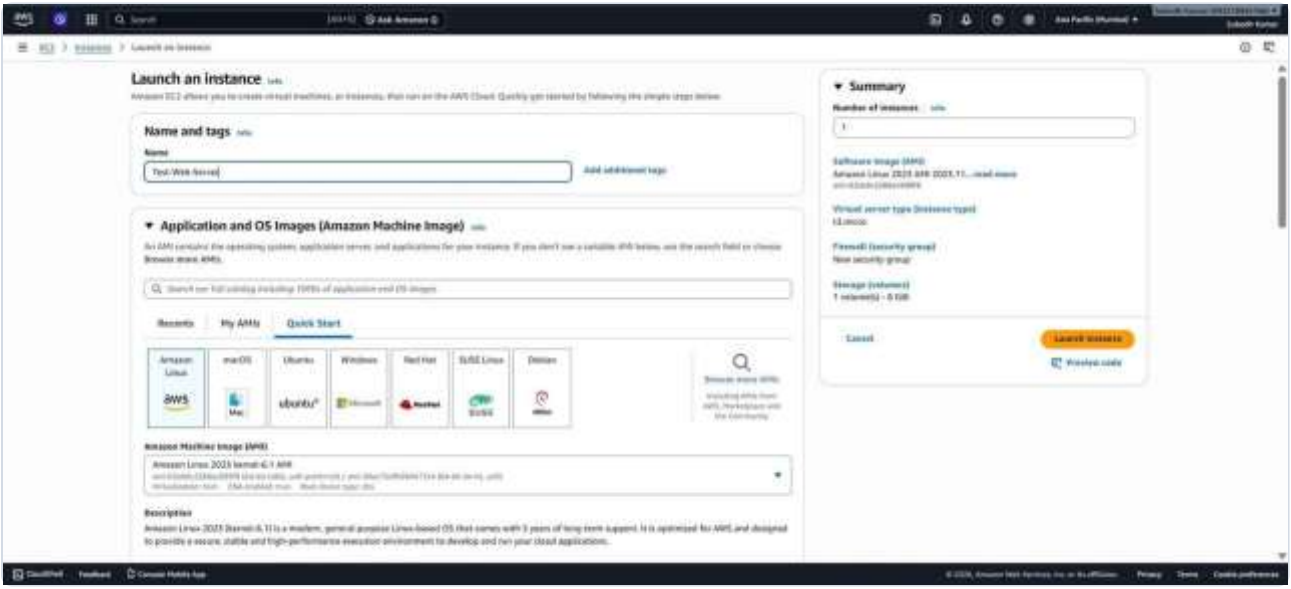


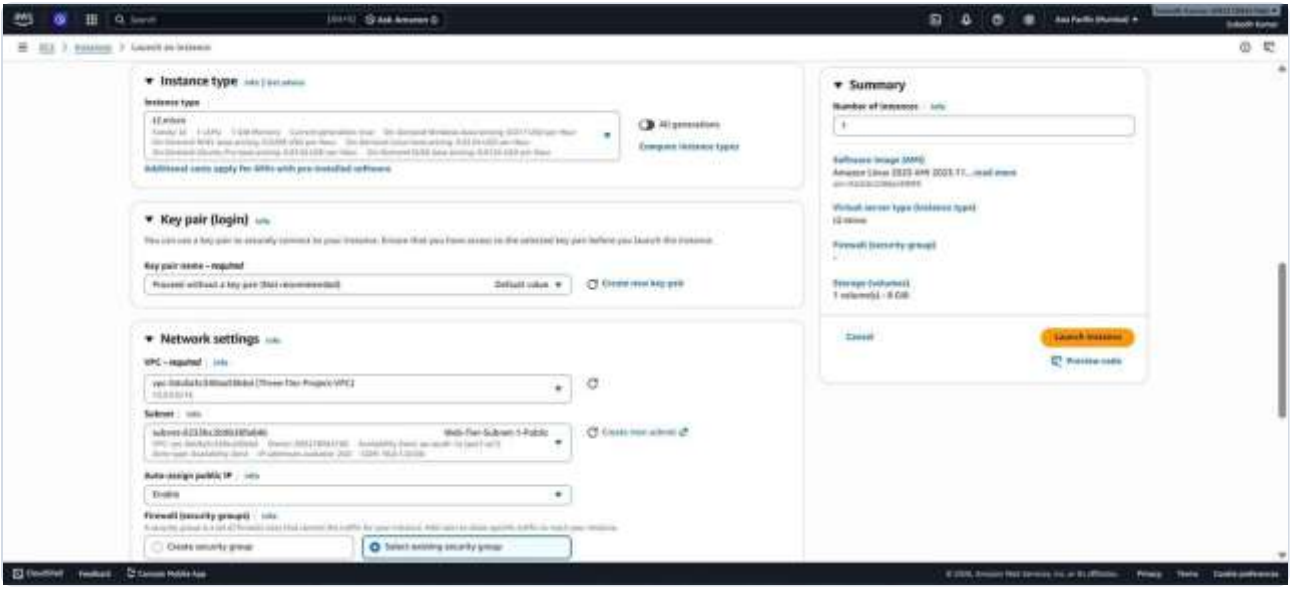


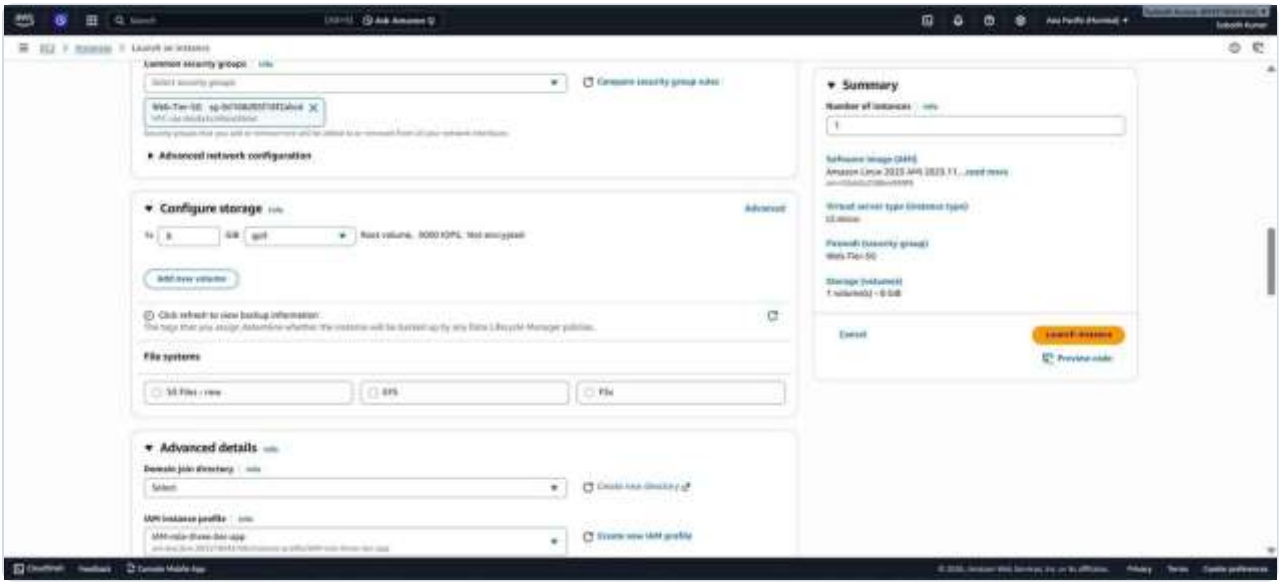
Step 10 Create Test Web Server and Set It Up

We have to follow the same process as followed in App server. Only **EC2 configuration** part **is different**. Configurations will be given below.

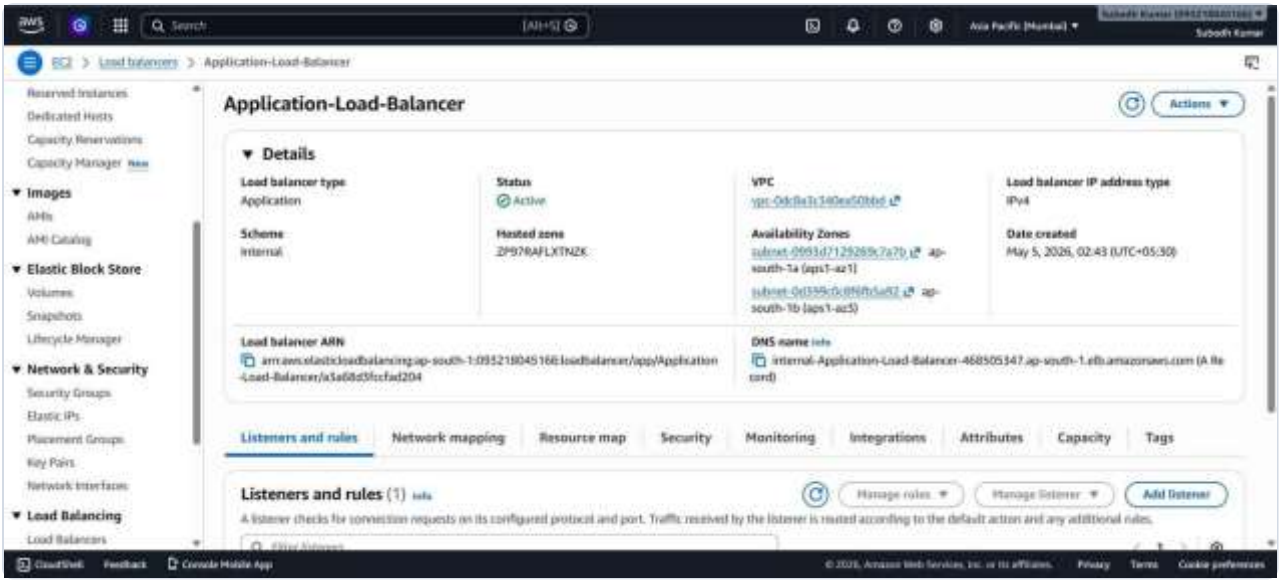
Now got EC2 instance > Launch instance and create a instance for Test Web Server. Change name (Test-WebServer), VPC, Subnet(Web-Tier-Subnet-1-Public), Security group (Web-Tier-SG) and also select IAM role under advanced details. Finally, Launch instance.



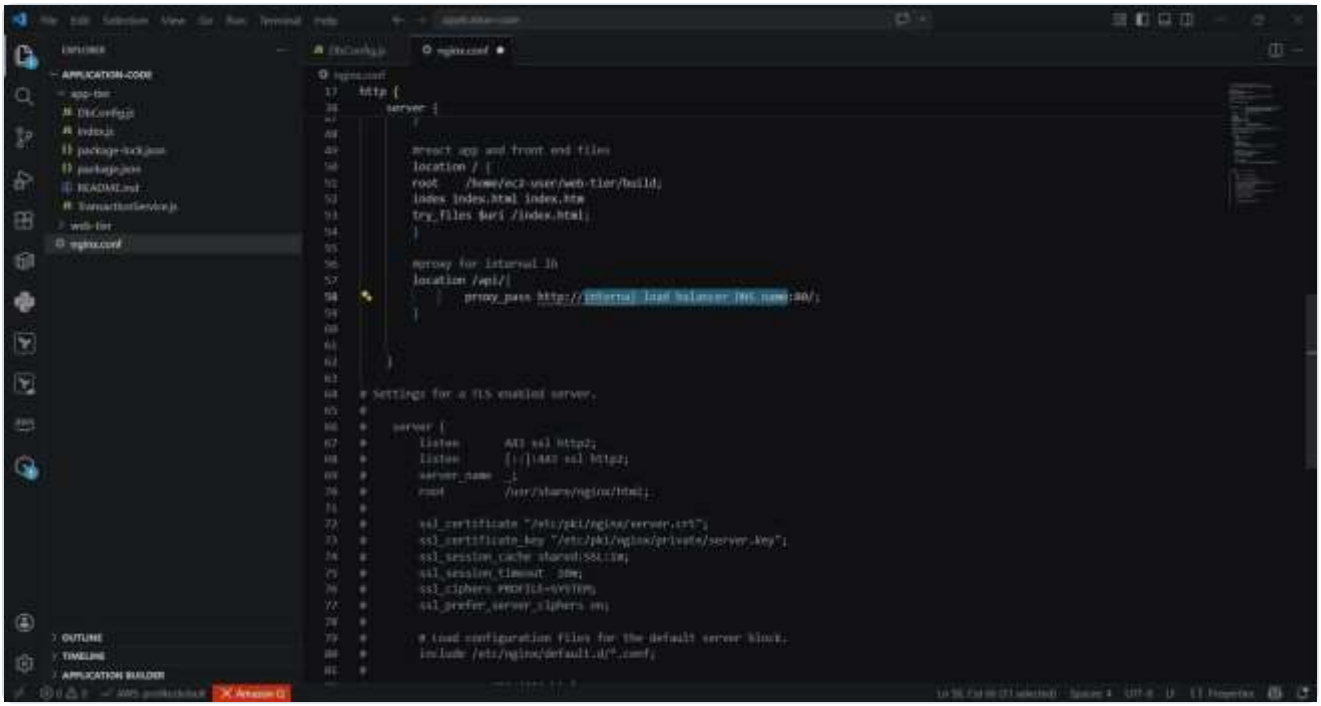


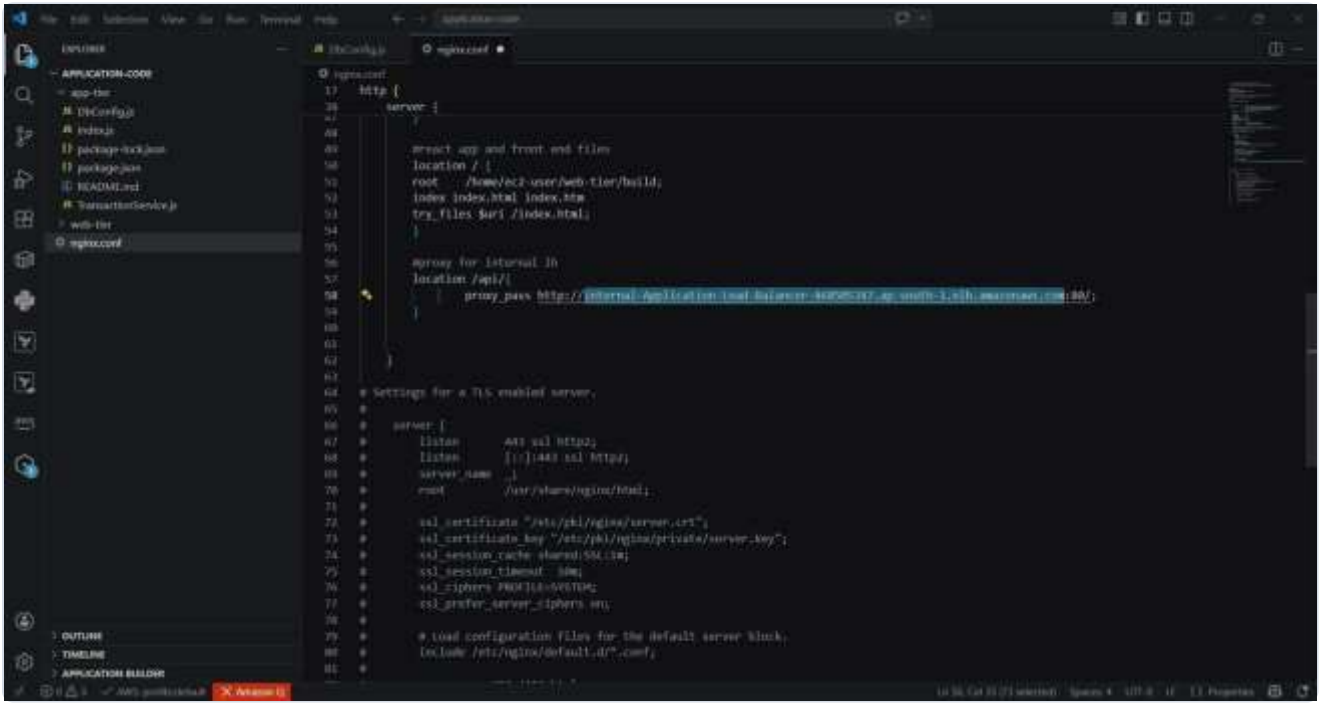


First go to the source code folder in your PC. Open **nginx.conf** file in an editor Now go to the **internal load balancer** we created and copy the **DNS name** of it.



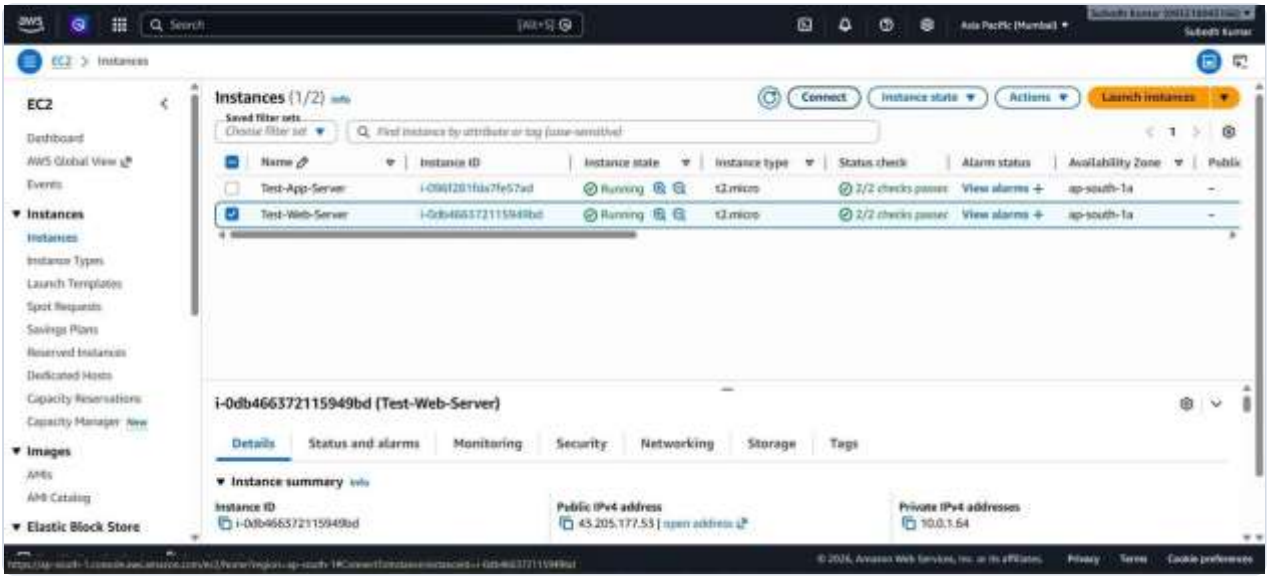
Go to the **nginx.conf** file and paste it in the following section.

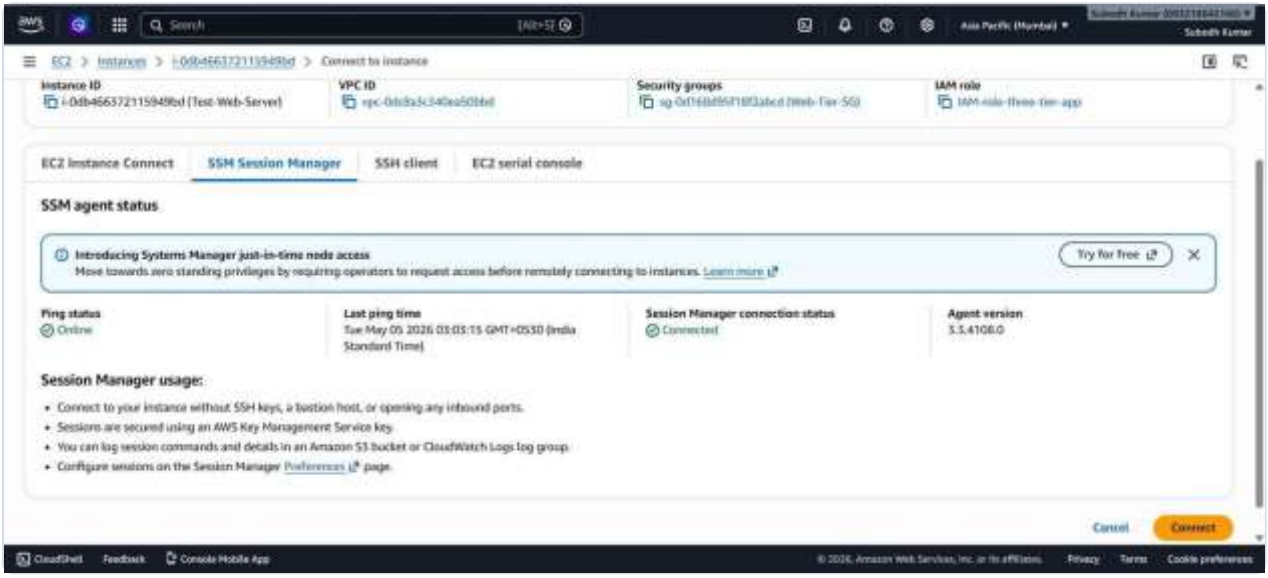




Now go to S3 bucket and delete the **nginx.conf** file and upload the new one.

Now Connect to the instance (**Test-Web-Server**) and do configurations as follows.





Switch users

```
sudo -su ec2-user
cd /home/ec2-user
```

Copy content from the S3 bucket

```
sudo aws s3 cp s3://<S3_BUCKET_NAME>/application-code/web-tier web-tier --recursive
sudo chown -R ec2-user:ec2-user /home/ec2-user
sudo chmod -R 755 /home/ec2-user
```

Install Node js

```
curl -o- https://raw.githubusercontent.com/nvm-sh/nvm/v0.39.7/install.sh | bash
source ~/.bashrc
nvm install 16
```

```
nvm use 16
```

```
cd /home/ec2-user/web-tier
npm install
```

Build the app

```
cd /home/ec2-user/web-tier
npm run build
```

Install nginx (Webserver)

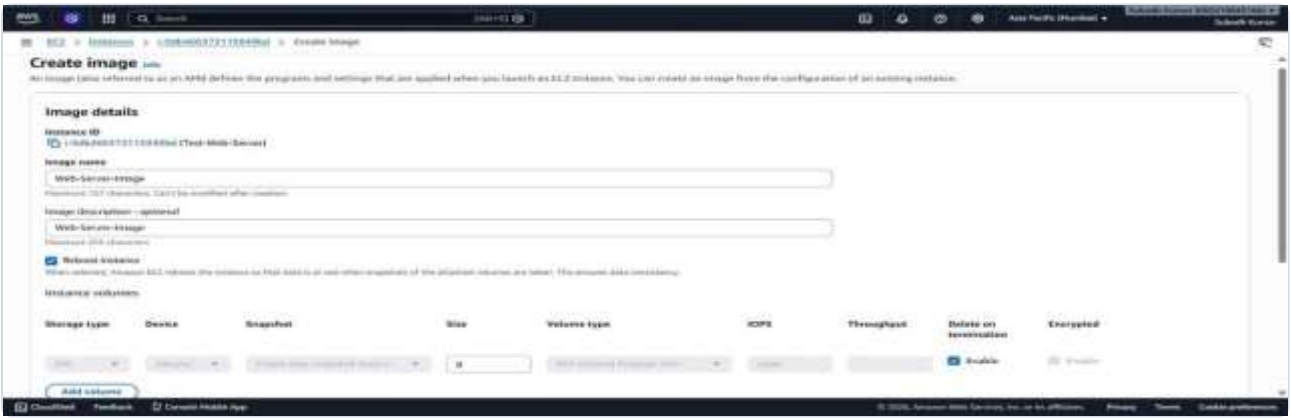
```
sudo yum install nginx -y
cd /etc/nginx
sudo mv nginx.conf nginx-backup.conf
```

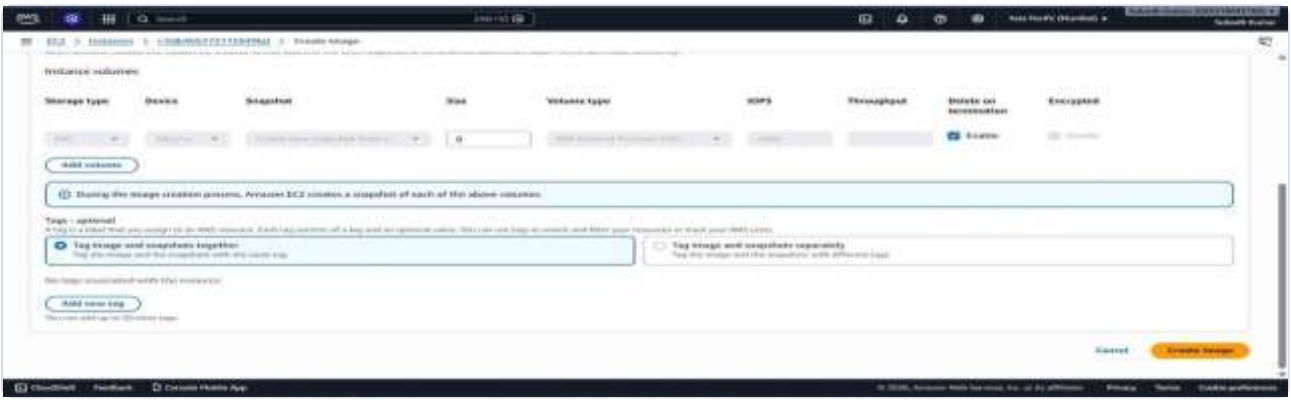
```
sudo aws s3 cp s3://<S3_BUCKET_NAME>/application-code/nginx.conf .
sudo chmod -R 755 /home/ec2-user
sudo service nginx restart
sudo chkconfig nginx on
```

Step 11 Create AMI of the Web Server

Go to instances and select Test-Web-Server > Actions > Image and templates > Create image.







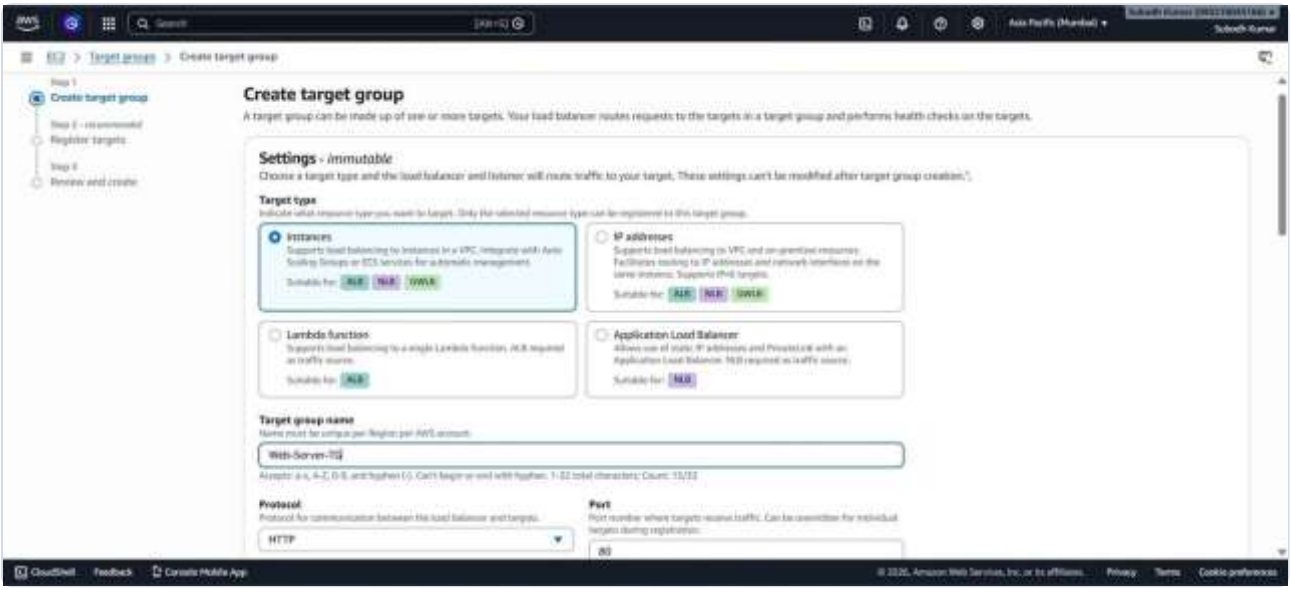
Step 12

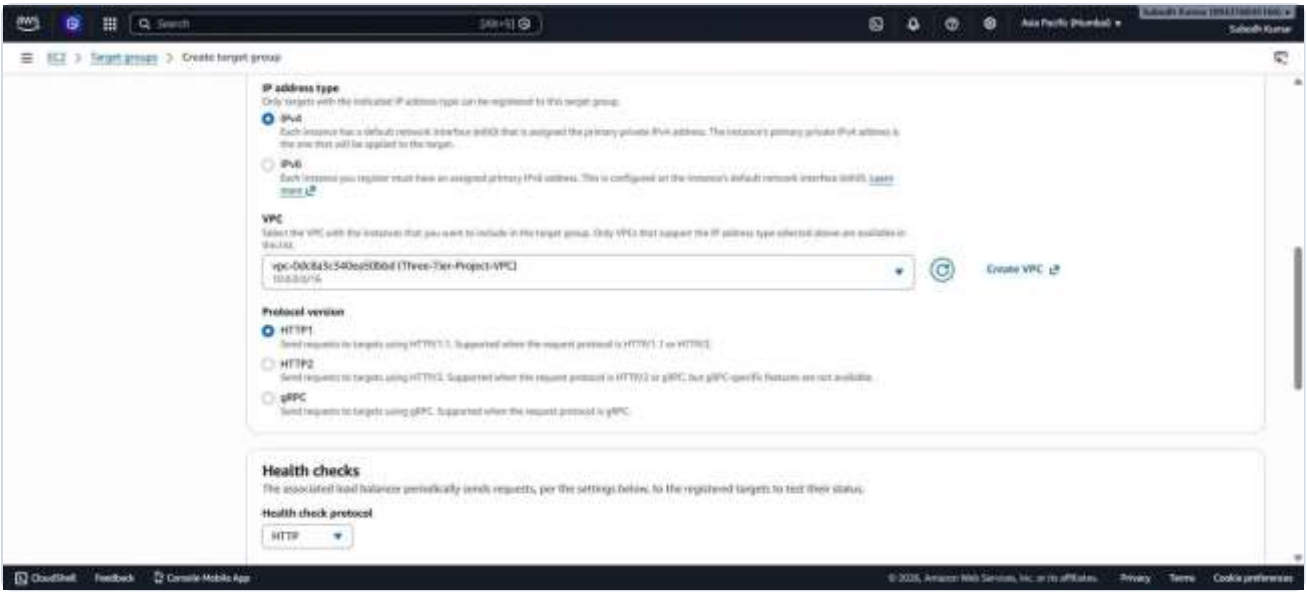
Create Target Group, Launch Template and External Load Balancer for Web Tier

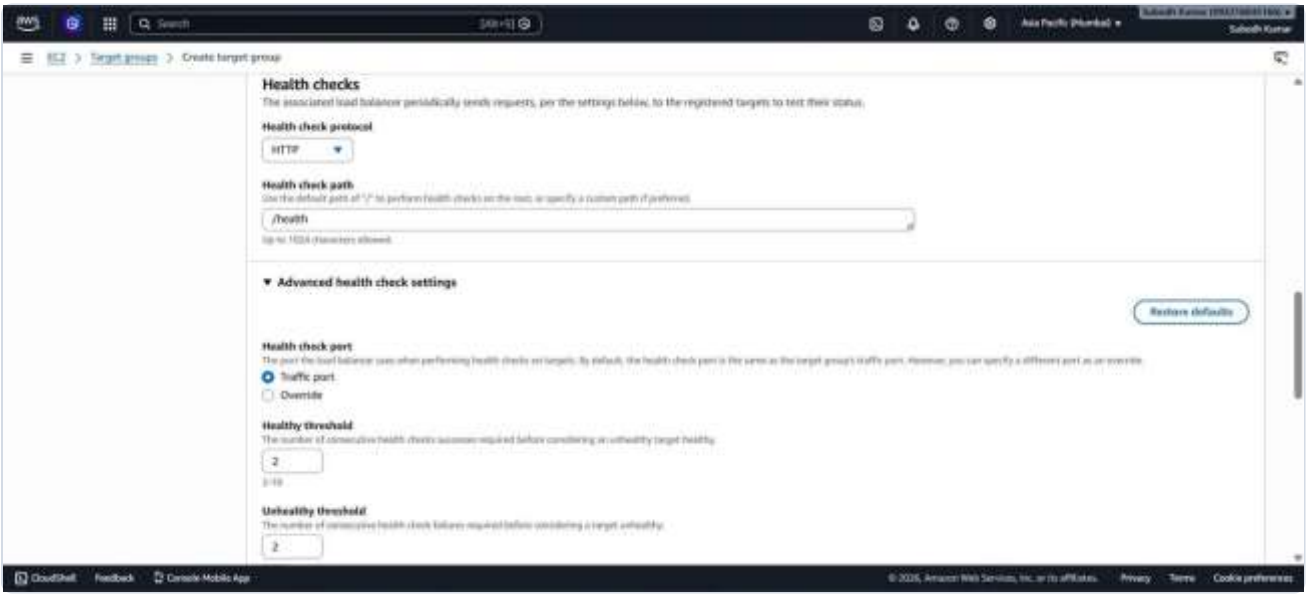
Same process

Create Target Group

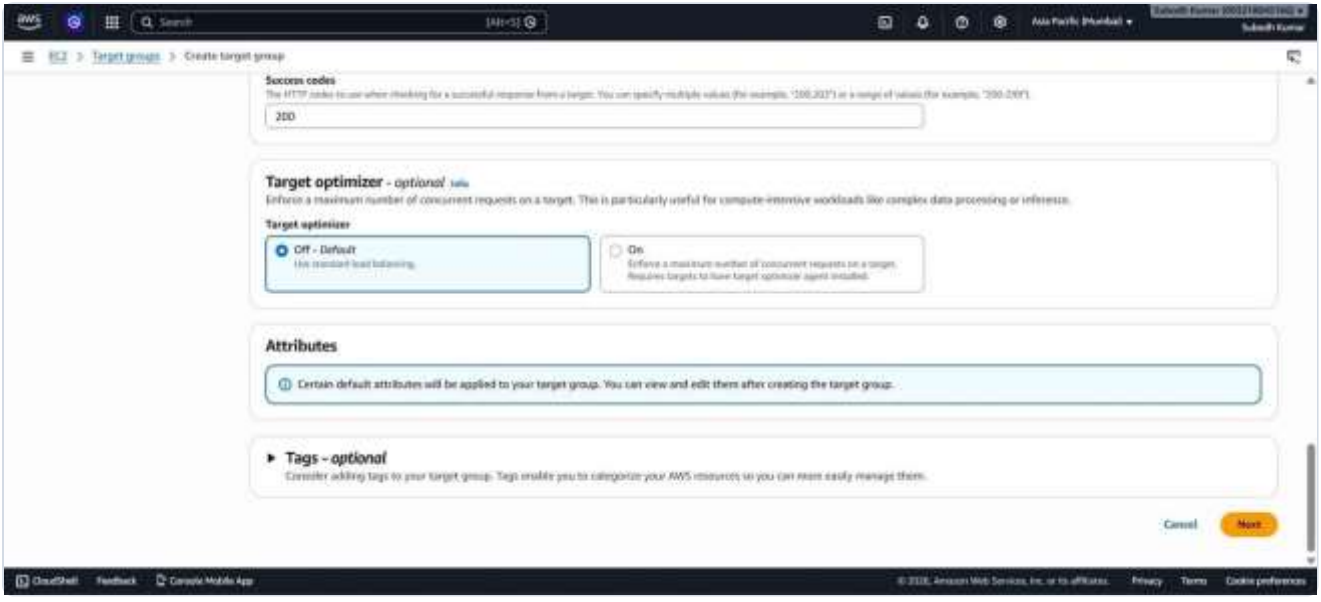
Change Protocol: Port to 80. Select VPC. Give name. Create.

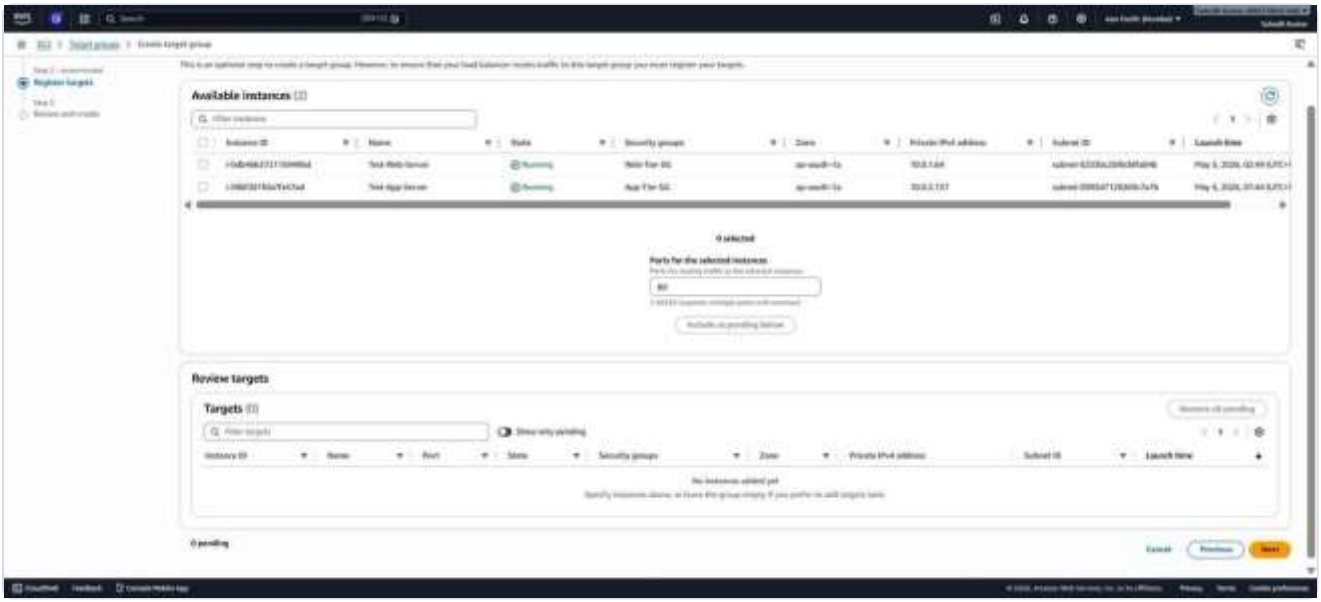


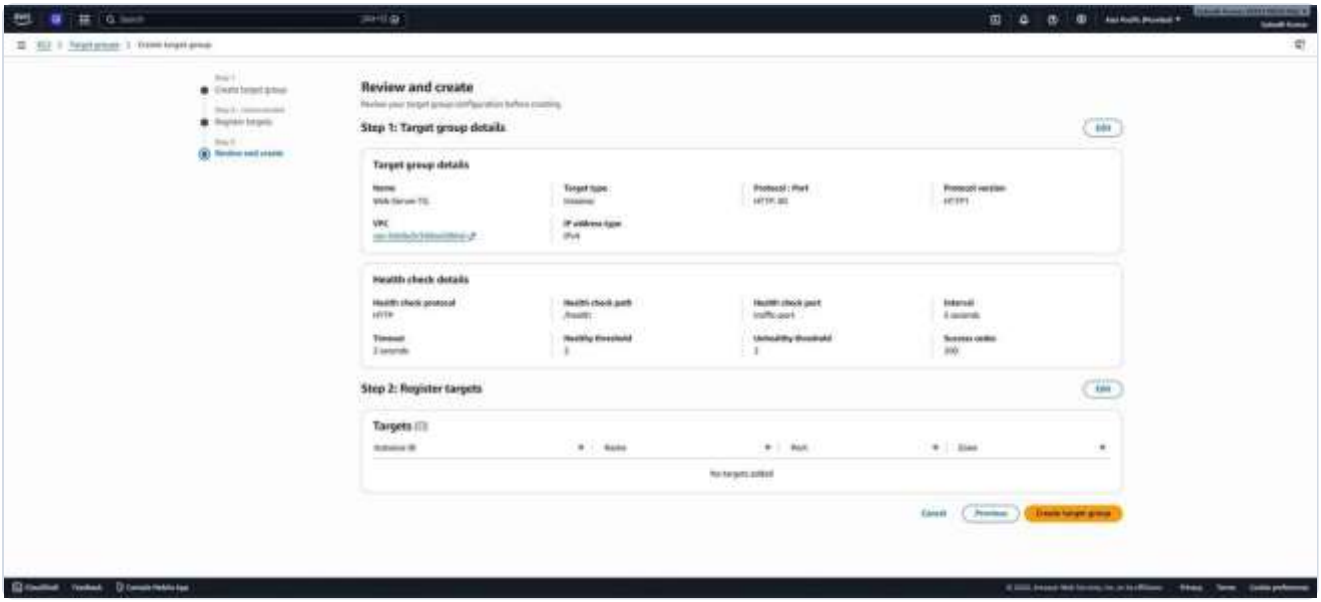



Health checks
Health check protocol
HTTP
Health check path
/health
Advanced health check settings
Restore defaults
Health check port
Traffic port
Override
Healthy threshold
2
Unhealthy threshold
2


Health check port
Traffic port
Override
Healthy threshold
2
Unhealthy threshold
2
Timeout
2
seconds
Interval
5
seconds
Success codes
200

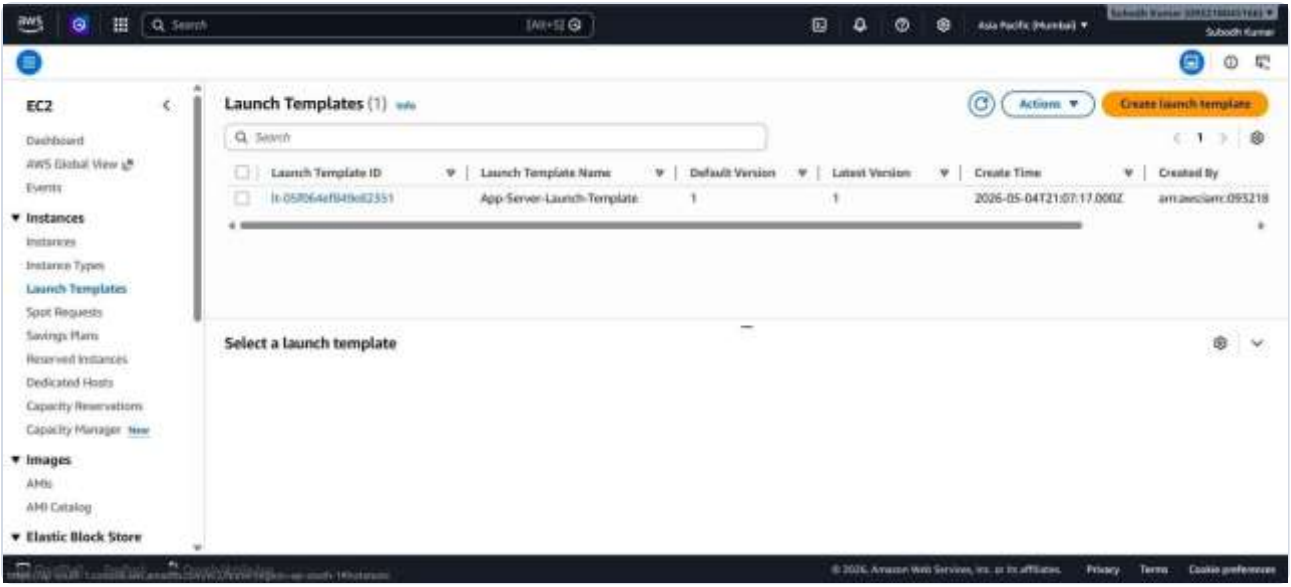

Success codes
200
Target optimizer - optional
Enforce a maximum number of concurrent requests on a target. This is particularly useful for compute-intensive workloads like complex data processing or inference.
Target optimizer
Off - Default
On
Attributes
Certain default attributes will be applied to your target group. You can view and edit them after creating the target group.
Tags - optional
Consider adding tags to your target group. Tags enable you to categorize your AWS resources so you can more easily manage them.
Cancel
Next

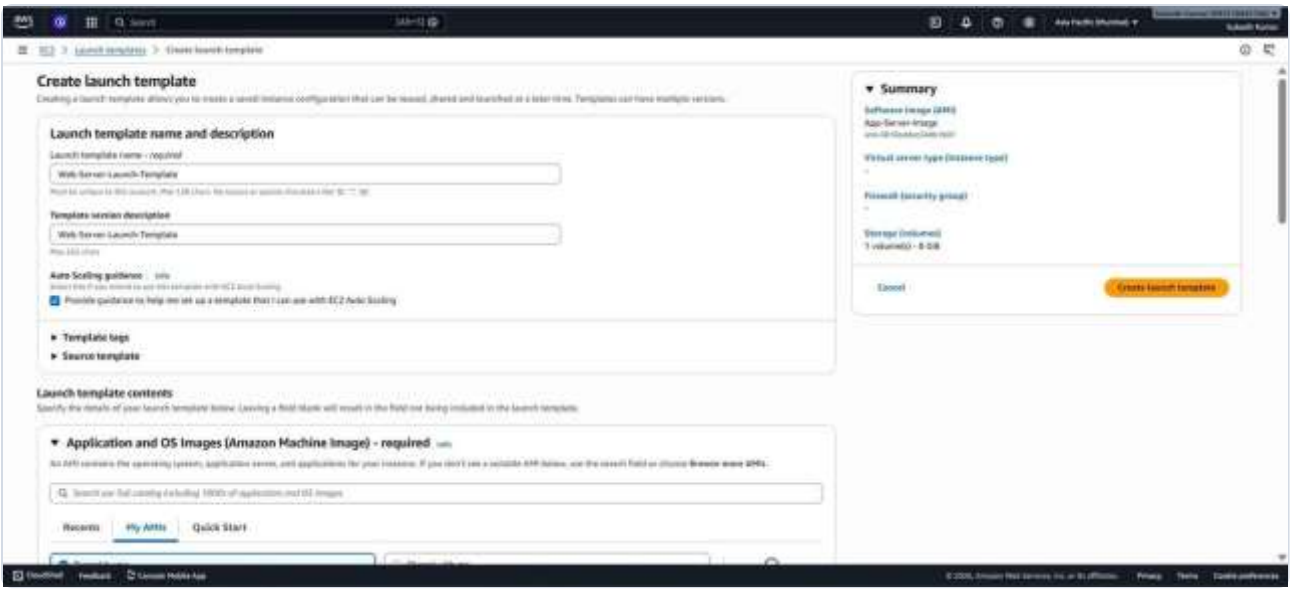

Available instances
Review targets
Targets
0 pending

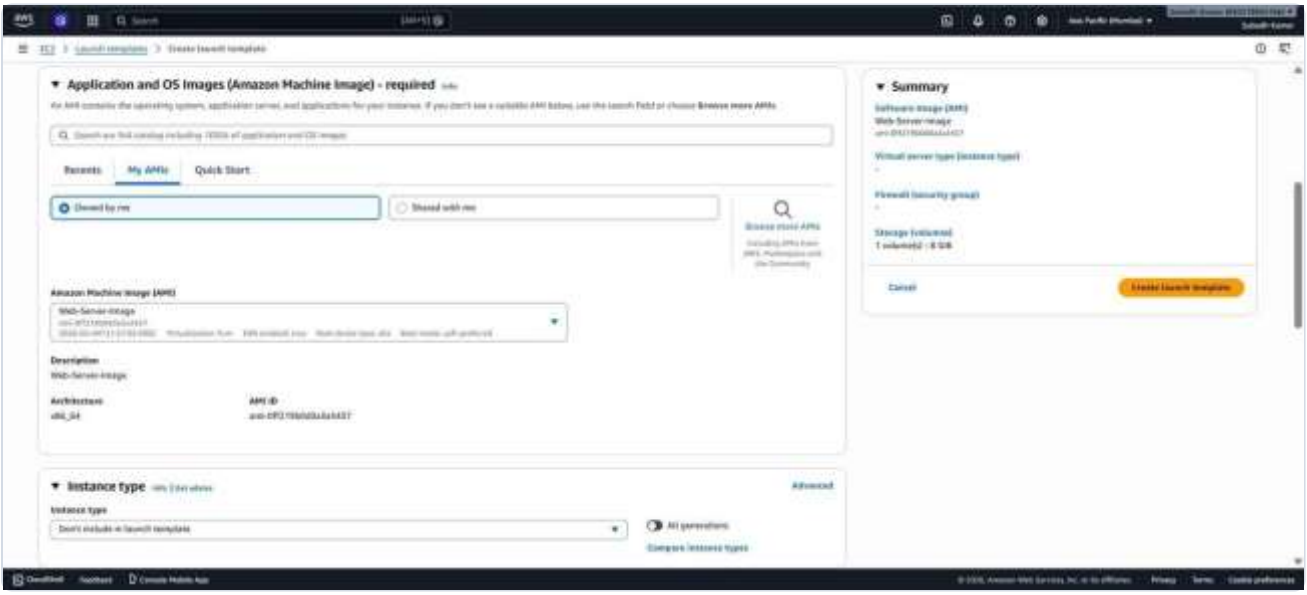

Review and create
Step 1: Target group details
Target group details
Health check details
Step 2: Register targets
Targets

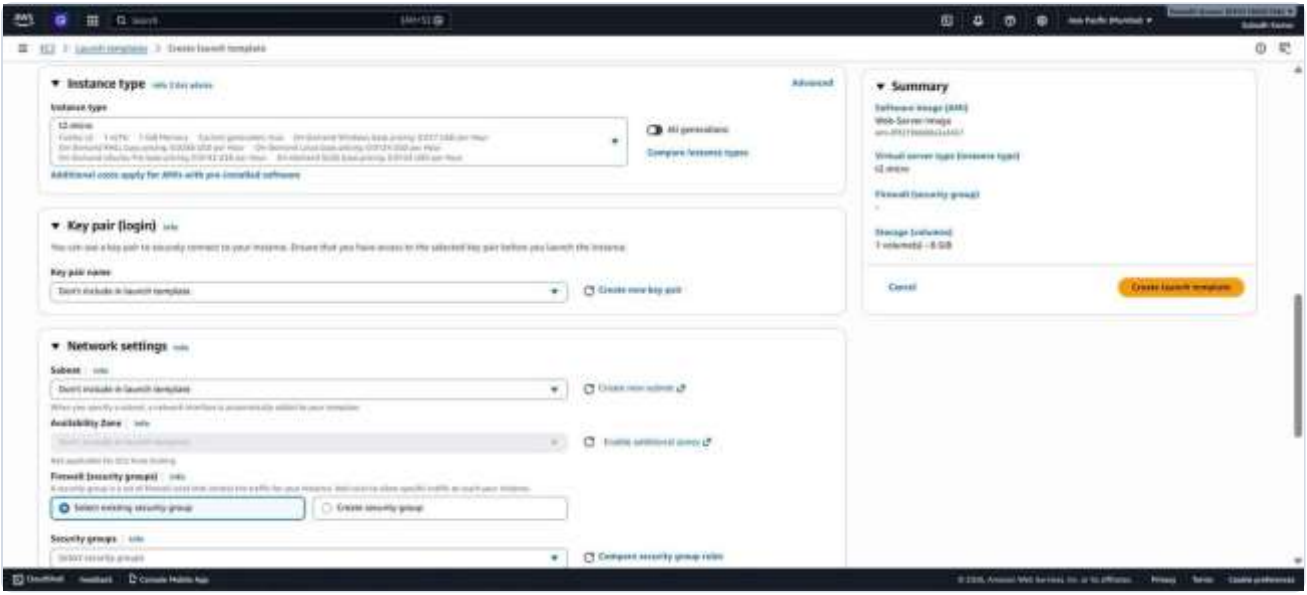
Create Launch Template

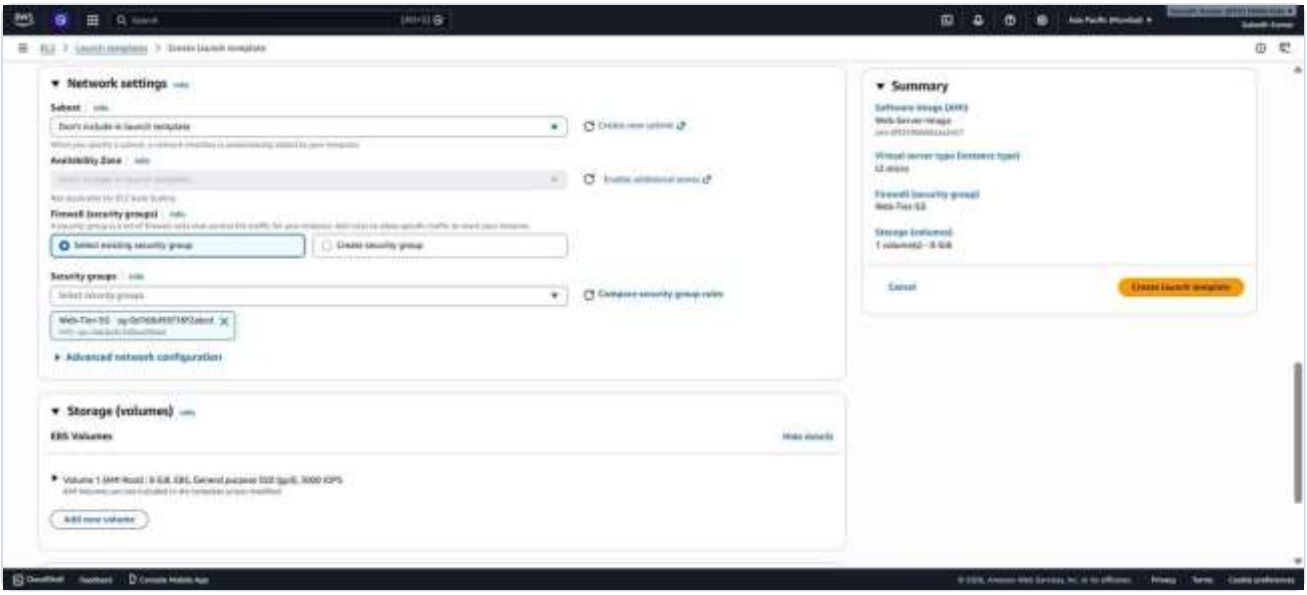
Choose Web-Server-Image for AMI and choose Web-Tier-SG. Other settings are same as above app tier.

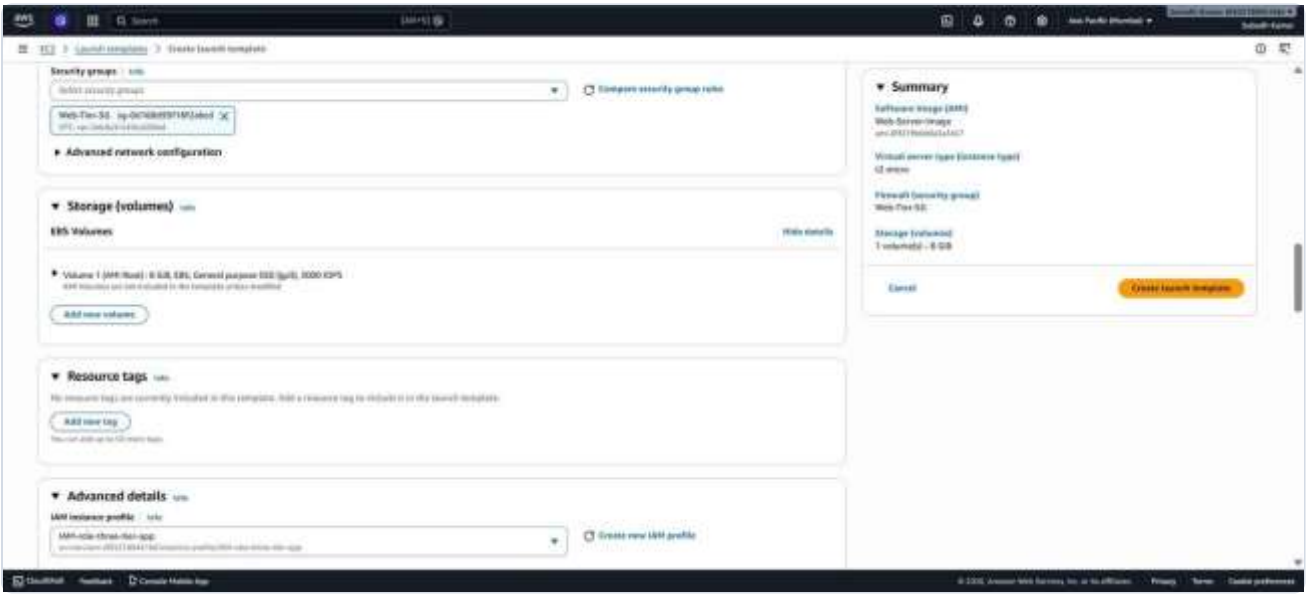





Application and OS Images (Amazon Machine Image) - required
Summary
Instance type

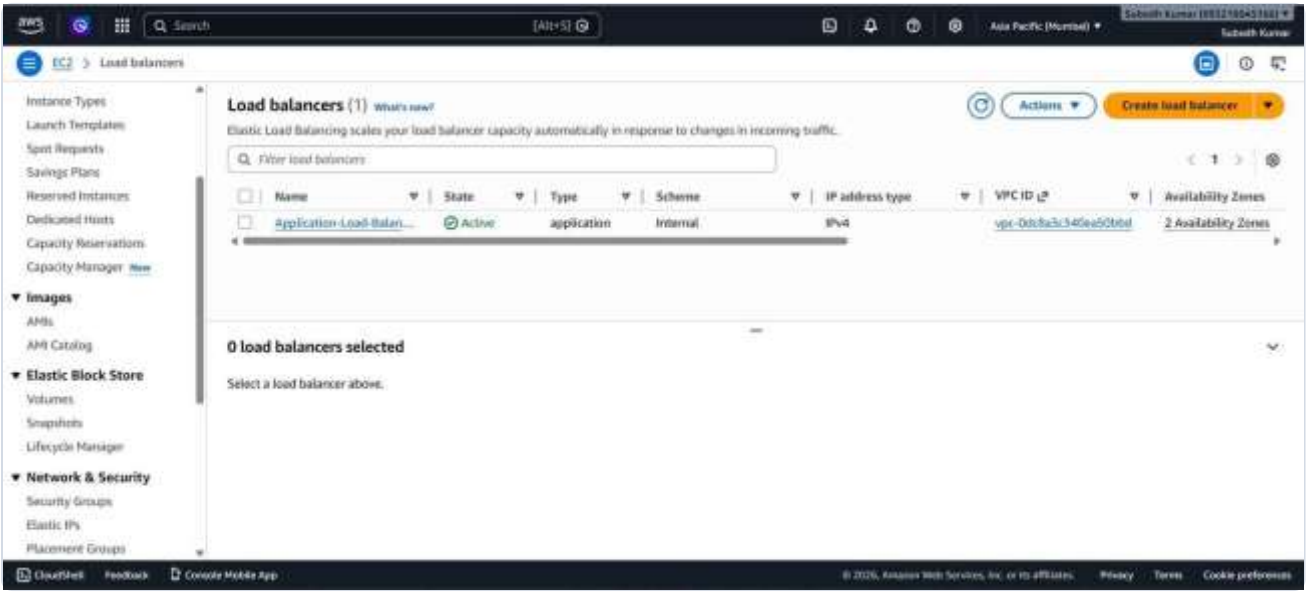

Instance type
Key pair (login)
Network settings
Summary

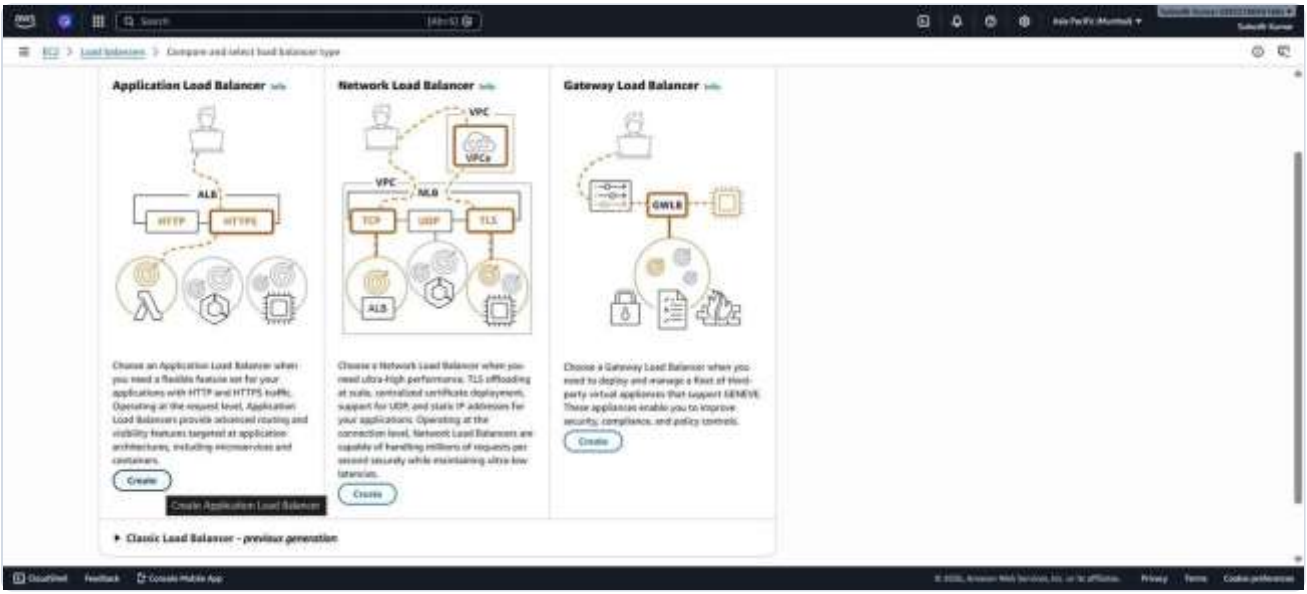

Network settings
Storage (volumes)
Summary

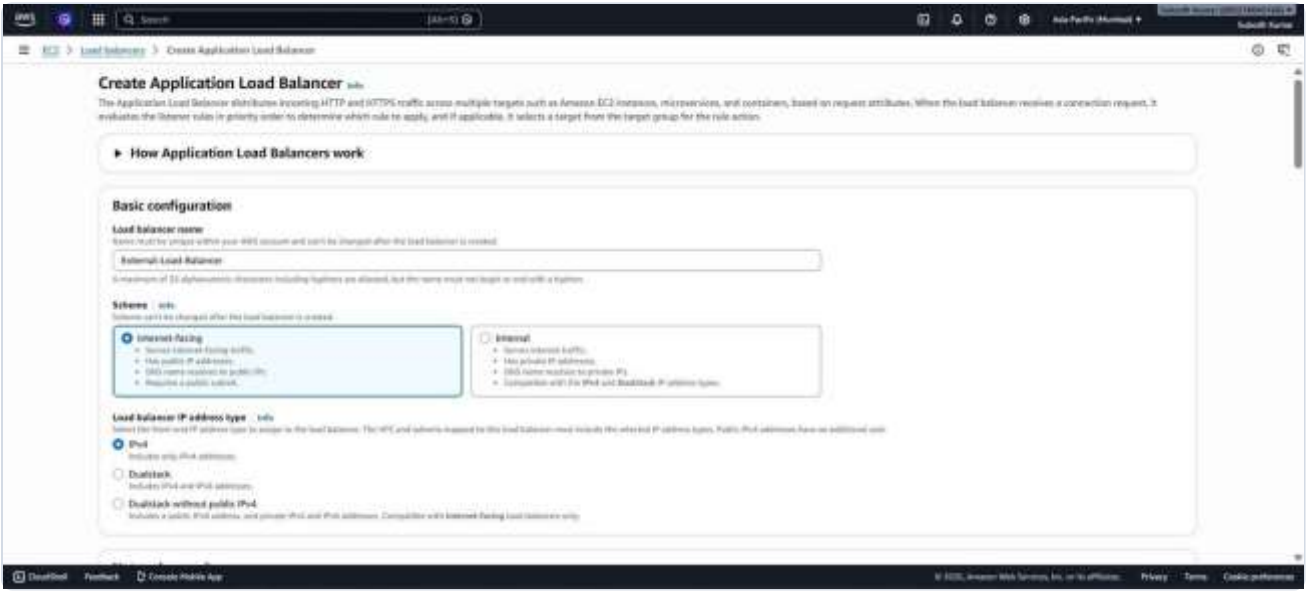


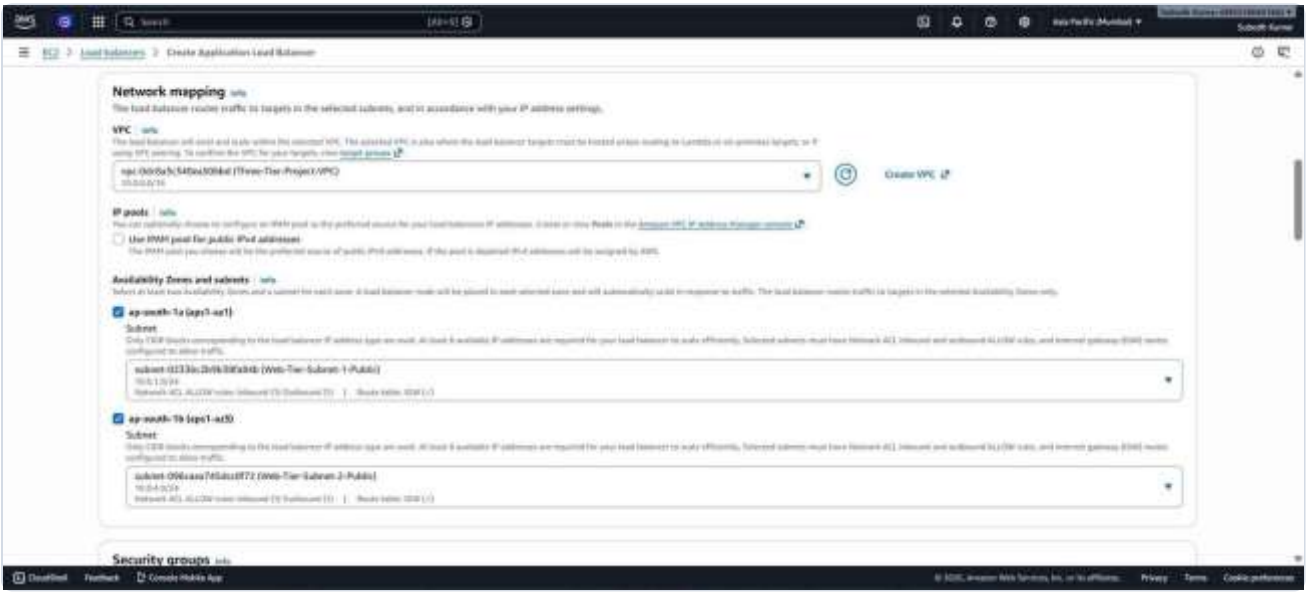
Create External Load Balancer

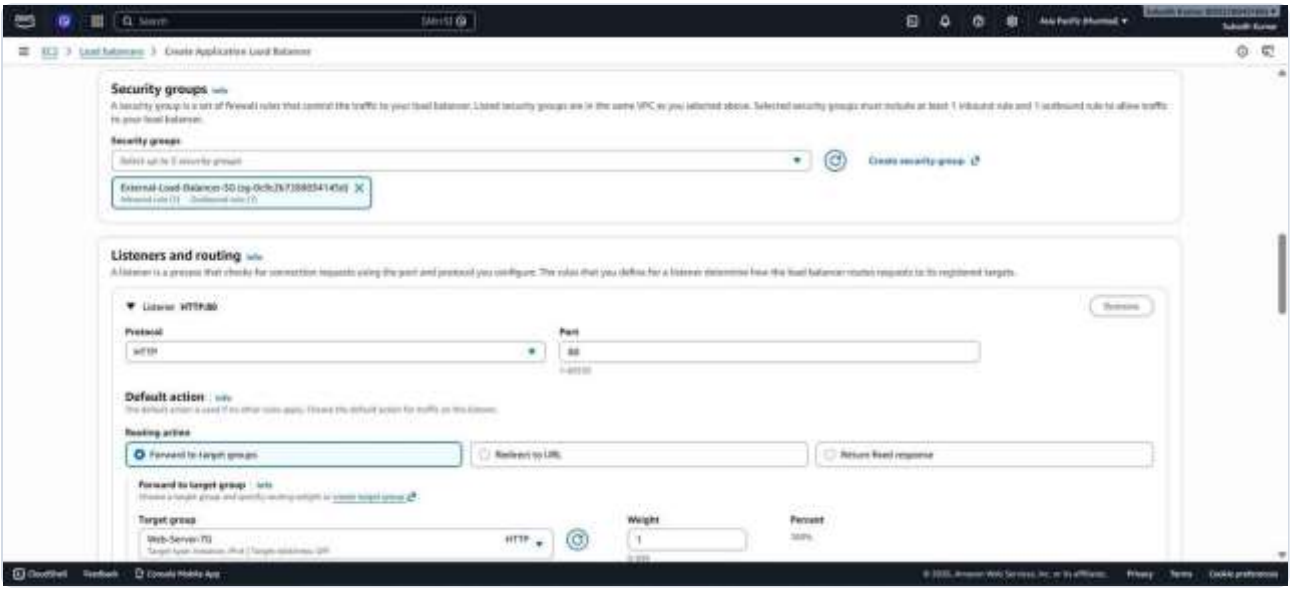
Choose **Scheme** as **Internet-facing**. Choose vpc. Choose **Availability Zones** of **web- tier**. Choose **External load balancer Security Group**. Choose target group in Listeners and routing. Create load balancer.

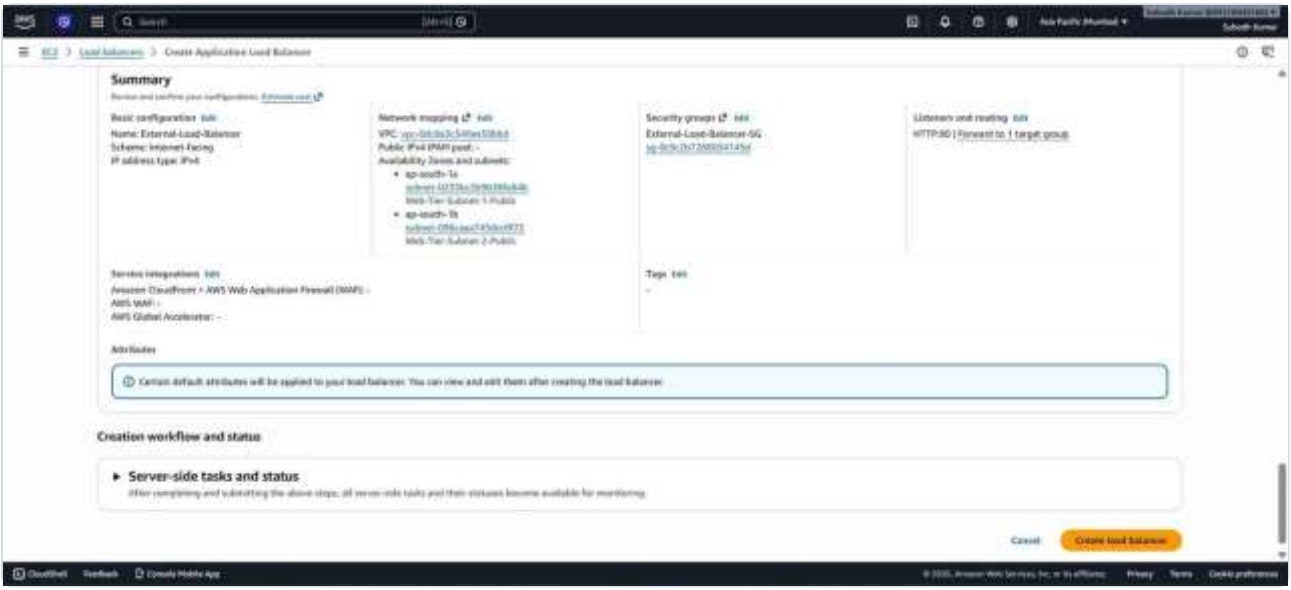



Application Load Balancer
Network Load Balancer
Gateway Load Balancer
Create
Classic Load Balancer – previous generation


Create Application Load Balancer
How Application Load Balancers work
Basic configuration
Load balancer name
Scheme
Internet-facing
Internal
Load balancer IP address type
IPv4
Dualstack
Dualstack without public IPv4


Network mapping
VPC
IP pools
Availability Zones and subnets
Subnet
Security groups


Security groups
Listeners and routing
Default action
Forward to target groups
Target group
Weight

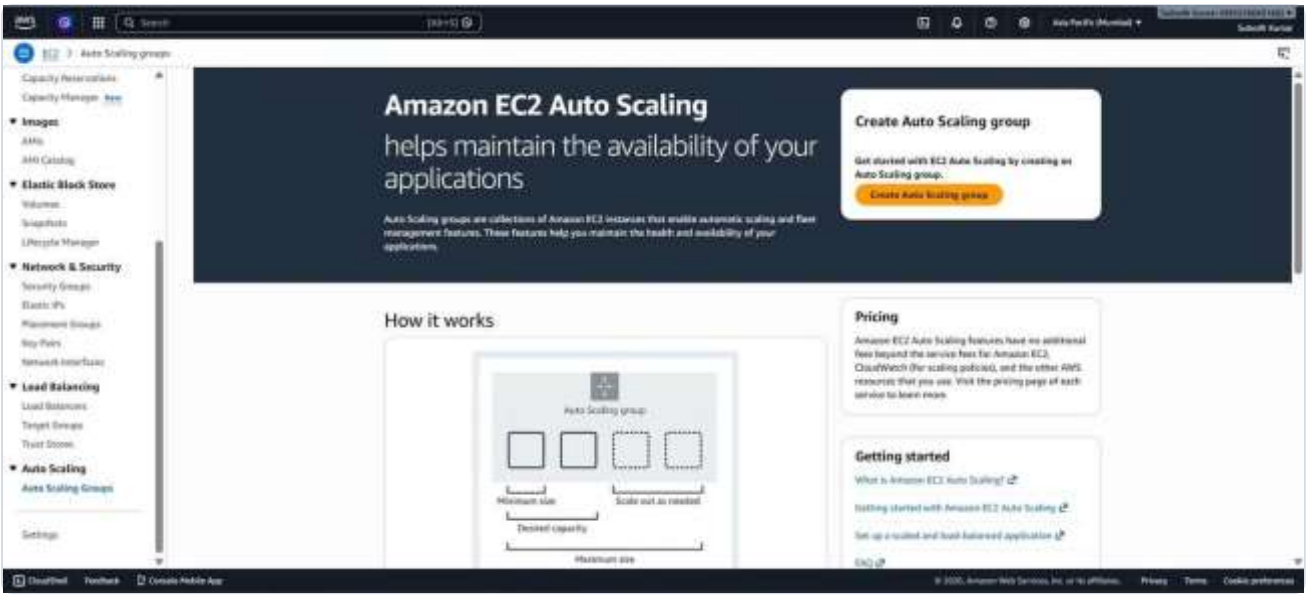

Summary
Creation workflow and status
Server-side tasks and status
Cancel
Create load balancer

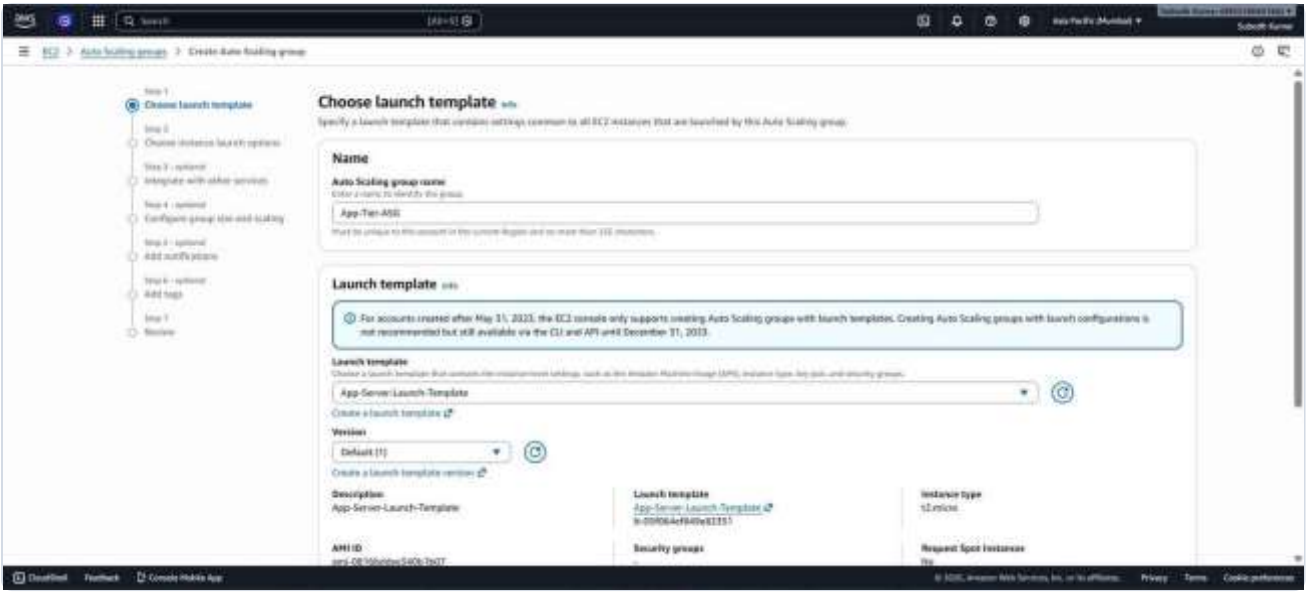
Step 13

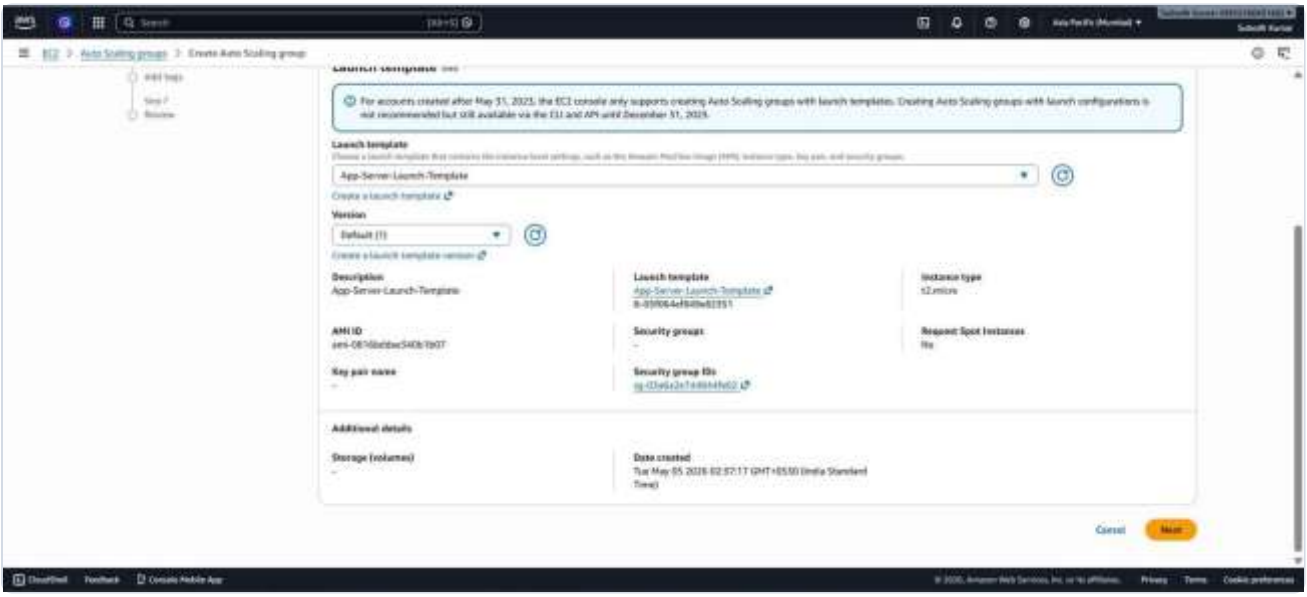
Create Auto Scaling Groups

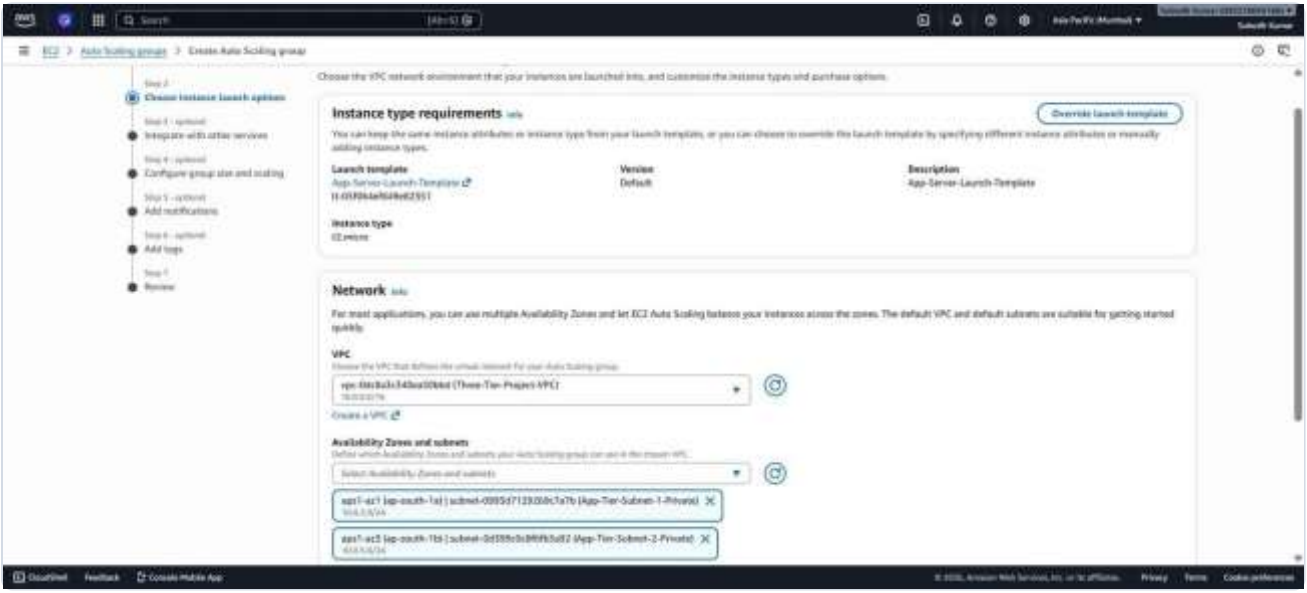
Auto Scaling Group for App Tier

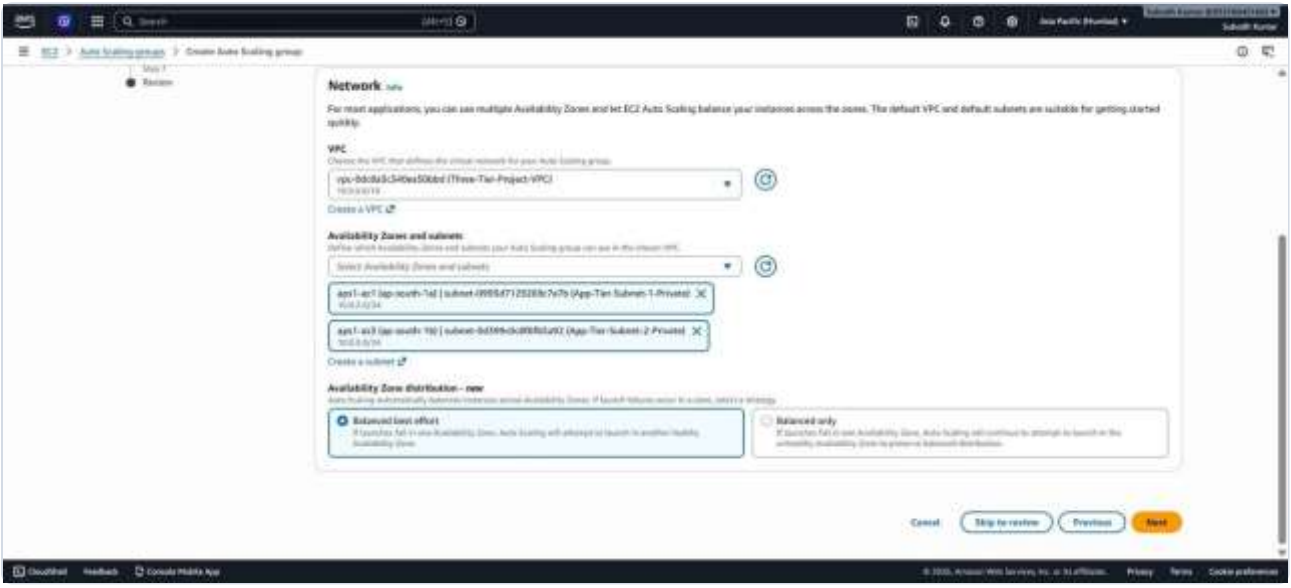
Go to EC2 > Create Auto Scaling group

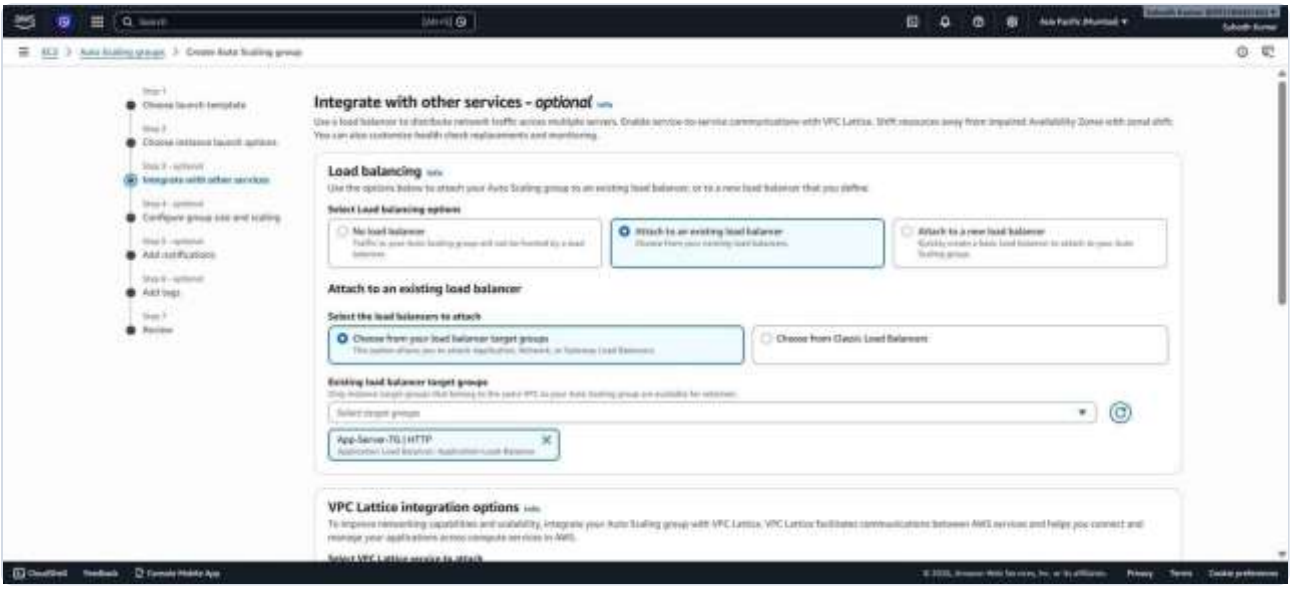


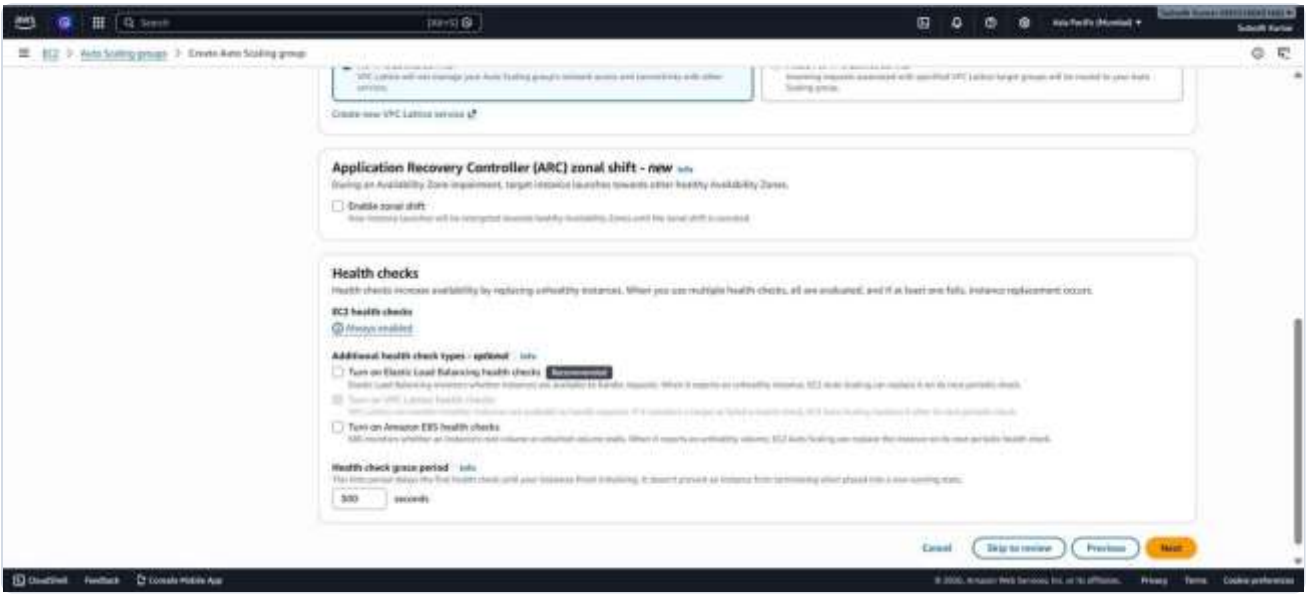


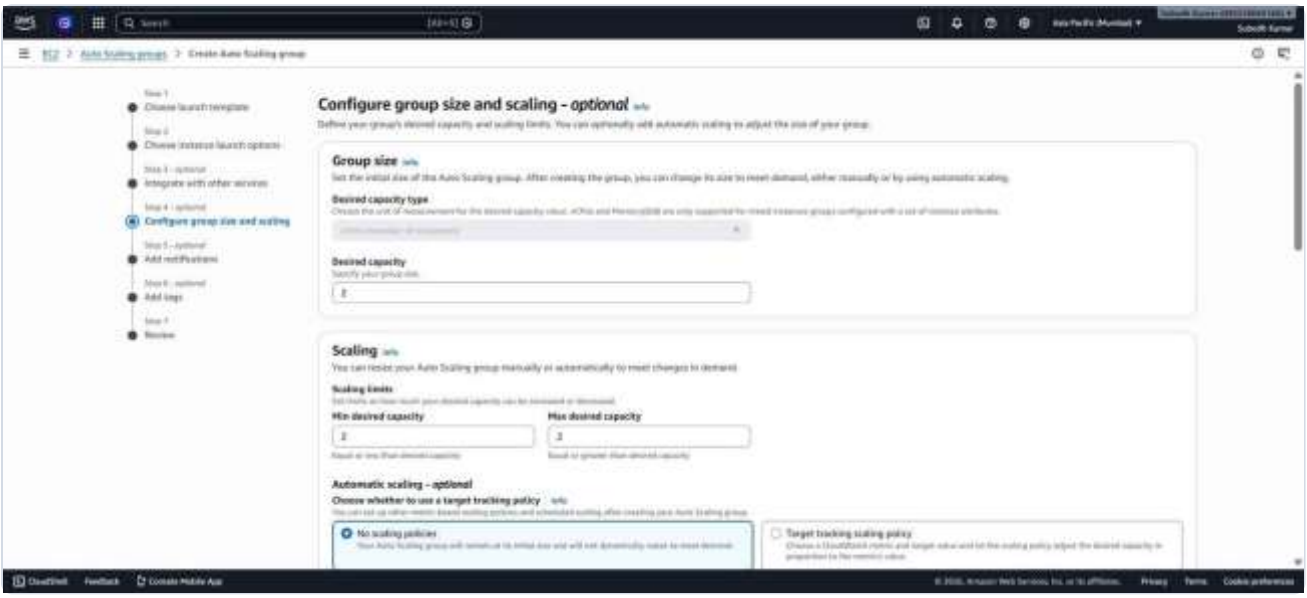


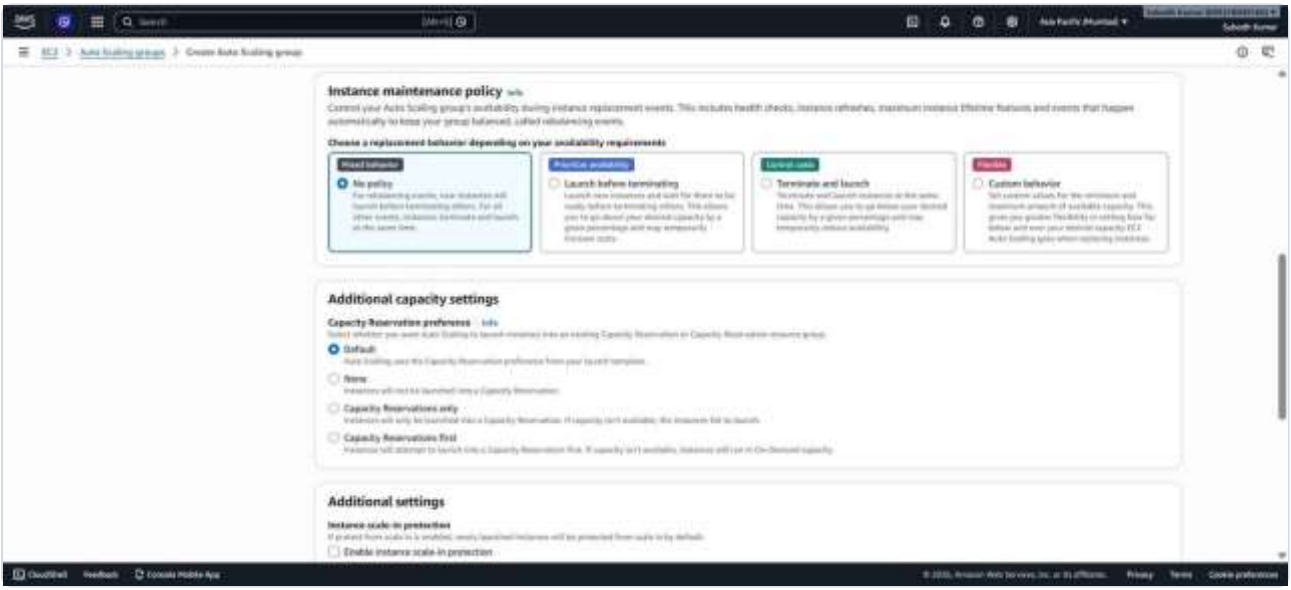


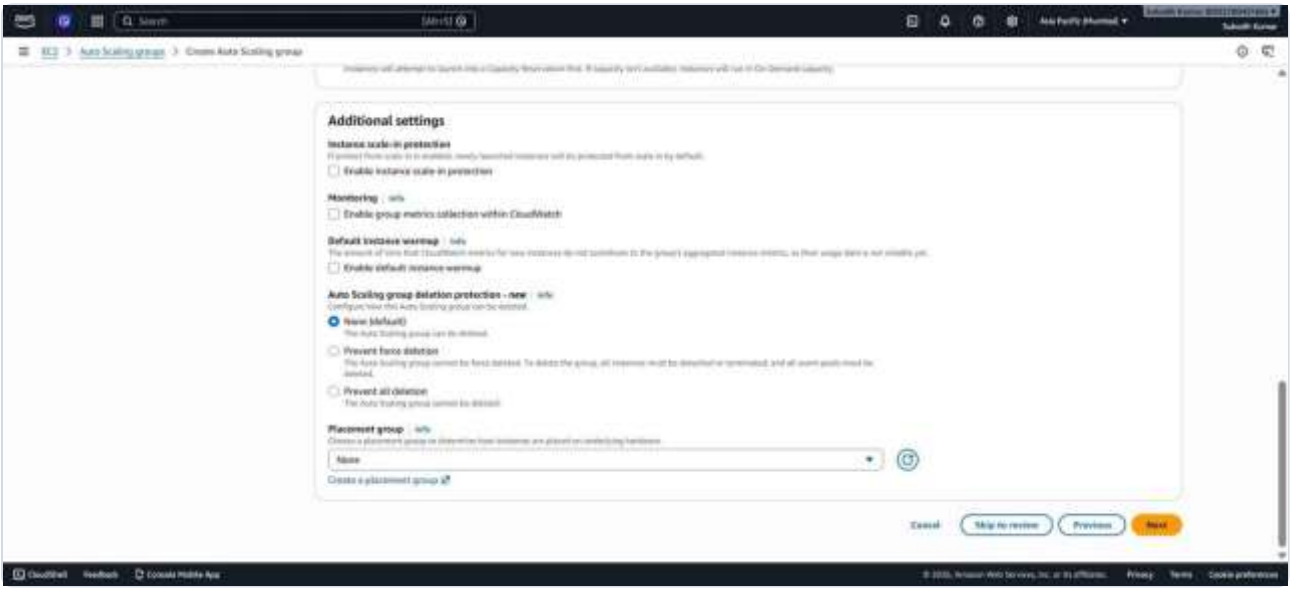


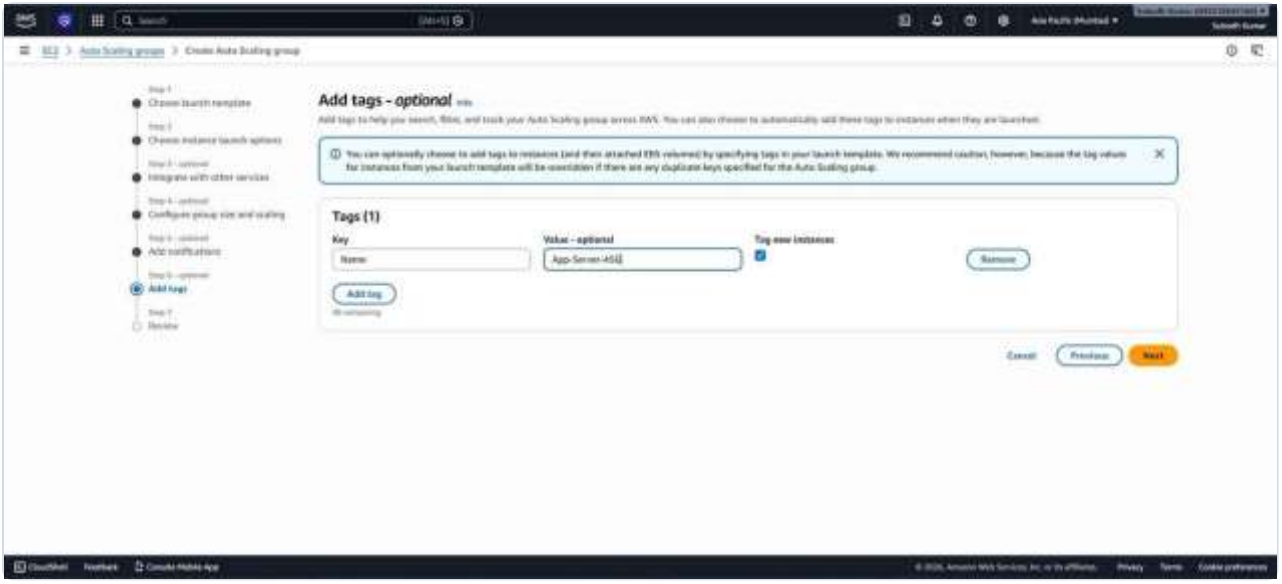

Integrate with other services - optional
Load balancing
Select Load balancing options
Attach to an existing load balancer
Select the load balancers to attach
Choose from your load balancer target groups
Choose from Classic Load Balancers
Existing load balancer target groups
App-Server-TG | HTTP
VPC Lattice integration options

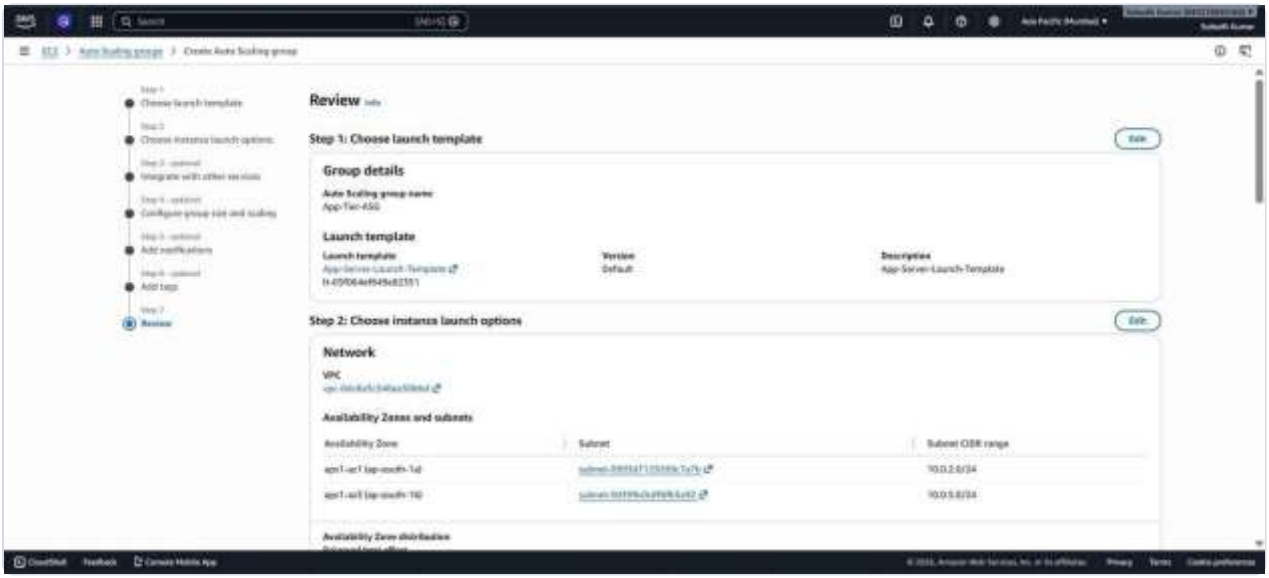

Application Recovery Controller (ARC) zonal shift - new
Enable zonal shift
Health checks
EC2 health checks
Always enabled
Additional health check types - optional
Turn on Elastic Load Balancing health checks
Turn on Amazon EBS health checks
Health check grace period
300
seconds
Cancel
Skip to review
Previous
Next

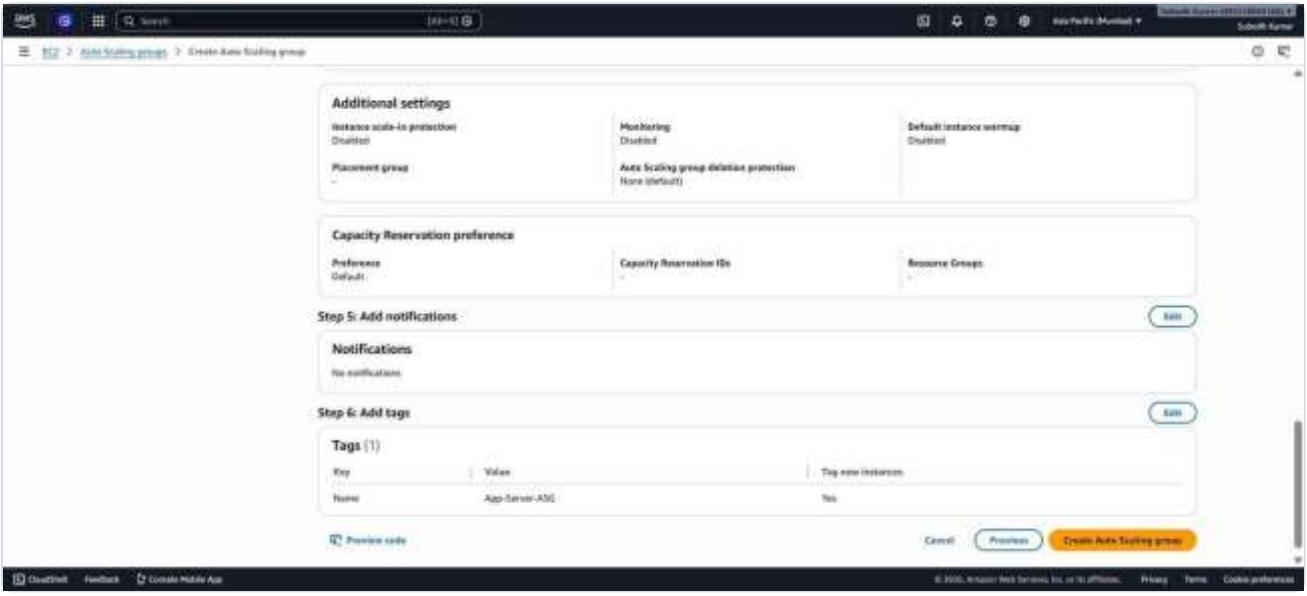

Configure group size and scaling - optional
Group size
Desired capacity type
Desired capacity
2
Scaling
Scaling limits
Min desired capacity
2
Max desired capacity
2
Automatic scaling - optional
Choose whether to use a target tracking policy
No scaling policies
Target tracking scaling policy


Instance maintenance policy
Additional capacity settings
Additional settings


Additional settings

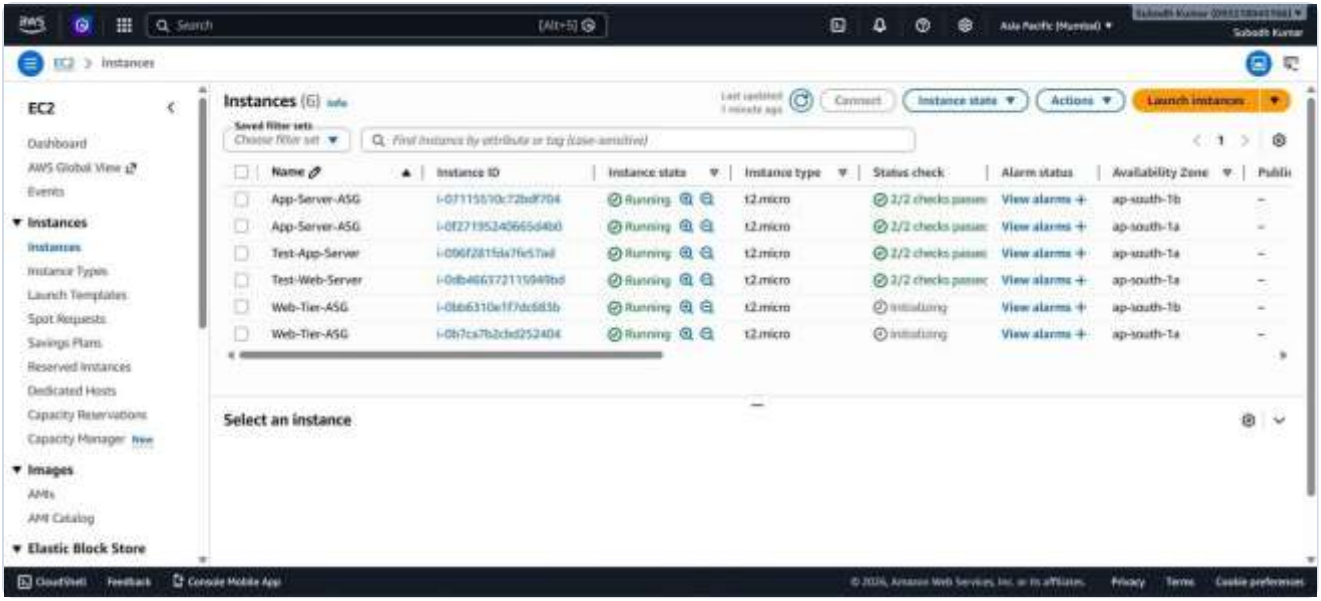

Add tags - optional
Tags (1)





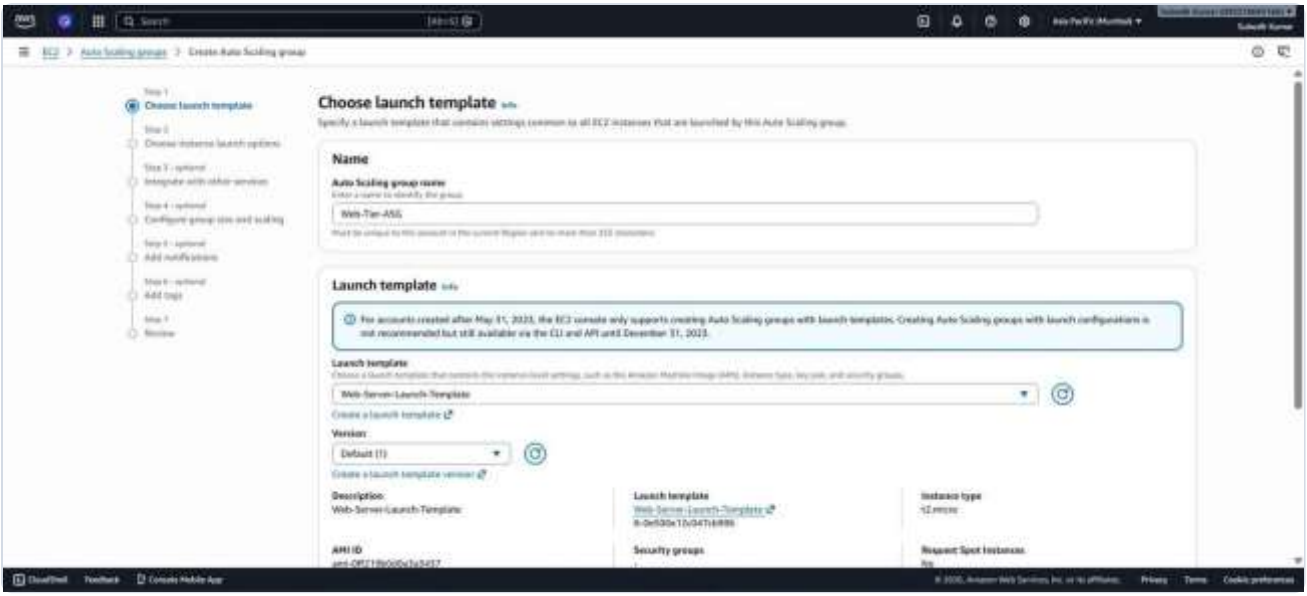
Keep other services as default and Create Auto scaling group.

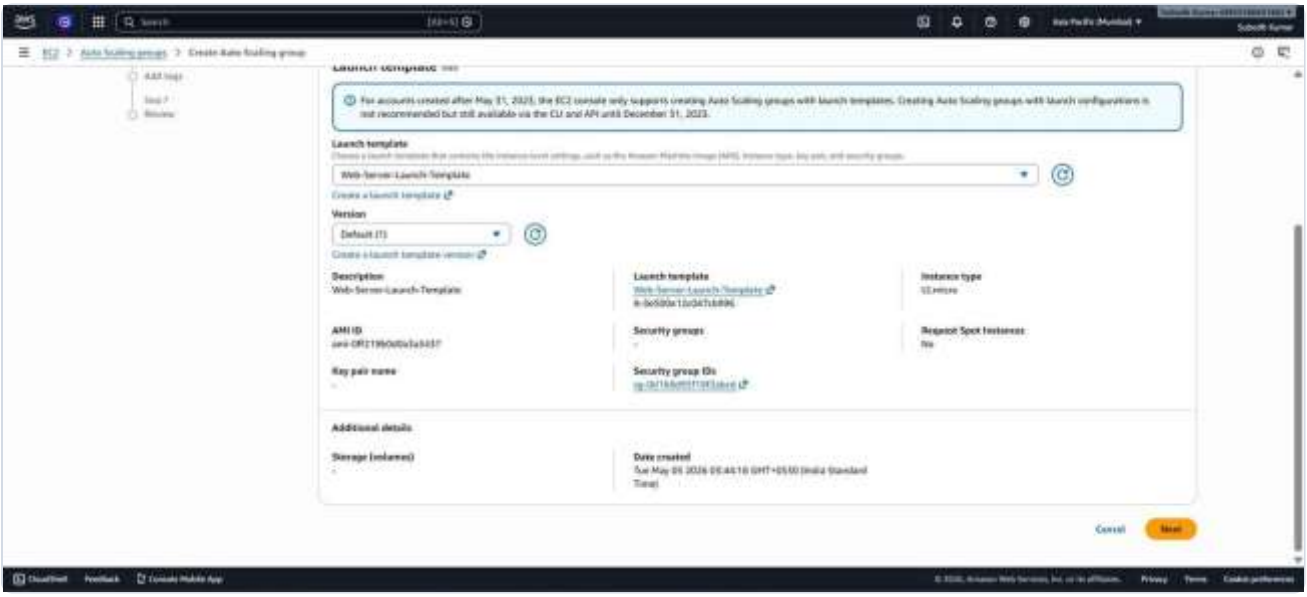
Now if you **check** the **instance list** you Can see that **two App Servers** are **creating**

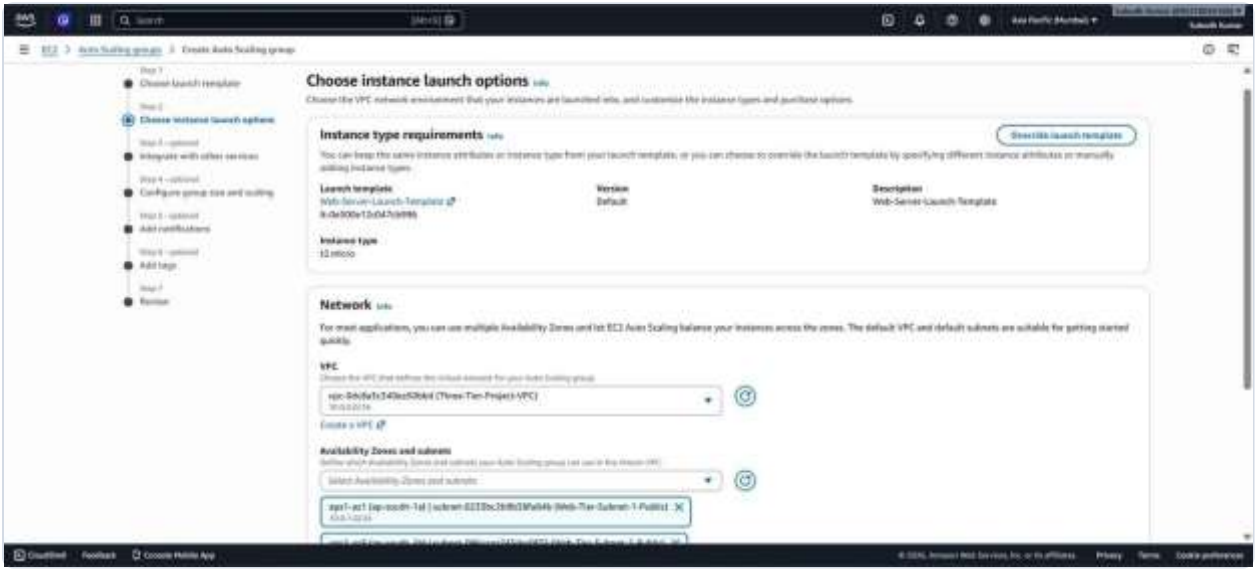


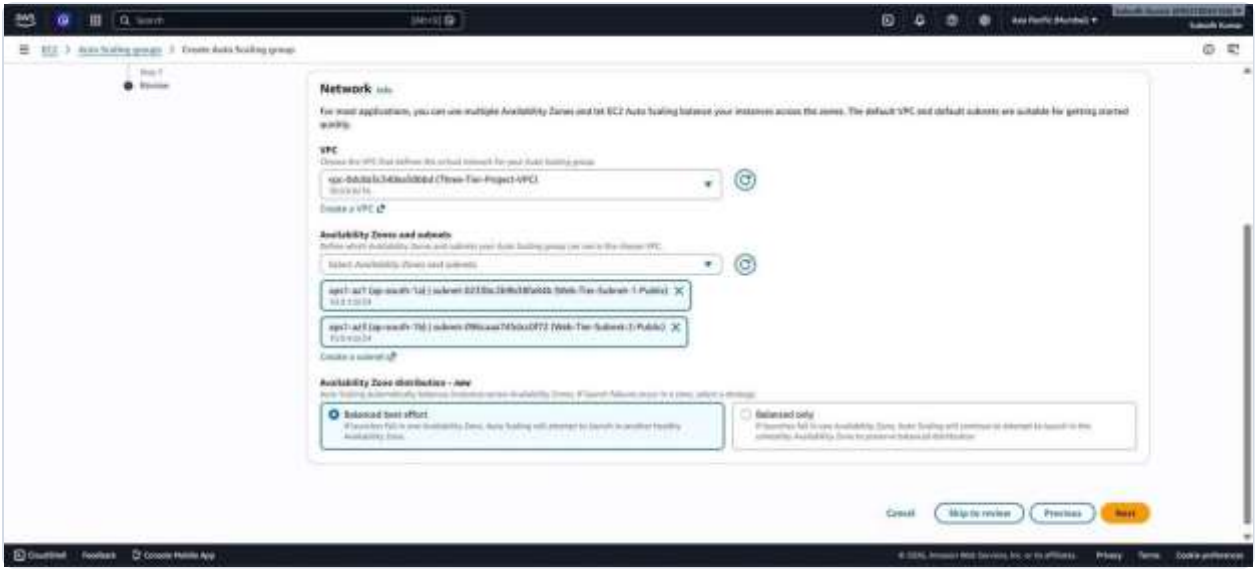
Auto Scaling Group for Web Tier

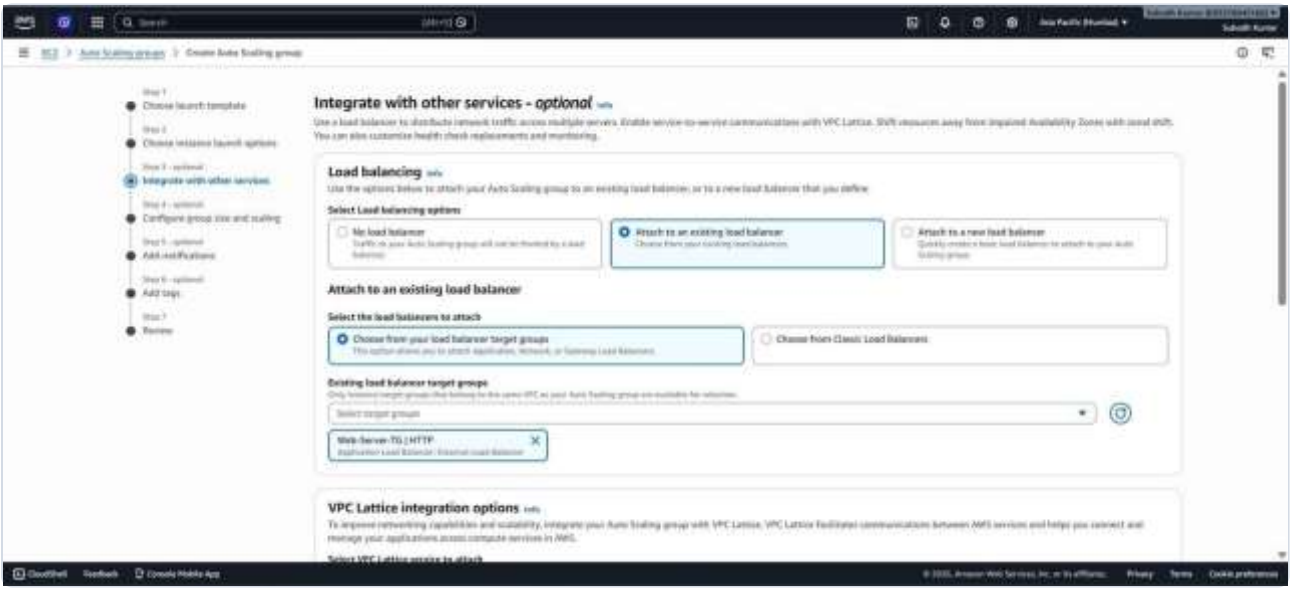
The process is the same. Choose components related to Web Tier.

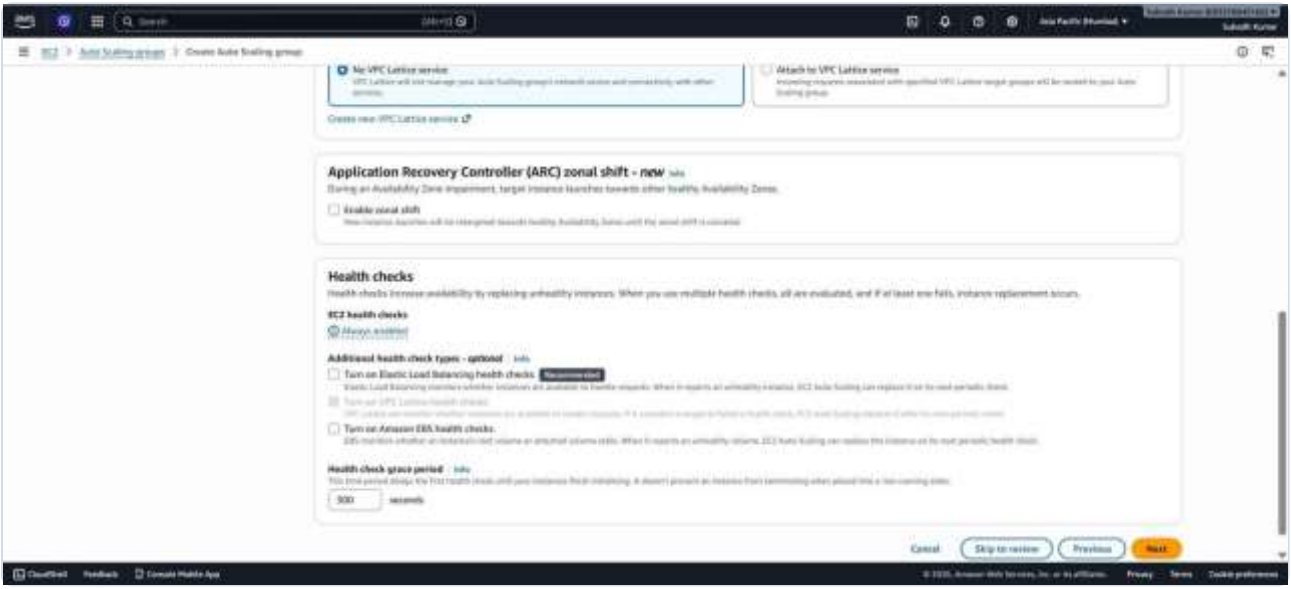


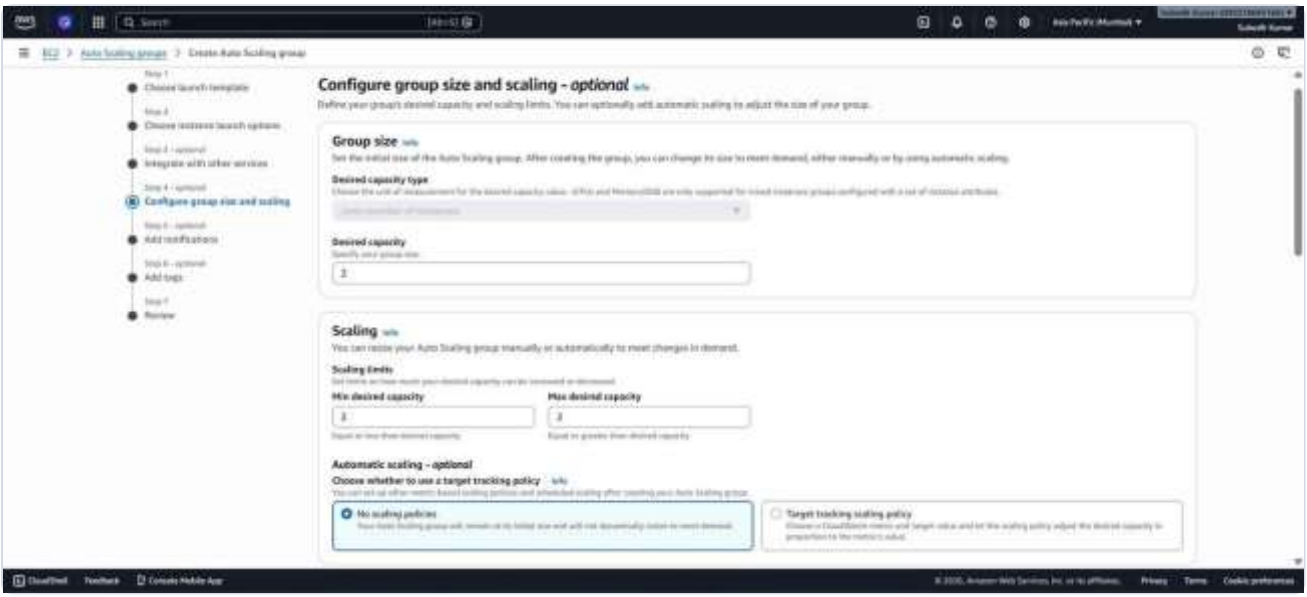

Launch template
Web-Server-Launch-Template
Version
Default (1)
Description
Web-Server-Launch-Template
Launch template
Instance type
AMI ID
Security groups
Request Spot Instances
Key pair name
Security group IDs
Additional details
Storage (volumes)
Date created
Cancel
Next

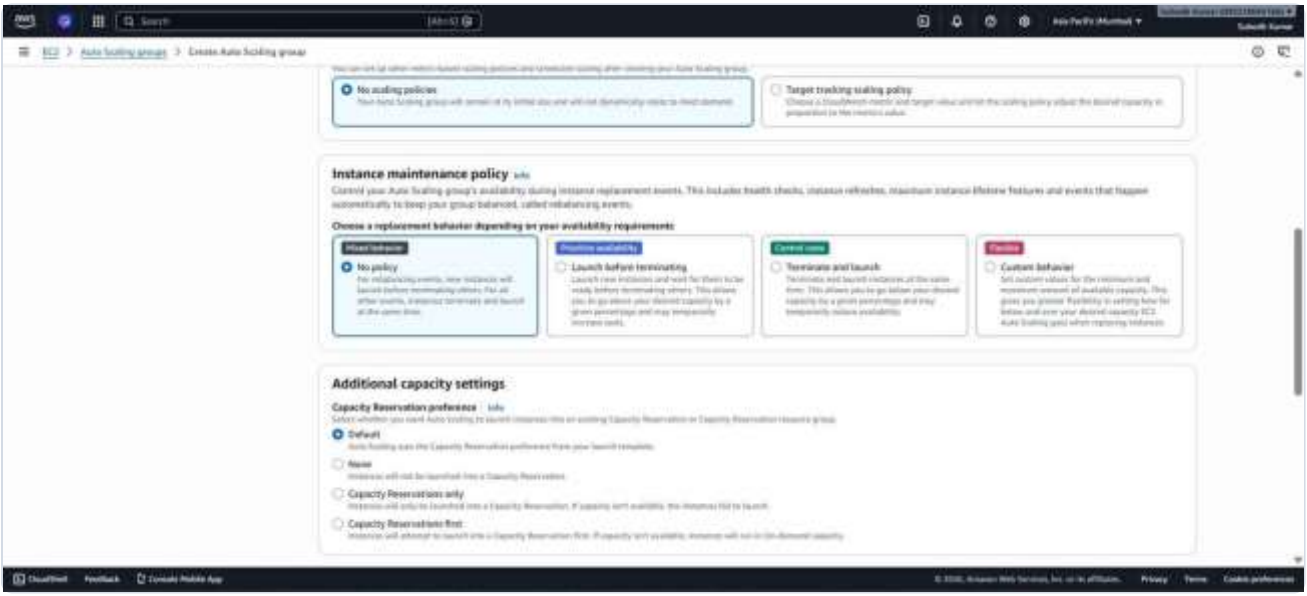

Choose instance launch options
Instance type requirements
Launch template
Web-Server-Launch-Template
Version
Default
Description
Web-Server-Launch-Template
Instance type
Network
VPC
Availability Zones and subnets

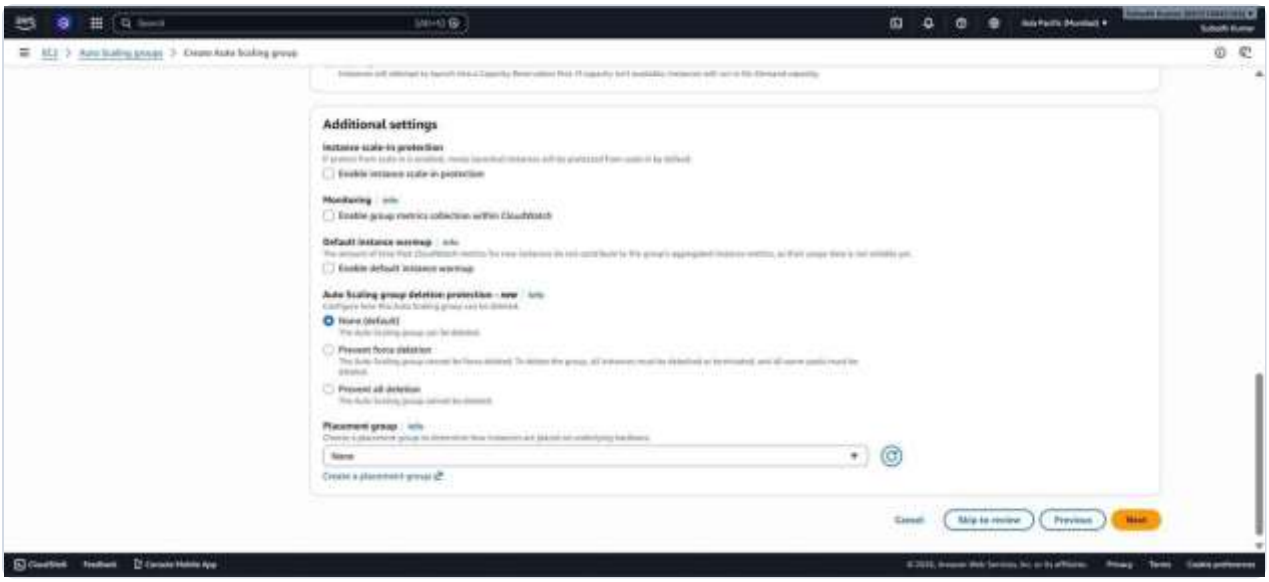

Network
VPC
Availability Zones and subnets
Availability Zone distribution - new
Cancel
Skip to review
Previous
Next

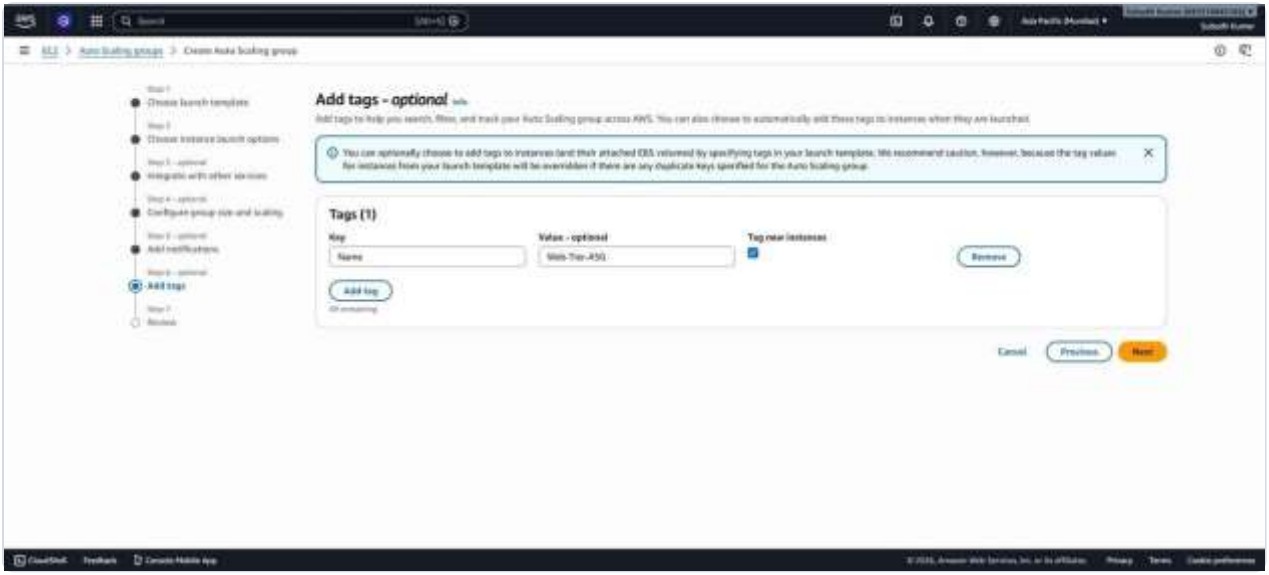

Integrate with other services - optional
Load balancing
Select Load balancing options
Attach to an existing load balancer
Attach to a new load balancer
Attach to an existing load balancer
Select the load balancers to attach
Choose from your load balancer target groups
Choose from Classic Load Balancers
Existing load balancer target groups
Web-Server-TG | HTTP
VPC Lattice integration options

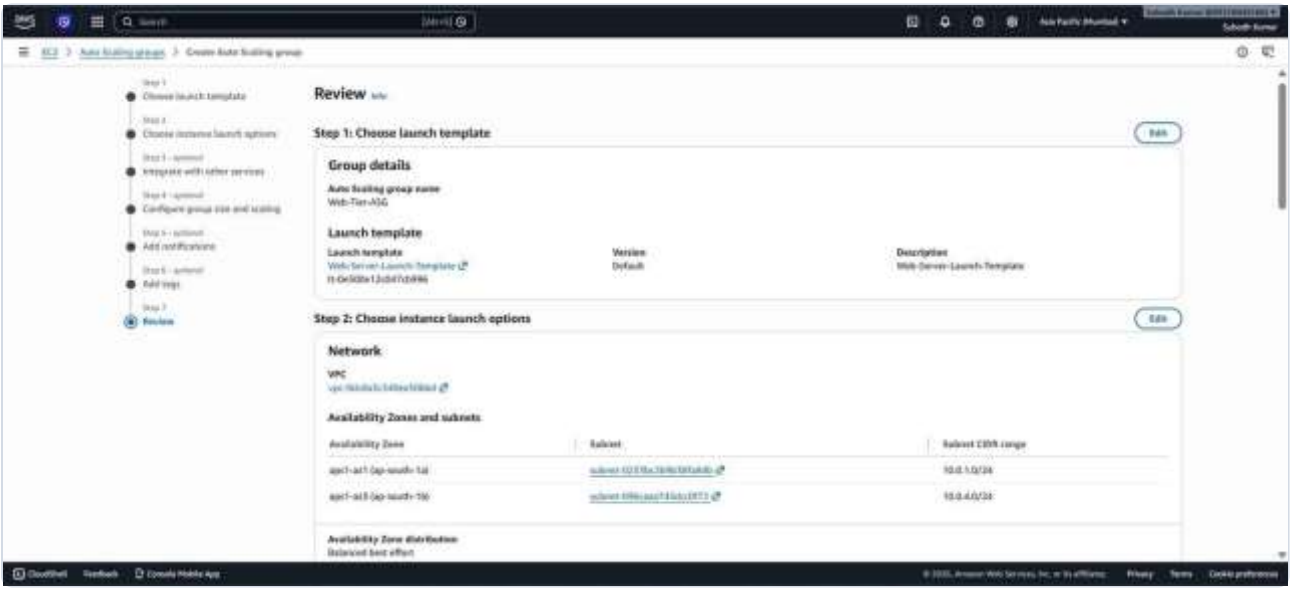

Application Recovery Controller (ARC) zonal shift - new
Enable zonal shift
Health checks
EC2 health checks
Always enabled
Additional health check types - optional
Turn on Elastic Load Balancing health checks
Turn on Amazon EBS health checks
Health check grace period
300
seconds
Cancel
Skip to review
Previous
Next

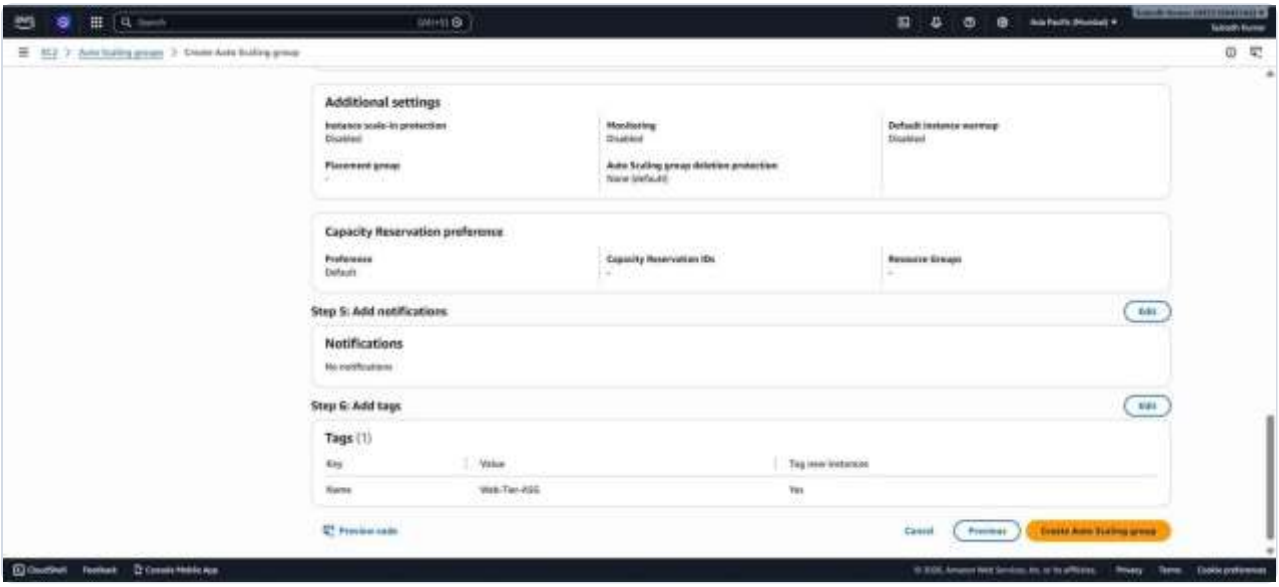

Configure group size and scaling - optional
Group size
Desired capacity type
Desired capacity
2
Scaling
Scaling limits
Min desired capacity
Max desired capacity
2
2
Automatic scaling - optional
Choose whether to use a target tracking policy
No scaling policies
Target tracking scaling policy


Instance maintenance policy
Additional capacity settings

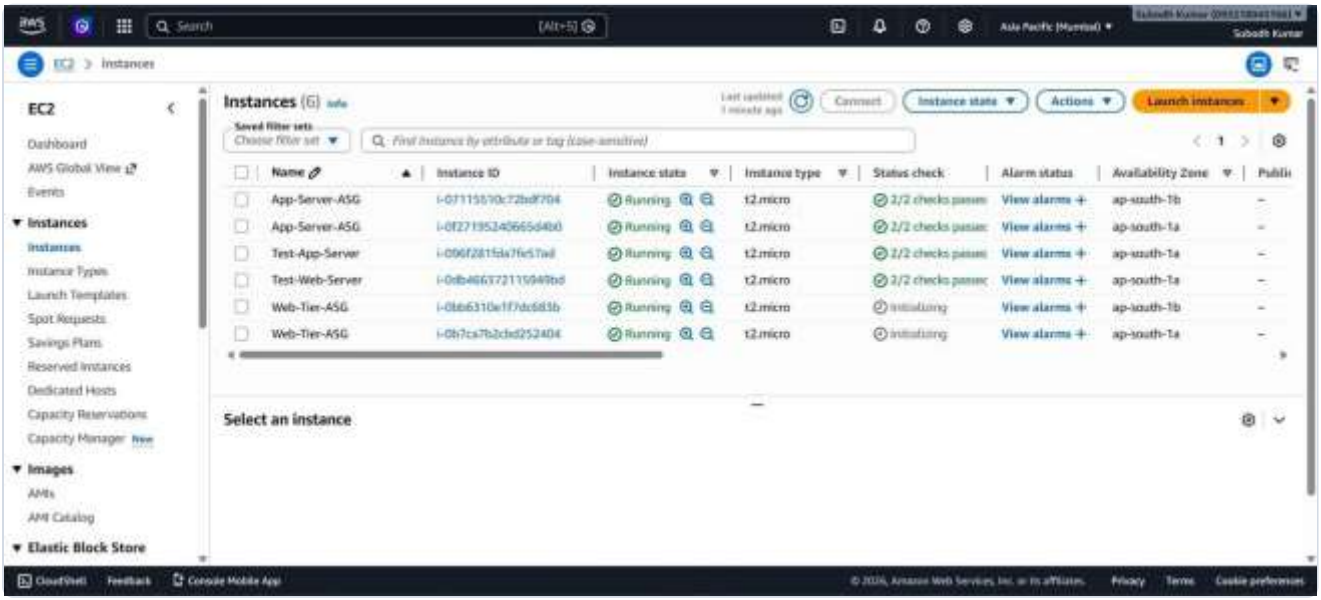

Additional settings


Add tags - optional
Tags (1)


Review
Step 1: Choose launch template
Group details
Launch template
Step 2: Choose instance launch options
Network


Additional settings
Capacity Reservation preference
Step 5: Add notifications
Notifications
Step 6: Add tags
Tags (1)

After you are creating it you can see 4 instance and test instance on the list. Now you can delete the test instances .

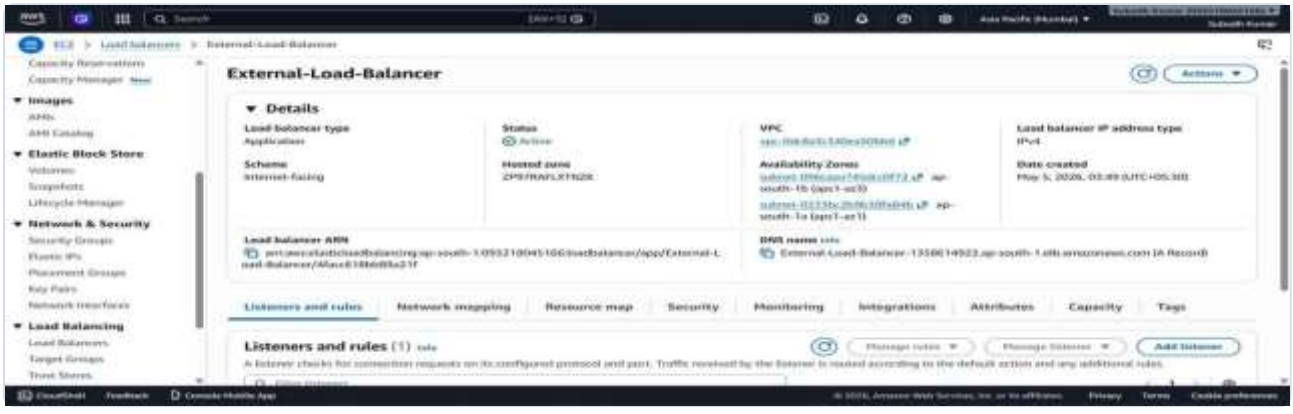


Cool You did a great job

Now We Can access our Web application.

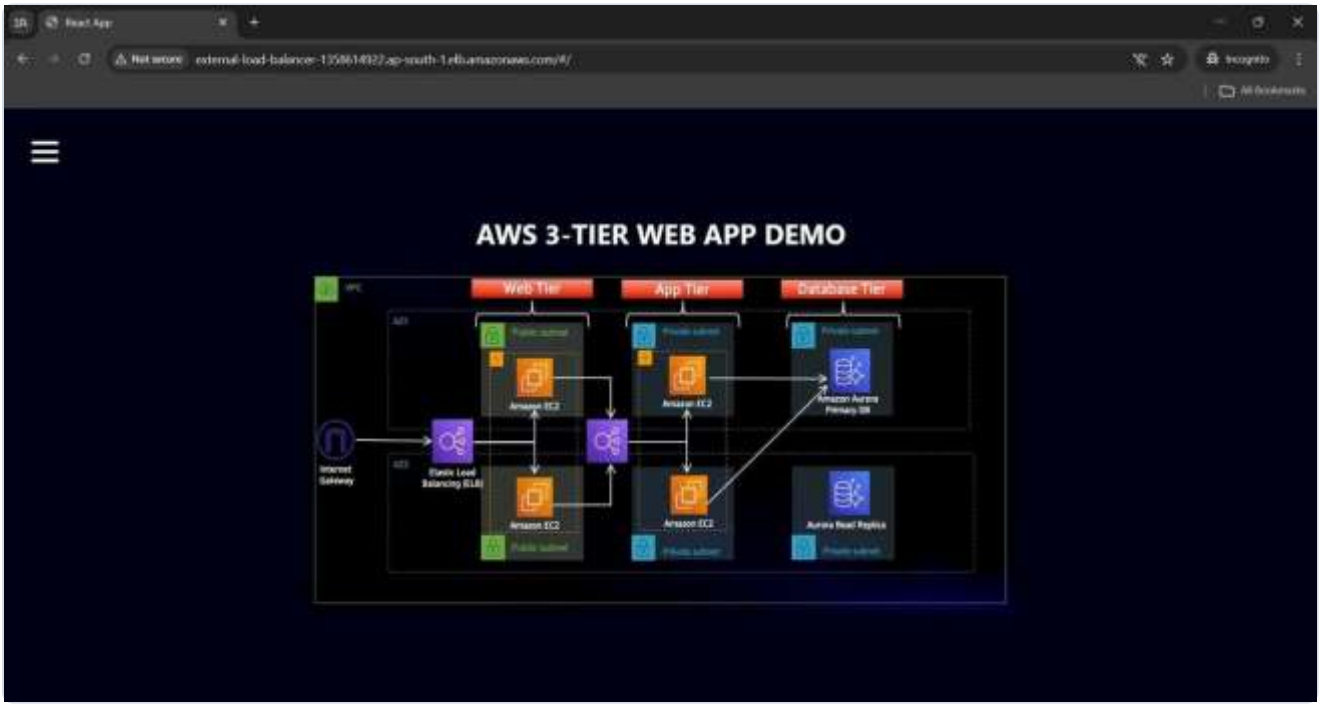
Step 14 Access the Application

Go to **EC2 > Load Balancers**. Select **External Load Balancer**. Copy the **DNS name** of it.



Paste It into your **Browser**. You will **see** the **Application** is **running**

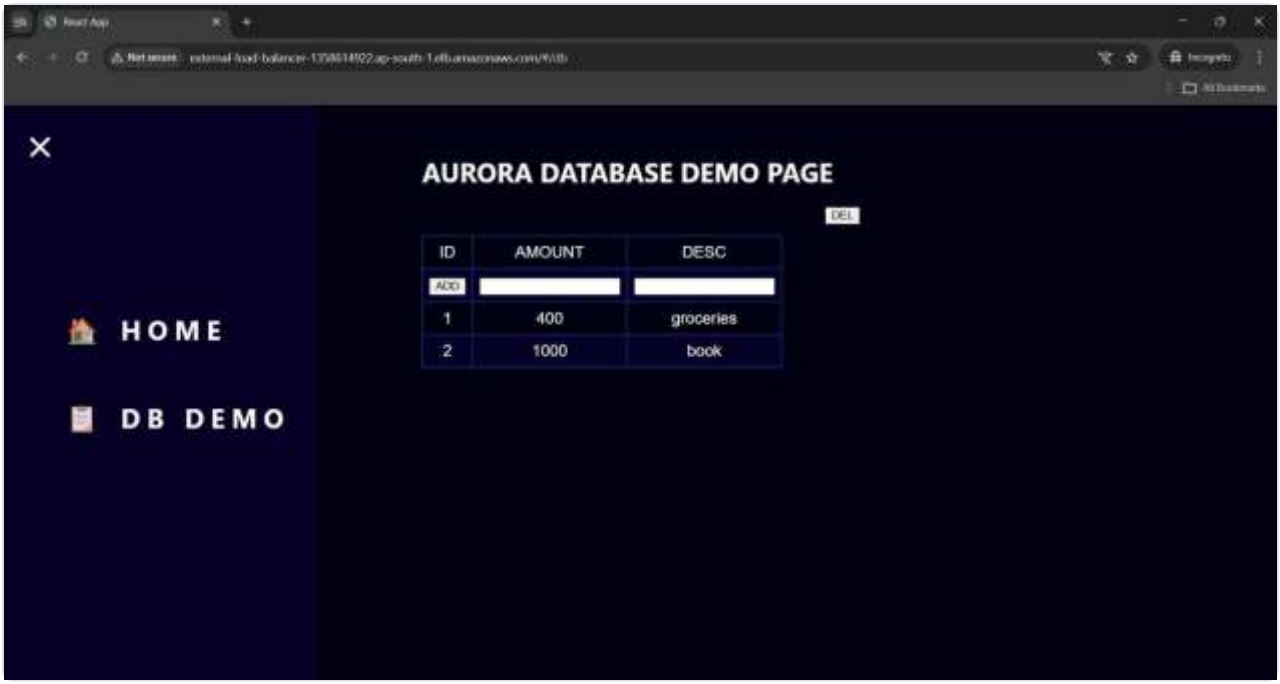
> **Note:** If the page does not open, please open it in Incognito mode.





Go to the **DB DEMO** section and check whether the DB is working.

Here you can see the example data we entered when setting up the DB, which means the application is receiving data from the DB.



You can add and delete items

Congratulations!!! You have successfully implemented Three Tier Web Application on AWS

> **Note:** In this guide we use the External Load Balancer DNS name as the URL, but it is not user friendly. We can purchase a domain name from a domain name service and set it up using **Route 53** to access the website with that domain name.

Step 15 Cleaning the Infrastructure

Since some services we used charge cost we have to delete our demo project.

Let's **delete** them according to the following order.

1. Auto Scaling Groups
2. In Application Load Balancers delete listeners first and then Load balancers
3. Target Groups
4. Launch Templates
5. AMIs
6. Snapshots
7. EC2 Instances
8. Aurora Database (First go to modify and disable delete protection. Then when deleting do not create snapshot. Tick I acknowledge)
9. NAT gateways
10. Elastic Ips
11. Route Tables (Remove all subnet associations first)
12. Internet Gateways
13. VPC
14. S3 Bucket
15. IAM Role

THANK YOU!

Three-Tier Web Application on AWS

Highly Available · Secure · Scalable

EC2	S3	VPC	ALB
Compute	**Source code**	**Network**	**Traffic**
ASG	RDS	IAM	NAT
Scaling	**Database**	**Access**	**Private egress**

PROJECT SUMMARY

Two Availability Zones · Public web tier · Private app and database tiers

External and internal load balancers · Launch Templates · Auto Scaling Groups

Source code repository: github.com/dhruvsonwane13-ship-it/Three-tier-project

KEEP LEARNING · KEEP BUILDING · KEEP GROWING!